



The Inner Workings of Shim

Analyzing with AI Assistance

SUSE Labs Conference 2026, České Budějovice

Joey Lee <jlee@suse.com>

Agenda

1. Source Code Analysis with AI
2. Analyzing shim with AI Assistance
 - Protocol, Load and Boot
 - MOK
 - MokManager
 - Fallback
 - SBAT
 - TPM

Source Code Analysis with AI





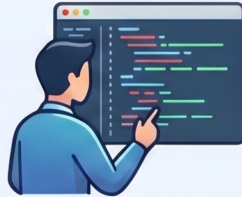


Image generated by Gemini AI

Source Code Analysis Process

WORKFLOW A: MY ORIGINAL SOURCE CODE REVIEW PROCESS

1. REVIEW SOURCE CODE



2. PRODUCE REVIEW NOTE

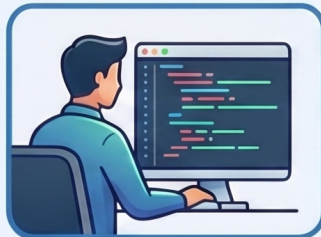


3. DRAW DIAGRAM



WORKFLOW B: PROCESS WITH AI ASSISTANCE

1. REVIEW
SOURCE CODE

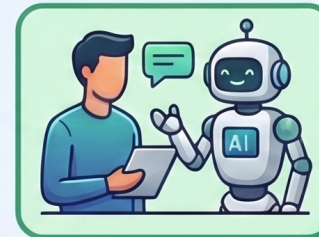


2. PRODUCE
REVIEW NOTE



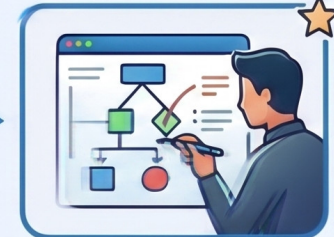
[AI SCANS
SOURCE CODE AND
PRODUCE REPORT]

3. PRODUCE
DIAGRAMS



[AI PRODUCE
DIAGRAM AND
PROMPT]

4. REVIEW AND
COMMENT DIAGRAMS



Source Code Analysis with Gemini

— Gemini CLI

- Scan source code, produce report (Source code files functionality integration, hierarchical call analysis)
- Produce diagram (Block diagram, Flow Chat)
 - DOT language (txt), GraphvizOnline
 - .dot → .svg / .png
 - dot -Tsvg xxx.dot -o xxx.svg
 - (Feed .dot file back to Gemini CLI for reproducing the style of diagram)
- - Produce prompt (for reproducing diagrams)

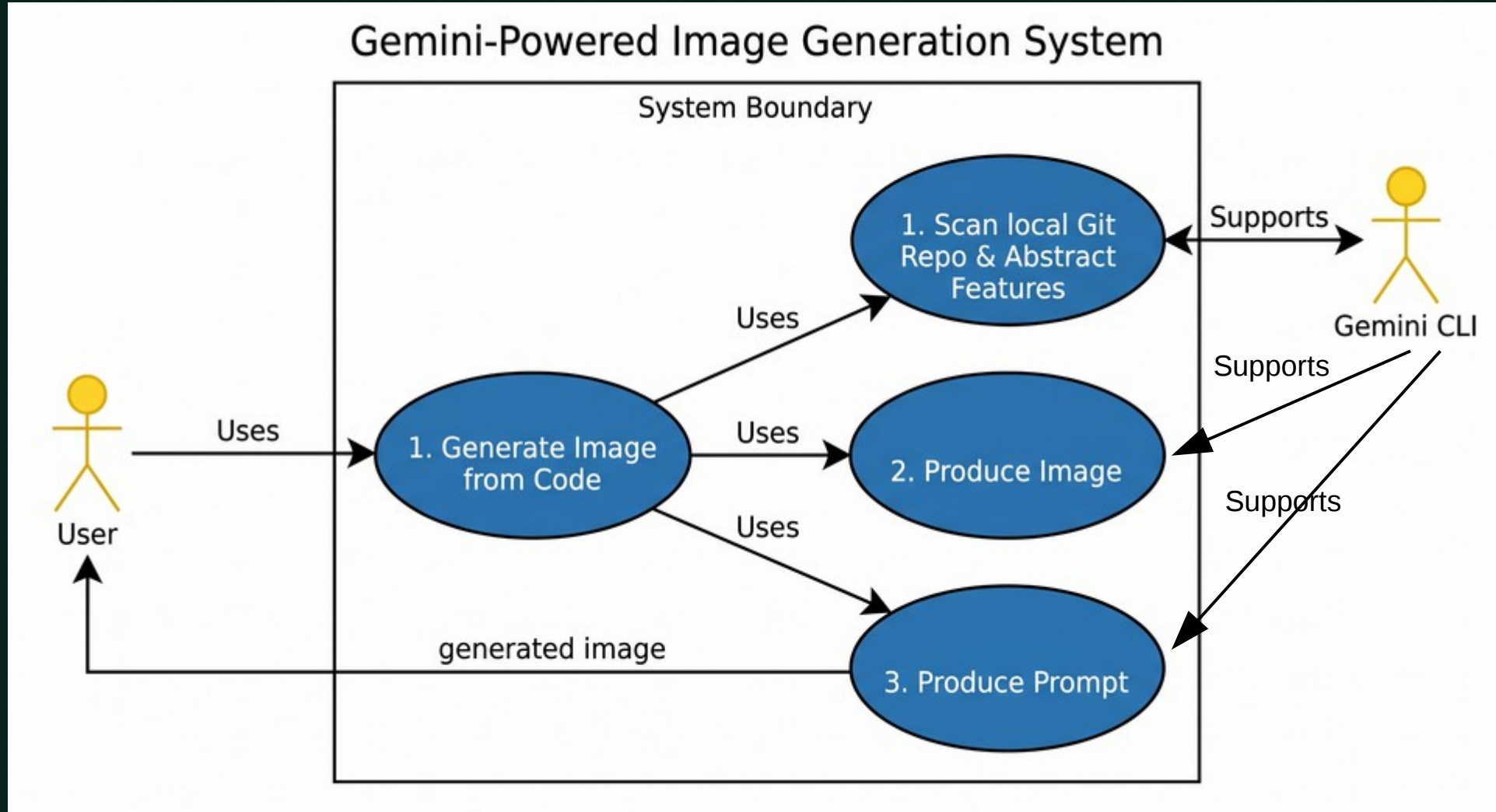
— Gemini

- - Produce diagrams (nano banana 2)

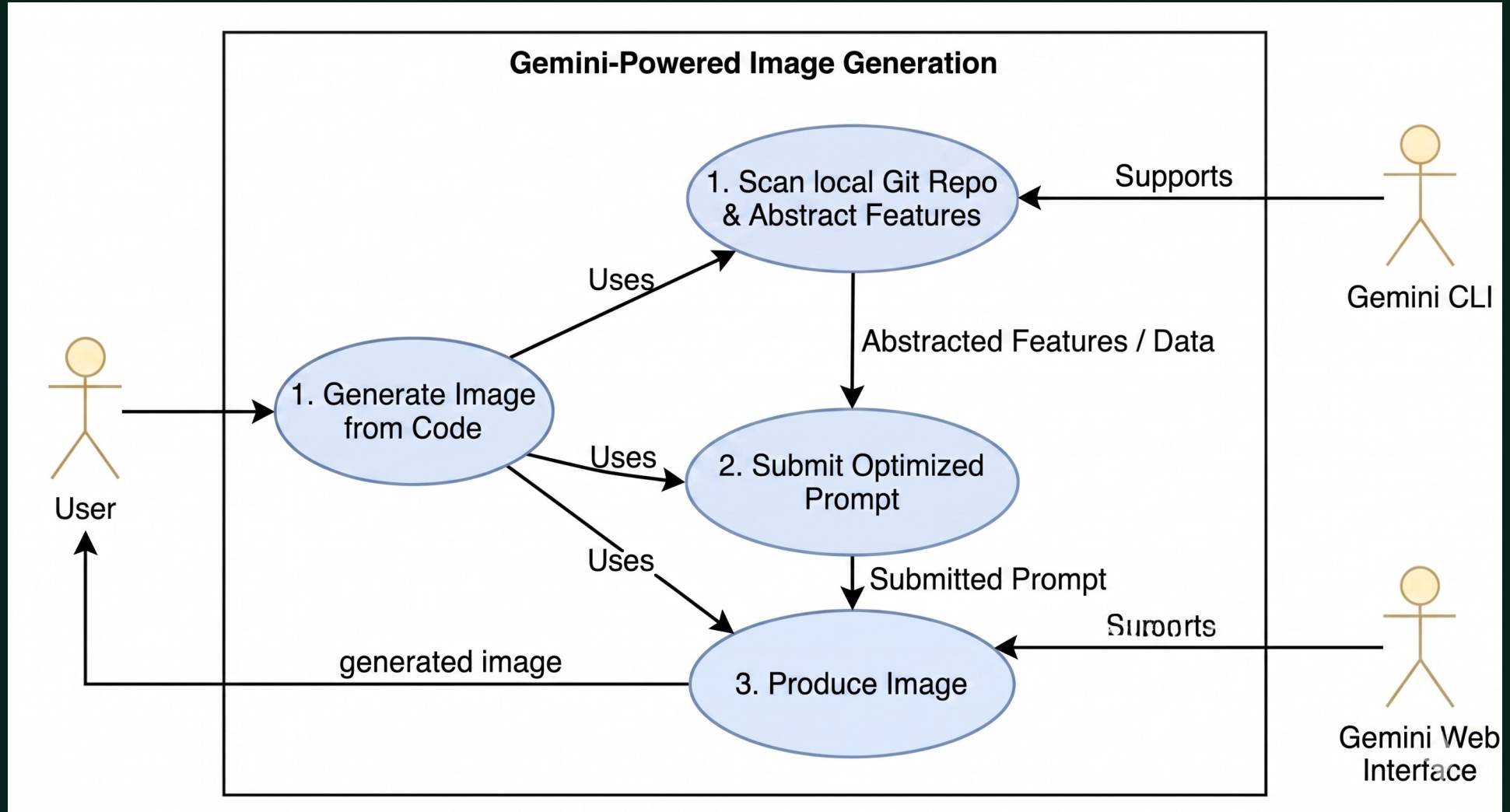
Unexpected behavior of Gemini

- Gemini CLI
 - Very long thinking (unlimited?) in some cases
- Gemini (nano banana 2)
 - Sometimes no picture output
 - Can not reproduce picture
 - Lost word or some details after modified picture

Use Cases: Gemini-Generated Image from Code



Use Cases: Gemini-Generated Image from Code (cont.)



Simple Summary for Source Code Analysis with AI

— Pros:

- Fast Iteration: Rapidly generates draft reports or baseline diagrams to accelerate the code review process.

— Cons:

- Context Switching & Distraction: Diverts focus from deep code tracing into time-consuming alignment and troubleshooting with the AI.
- Unpredictable Output & Editing Overhead

— Simple Principle: Communication Over Perfection

- Let AI Go: If the output is good enough, accept the result and comment it.
- Manual Intervention: Fill the gap manually rather than getting trapped in endless prompting loops.

Analyzing shim with AI Assistance



Analyzing shim with AI Assistance

— Why shim?

- Open source
 - License: BSD-2-Clause-Patent
- Not too big
- stable (currently)
- It falls under my work area.

— Why analyze?

- Understand the details of existing source code.

— Target: Shim 16.1

- These AI diagrams have been reviewed and commented on by human based on the source code.

Diagrams in this slides

— <https://github.com/joeyli/shim-analyze/>

- shim-16.1 branch

<https://github.com/joeyli/shim-analyze/blob/shim-16.1/The-Inner-Workings-of-Shim-20260519.pdf>

Source Code Review Note with Gemini CLI Assistance

```
## Root Directory Files

[build]
* 'Make.defaults': Contains default variable definitions for the build system.
    包含建置系統的預設變數定義。 [build]
* 'Make.rules': Defines common rules and patterns for the make-based build system.
    為基於 make 的建置系統定義通用規則和模式。 [build]
* 'Makefile': The main makefile that orchestrates the entire build process.
    負責協調整個建置過程的主 Makefile 檔案。 [build]
* 'elf_*.lds': Linker scripts for various architectures (aarch64, arm, ia32, ia64, x86_64) that control how the executable is laid out in memory
    適用於各種架構 (aarch64、arm、ia32、ia64、x86_64) 的連結器腳本，用於控制可執行檔在記憶體中的佈局方式。 [build]
* 'model.c': Part of the fuzzing infrastructure. [build check?]
    模糊測試基礎設施的一部分。 <-- 錯! modeling file for coverity Coverity 的建模文件 71bc0a8c86e2e5234d99c653700438bc4e4455c4 Ad
erity.
    This is useful to hide some false positives from the covscan results. We never build it

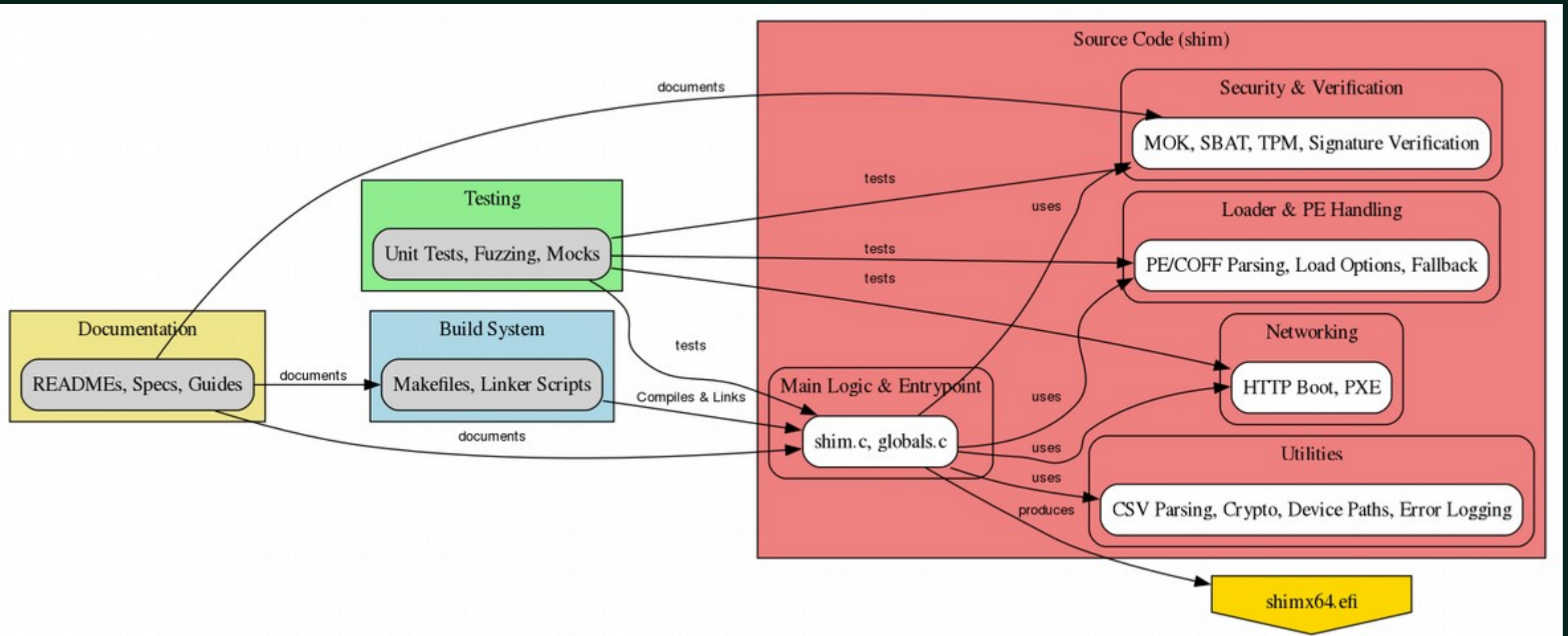
[src]
* '.clang-format': Configuration file for the clang-format code formatter, ensuring consistent code style.
    clang-format 程式碼格式化程式的設定文件，確保程式碼風格的一致性。 [src]
* '.gitignore': Specifies files and directories that Git should ignore, such as build artifacts.
    指定 Git 應該忽略的檔案和目錄，例如建置產物。 [src]
* '.gitmodules': Defines submodules used in the project, like 'gnu-efi'.
    定義專案中使用的子模組，例如 'gnu-efi'。 [src]

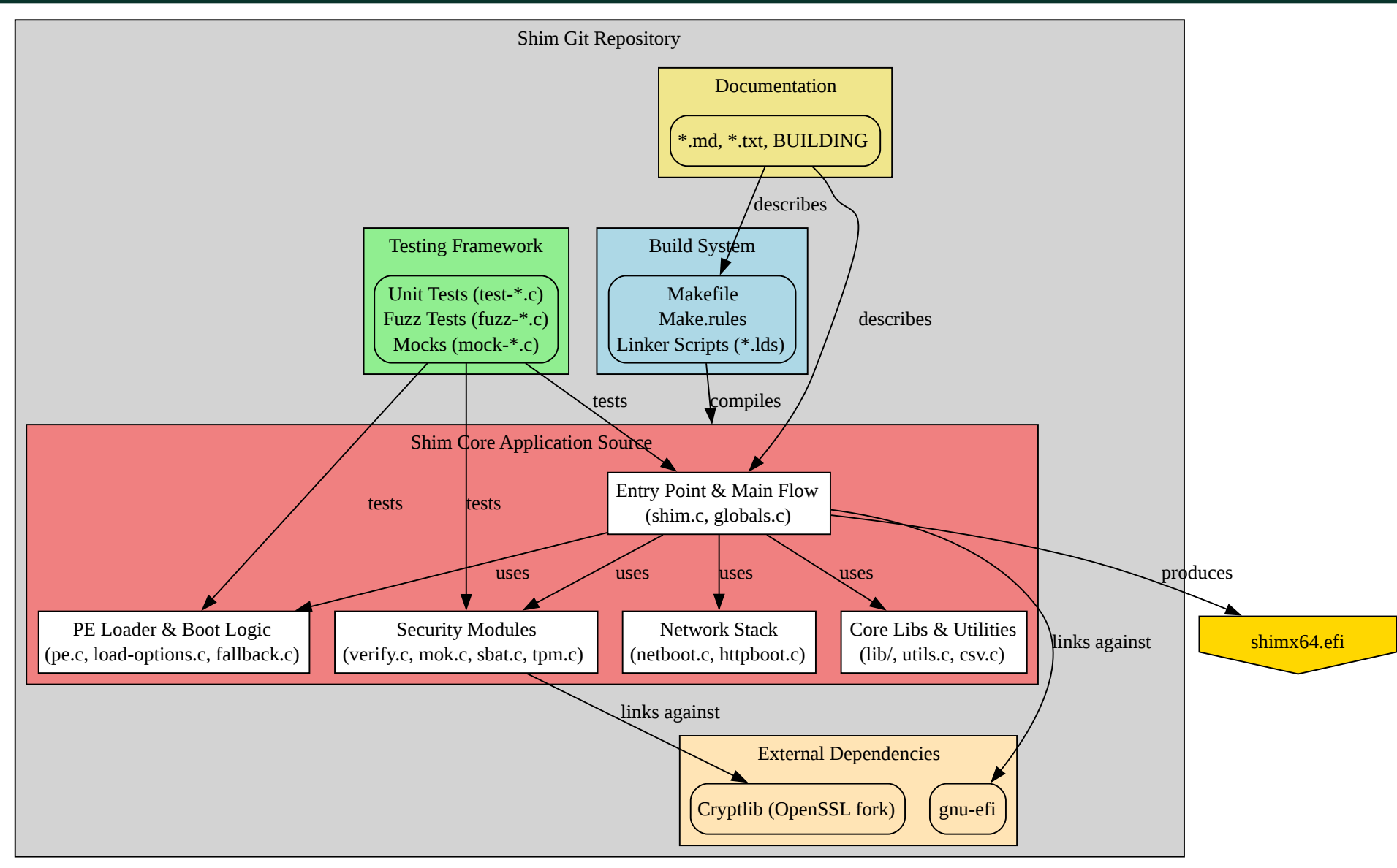
* 'builddid.c': A small utility to embed a unique build ID into the final binary for identification.
    一個用於將唯一建置 ID 嵌入到最終二進位檔案中以進行識別的小工具。 [src, debug] <-- for debugging? yes 可以講講
* 'errlog.c': Implements error logging functionality for debugging.
    實作了錯誤日誌記錄功能，方便除錯。 [src, debug log]

* 'csv.c': Utility functions for parsing CSV (Comma Separated Values) files, primarily used for handling SBAT data.
    用於解析 CSV (逗號分隔值) 檔案的實用函數，主要用於處理 SBAT 資料。 [src, sbat]
* 'generate_sbat_var_defs.c': A utility that generates C header files from SBAT CSV data.
    一個可以根據 SBAT CSV 資料產生 C 頭檔的工具程式。 [src, sbat] <-- 可以講講
* 'sbat_var.S': Assembly code related to SBAT UEFI variables.
    與 SBAT UEFI 變數相關的彙編程式碼。 [src, sbat] <-- 主要放入 SBAT ASCII payload, 可以用 objdump 察看
* 'sbat.c': The core implementation of the SBAT (Secure Boot Advanced Targeting) revocation mechanism.

[lines 1-35]
```

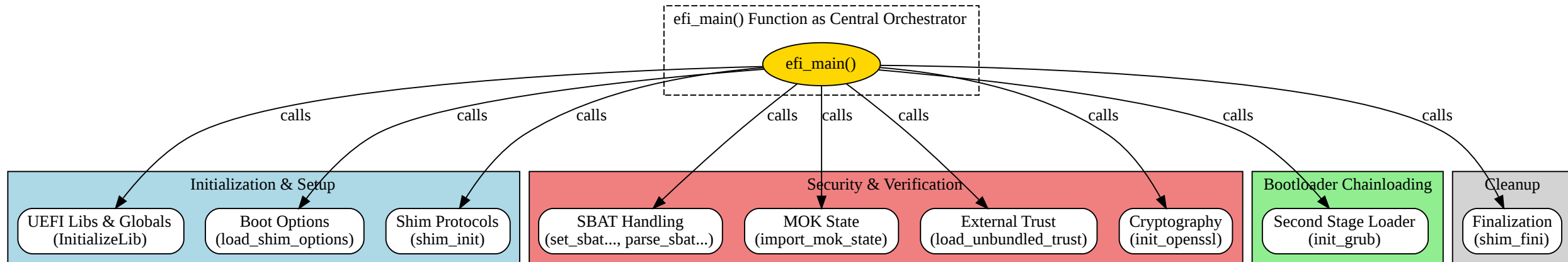
Block diagram from Gemini scan result



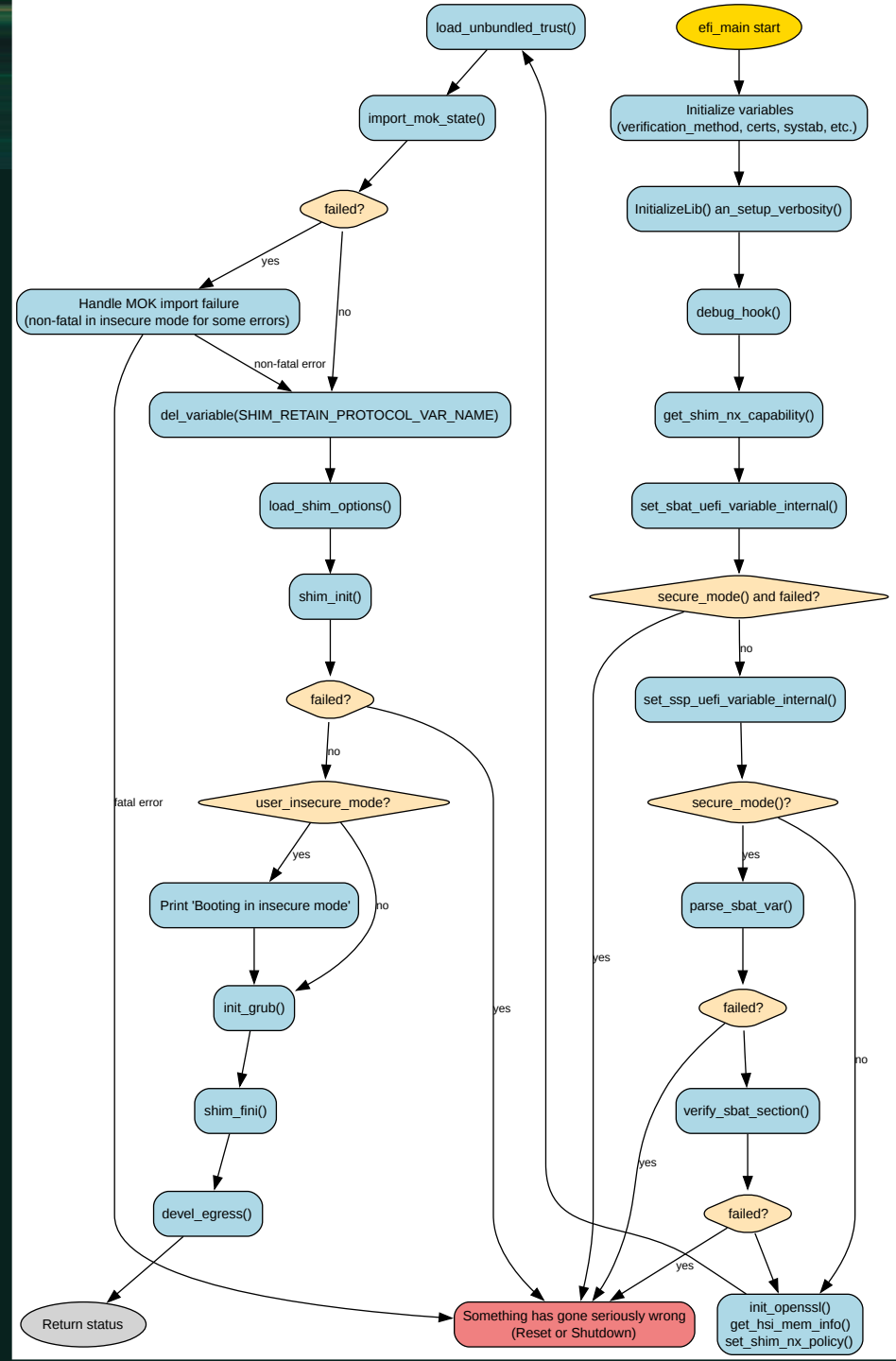


Block diagram from Gemini scans efi_main()

- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the **efi_main()** function and its interactions."



Flow Chart from Gemini scans efi_main()

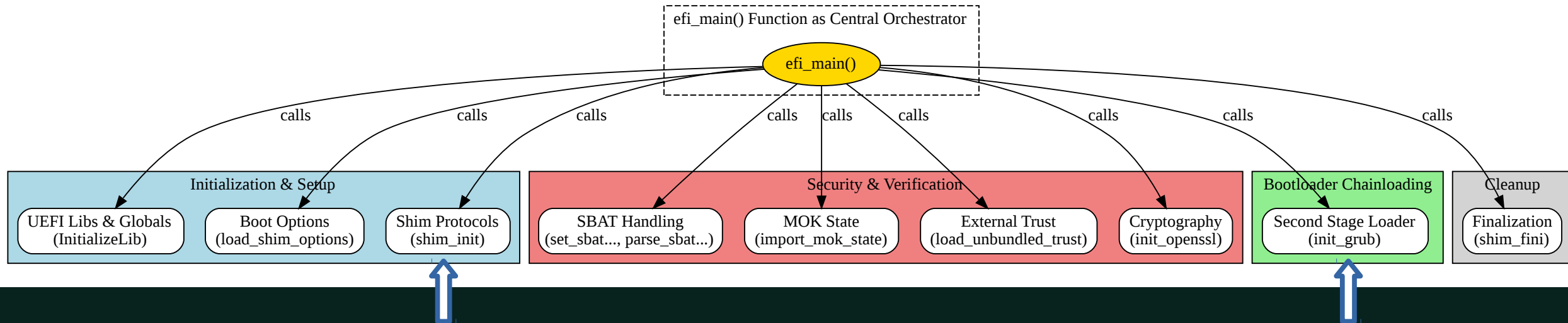


Protocol, Load and Boot



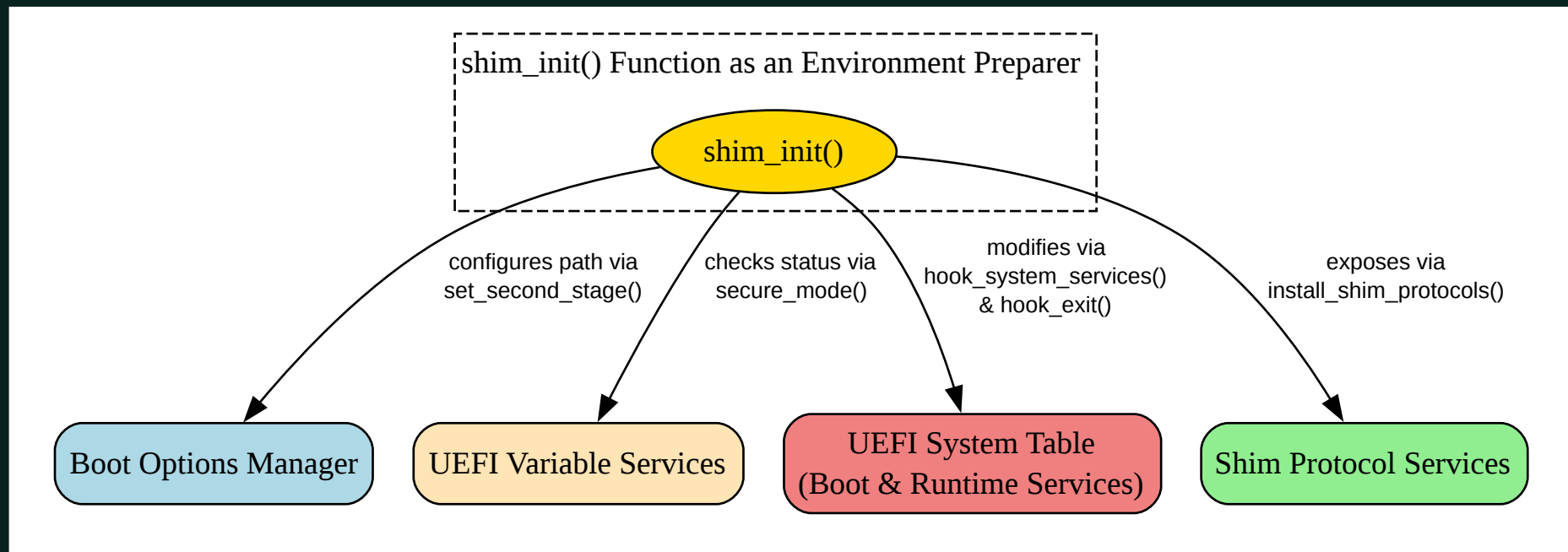
Shim and grub initial in efi_main()

- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the **efi_main()** function and its interactions."



Block diagram from Gemini scans shim_init()

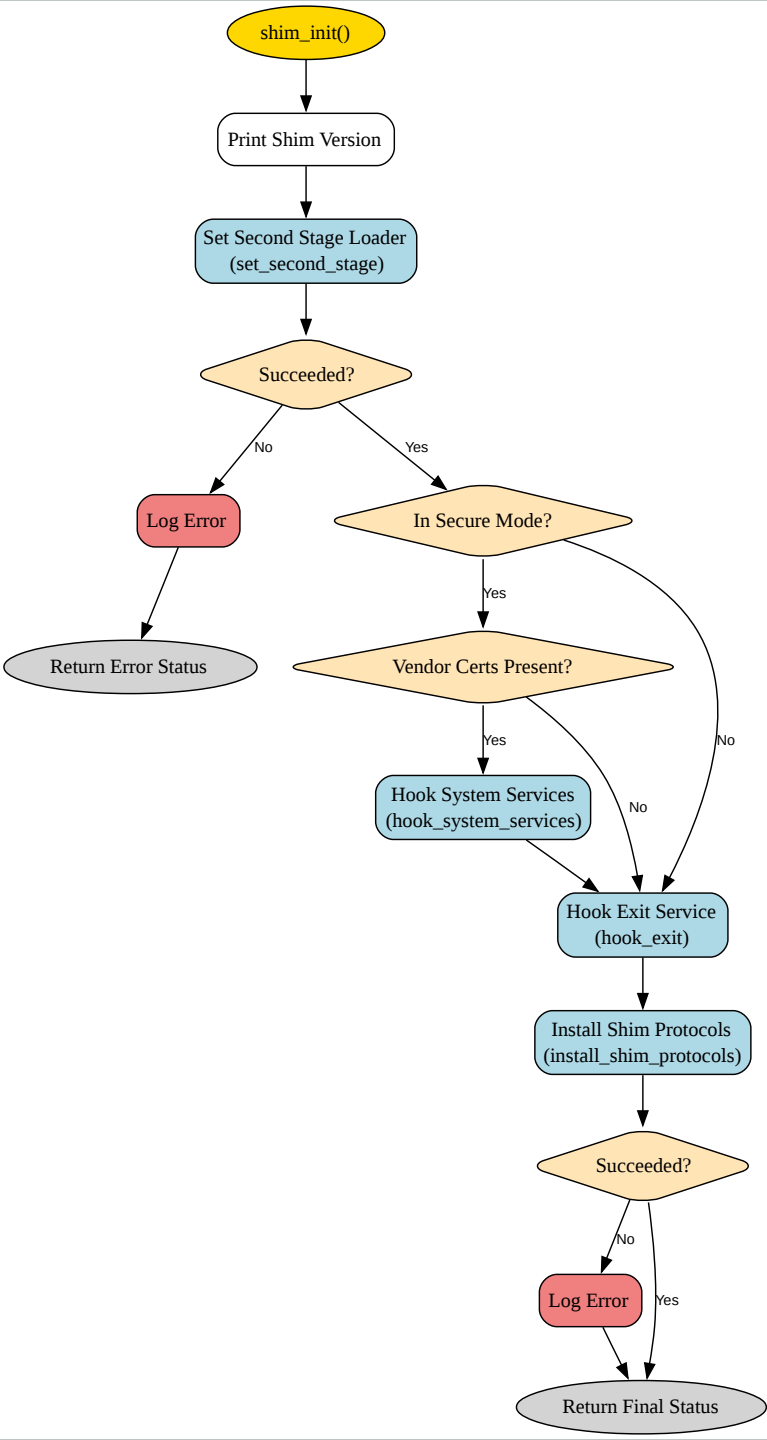
- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the **shim_init()** function and its interactions."



FlowChart from Gemini scans shim_init()

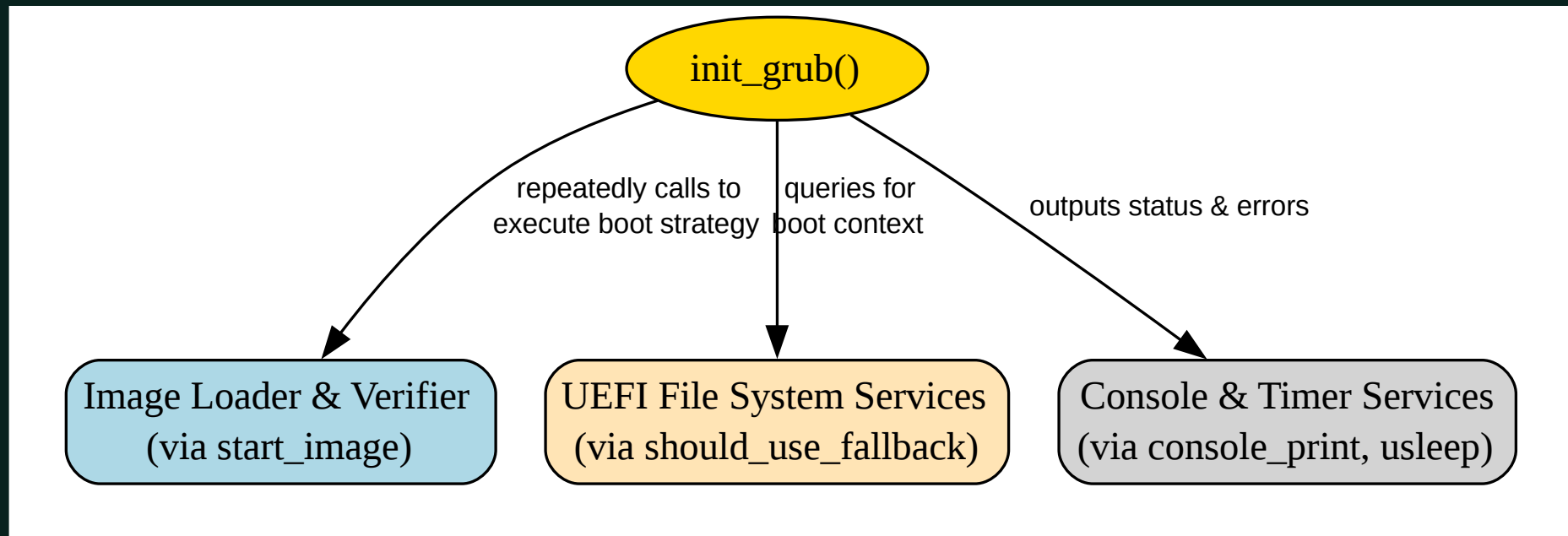
- Prompt: Please generate a **flow chart**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the flow chart with a focus on the **shim_init()** function and its interactions."

Flow Chart from Gemini scans shim_init()



Block diagram from Gemini scans init_grub()

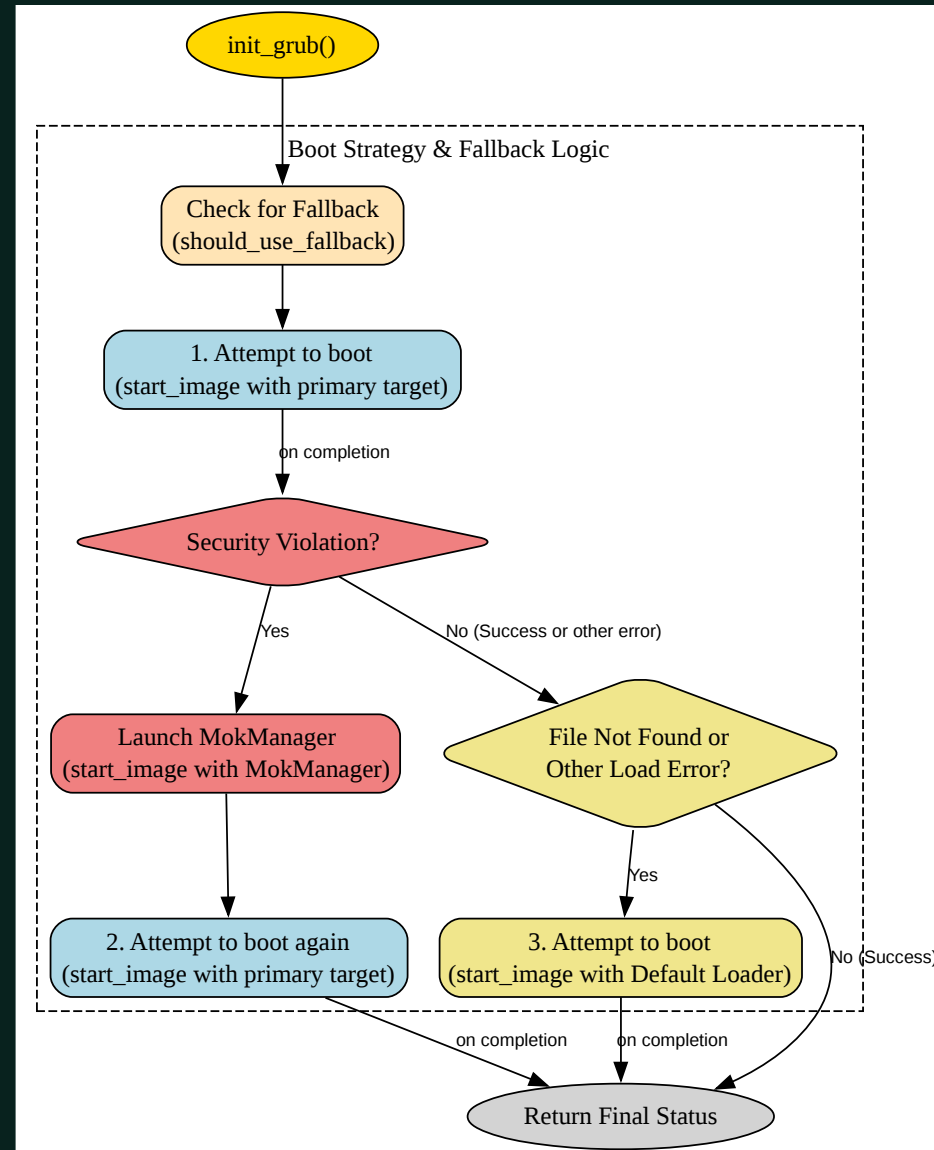
- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the **init_grub()** function and its interactions."



Flow Chart from Gemini scans init_grub()

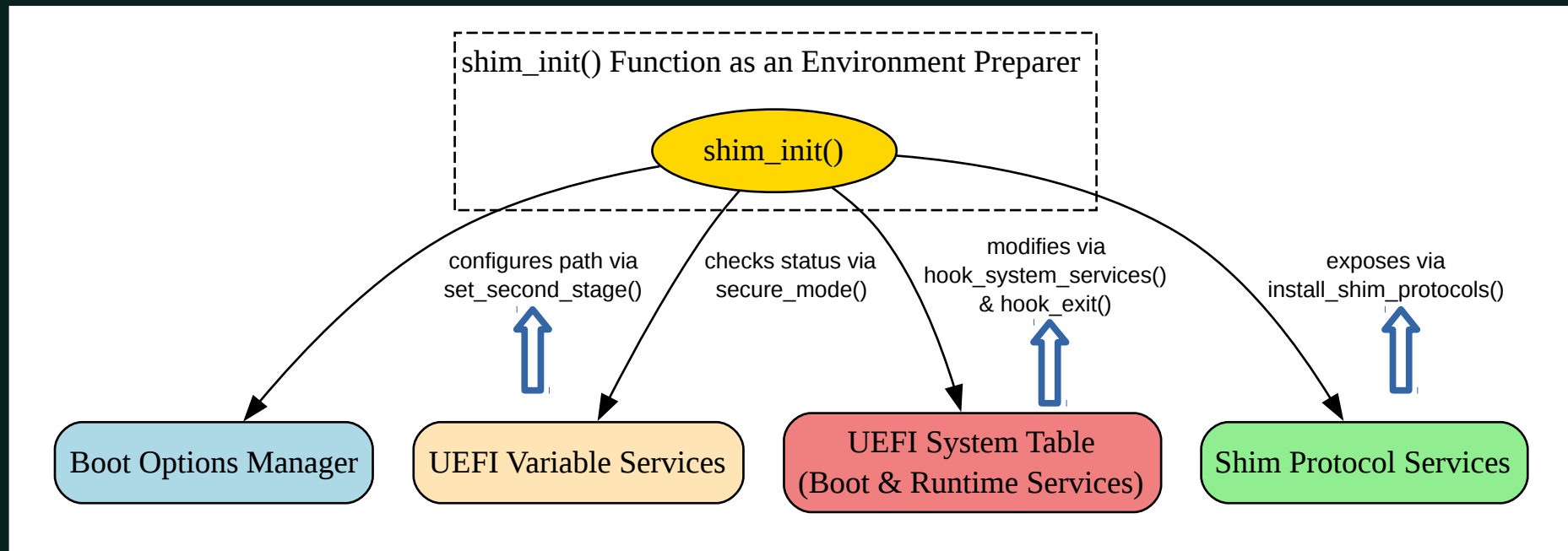
- Prompt: Please generate a **flow chart**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the flow chart with a focus on the **init_grub()** function and its interactions."

Flow Chart from Gemini scans init_grub()

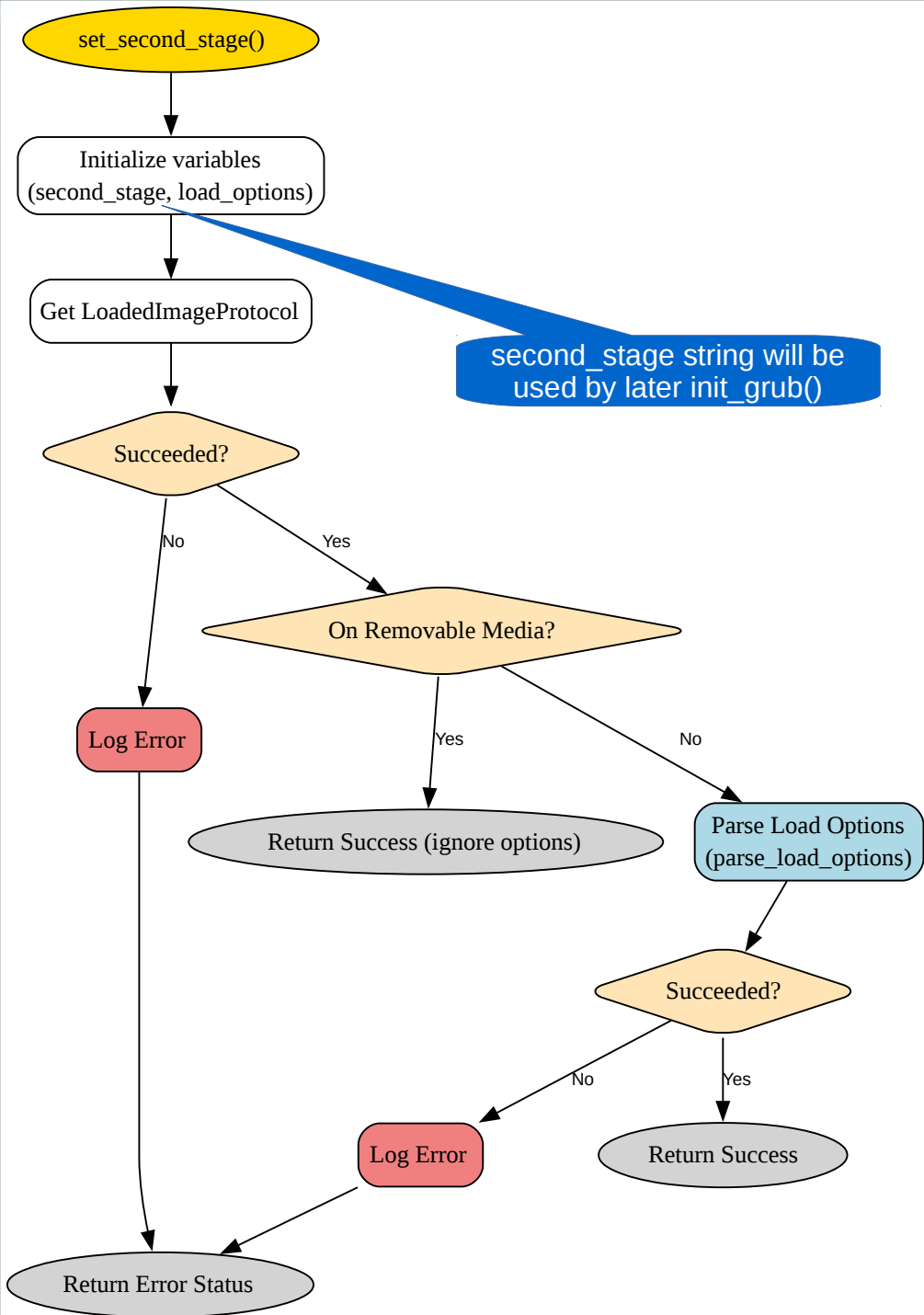


Digshim_init()

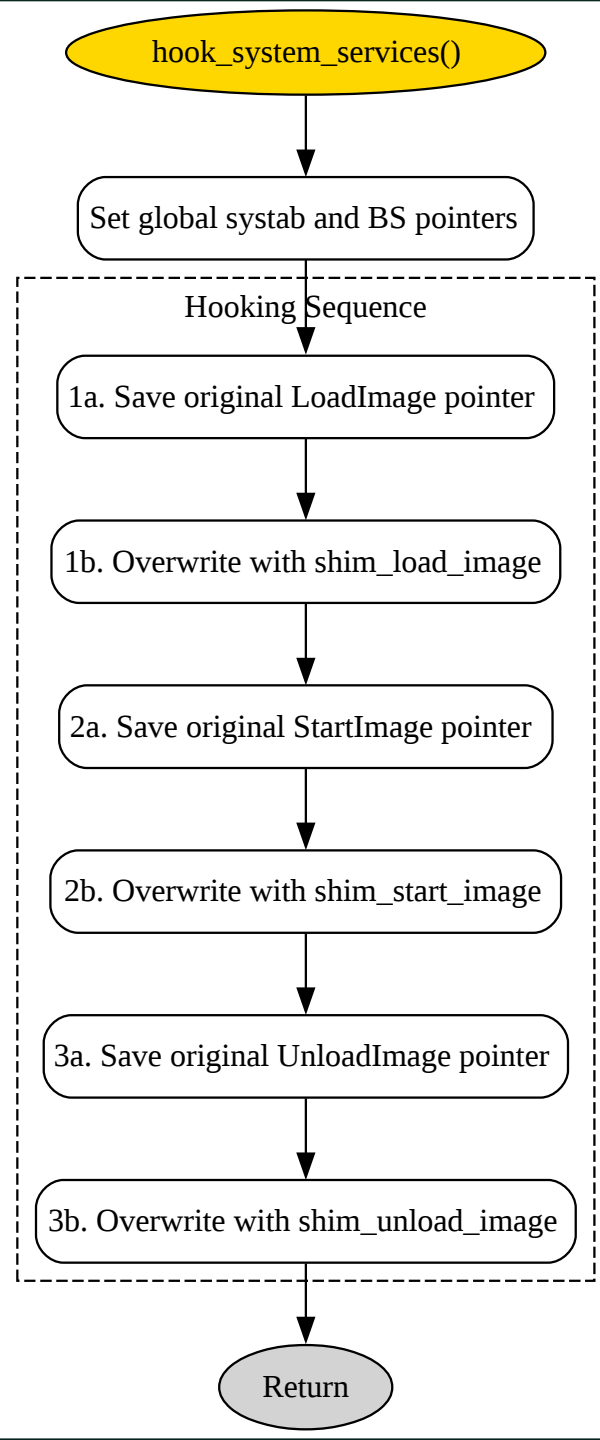
- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the **shim_init()** function and its interactions."



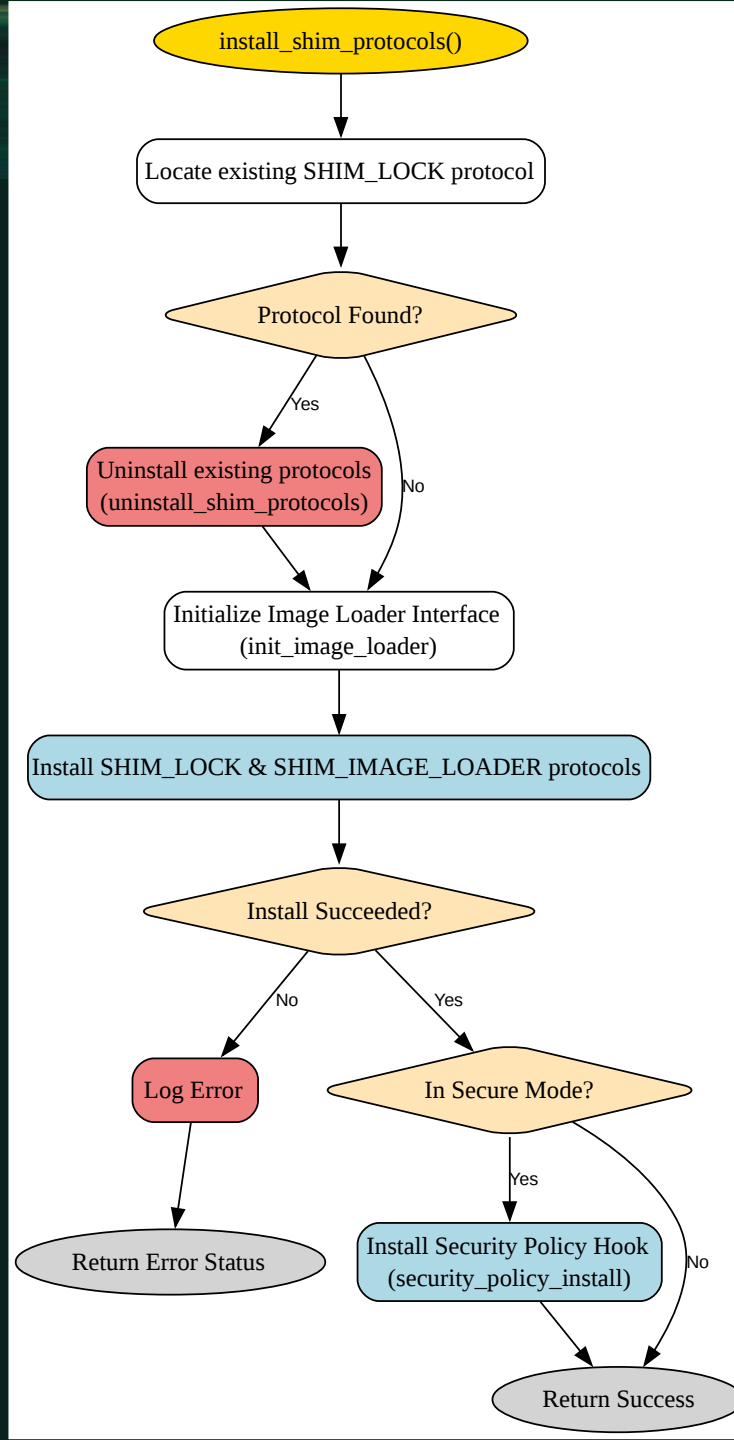
Flow Chart from Gemini scans set_second_stage()



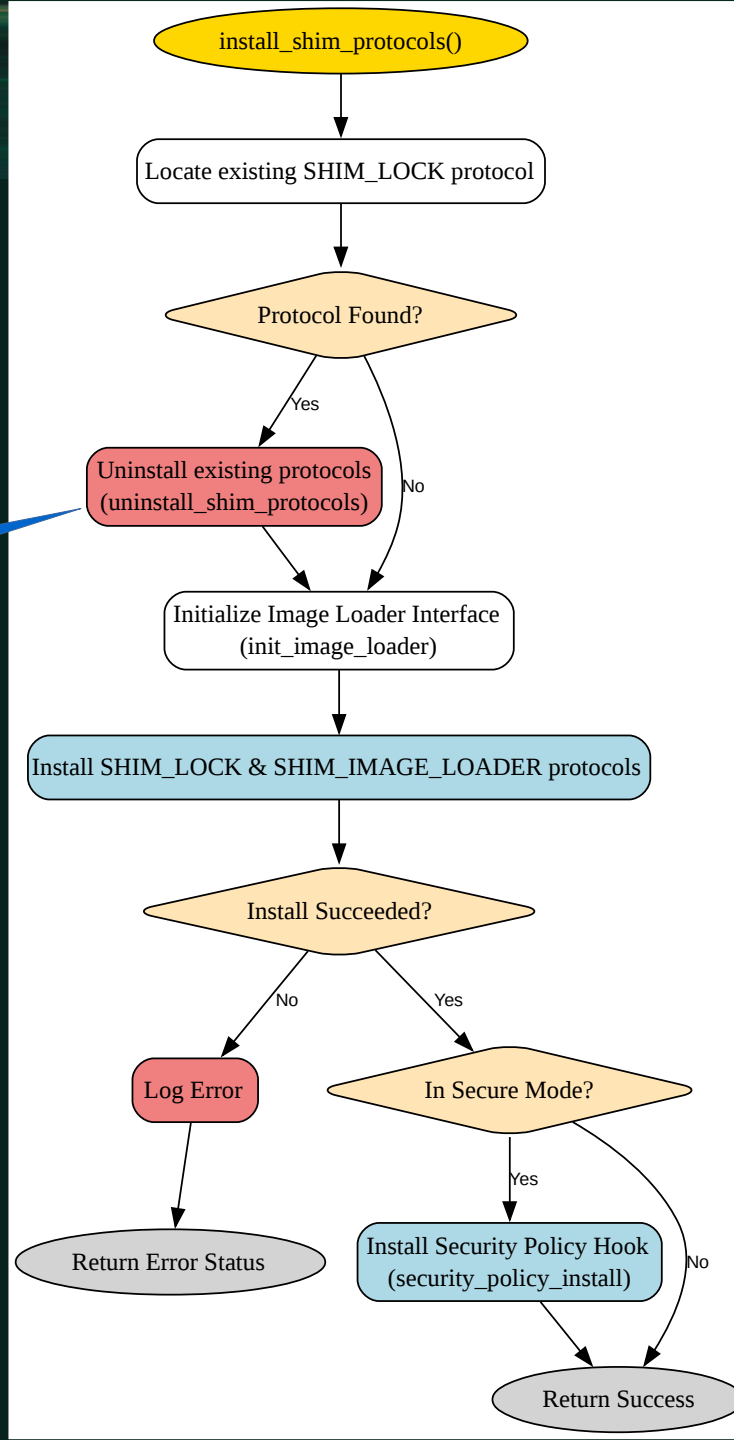
Flow Chart from Gemini scans hook_system_services()



Flow Chart from Gemini scans install_shim_protocols()



Flow Chart from Gemini scans install_shim_protocols()

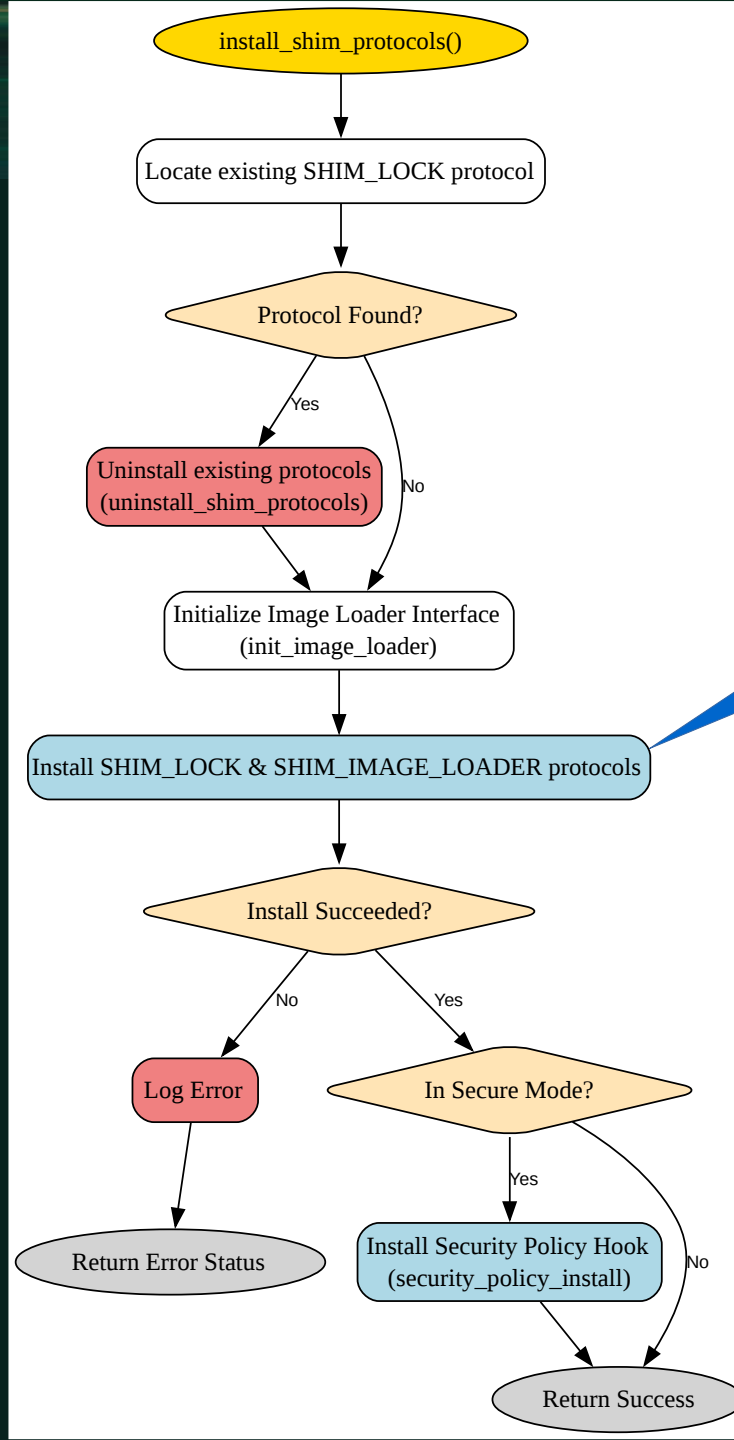


```

shim->fallback->shim->grub
^               ^      ^
|               |      |
|               |      \- gets protocol #0
|               \- installs its protocol (#1)
\ - installs its protocol (#0)

```

Flow Chart from Gemini scans install_shim_protocols()

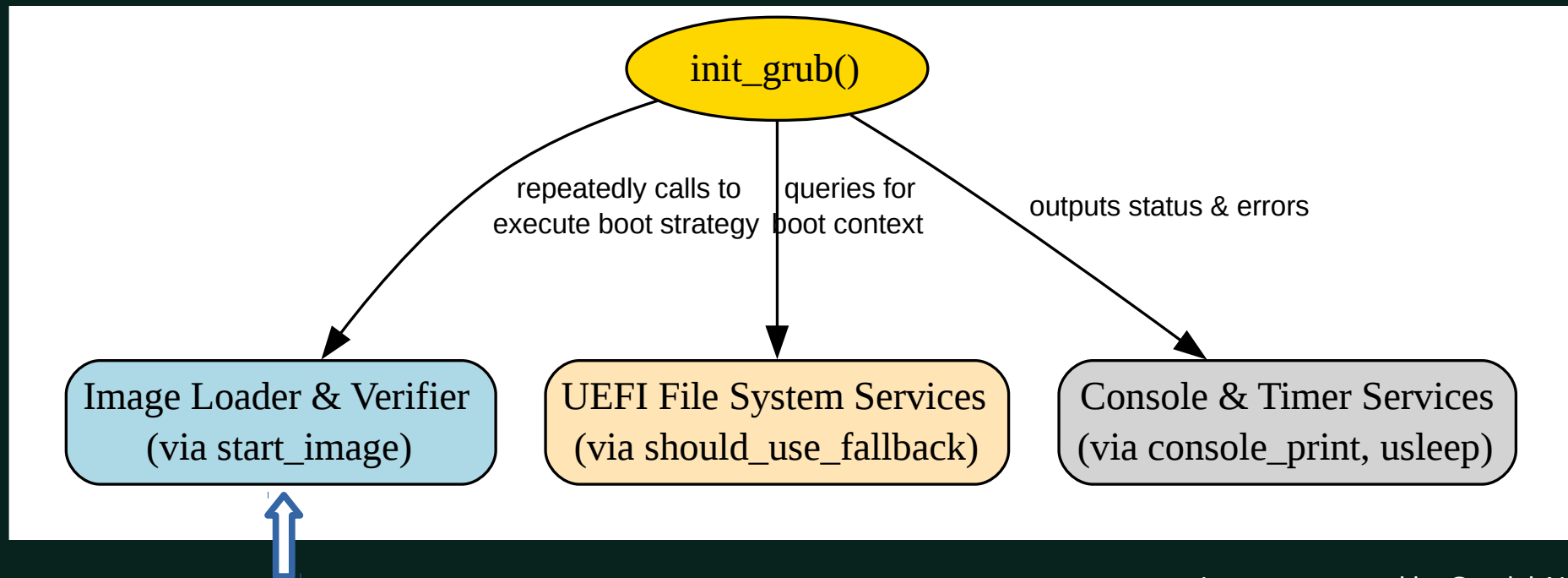


In grub2

```
if SHIM_IMAGE_LOADER != NULL
    unload_image
    load_image
if SHIM_LOCK != NULL
    verify
```

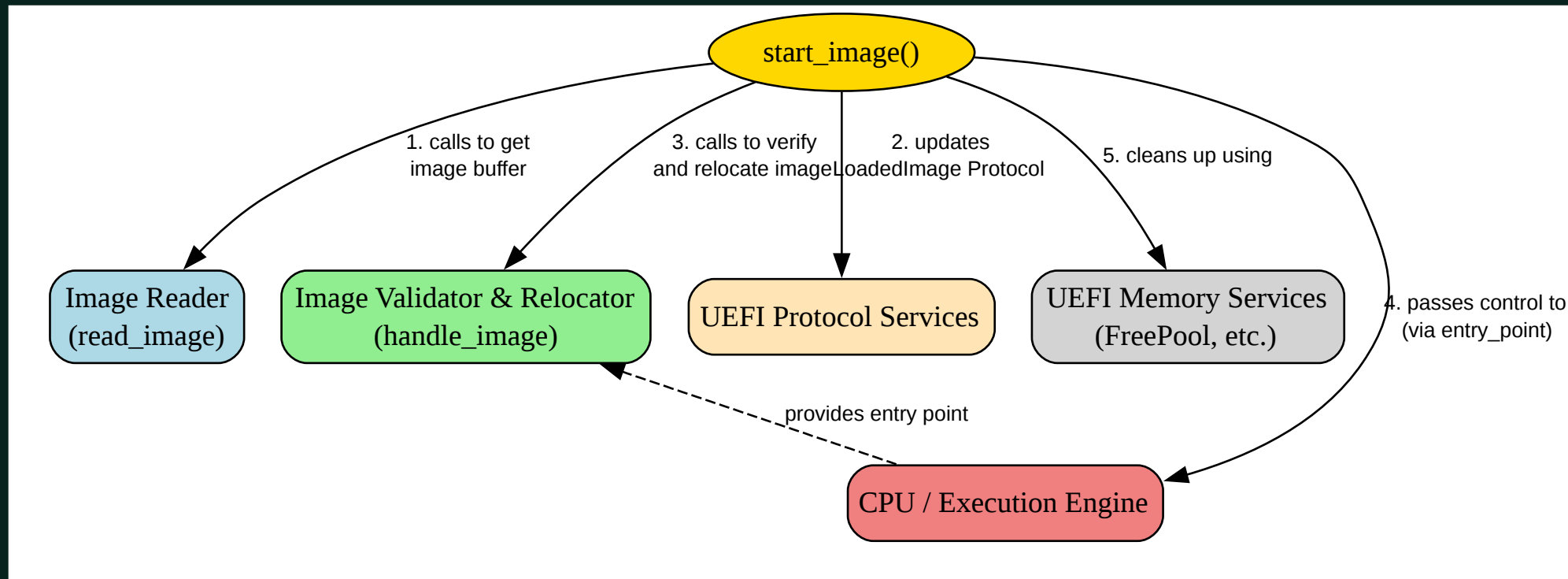
Block diagram from Gemini scans init_grub()

- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the **init_grub()** function and its interactions."



Block diagram from Gemini scans start_image()

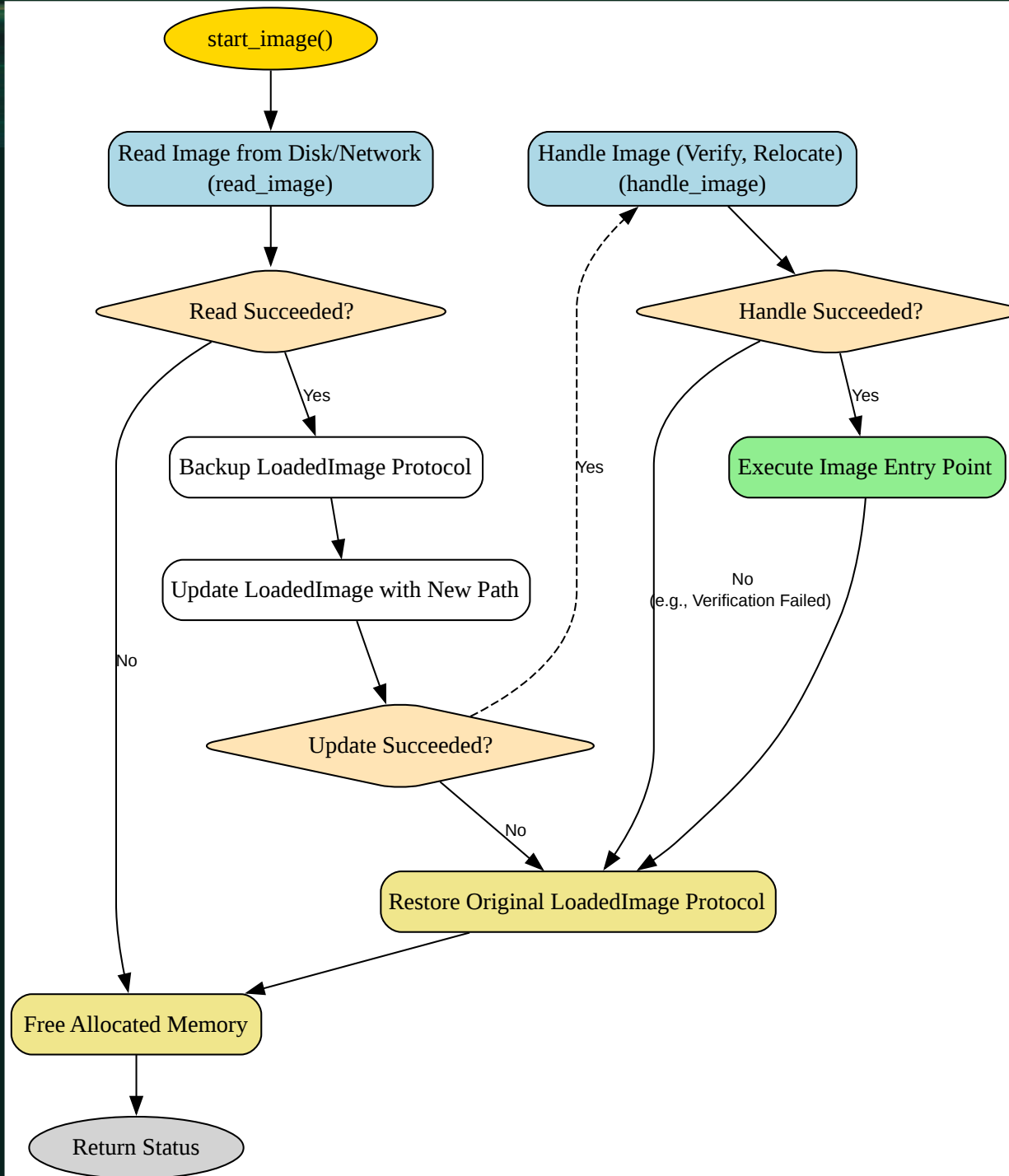
- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the **start_image()** function and its interactions."



Flow Chart from Gemini scans start_image()

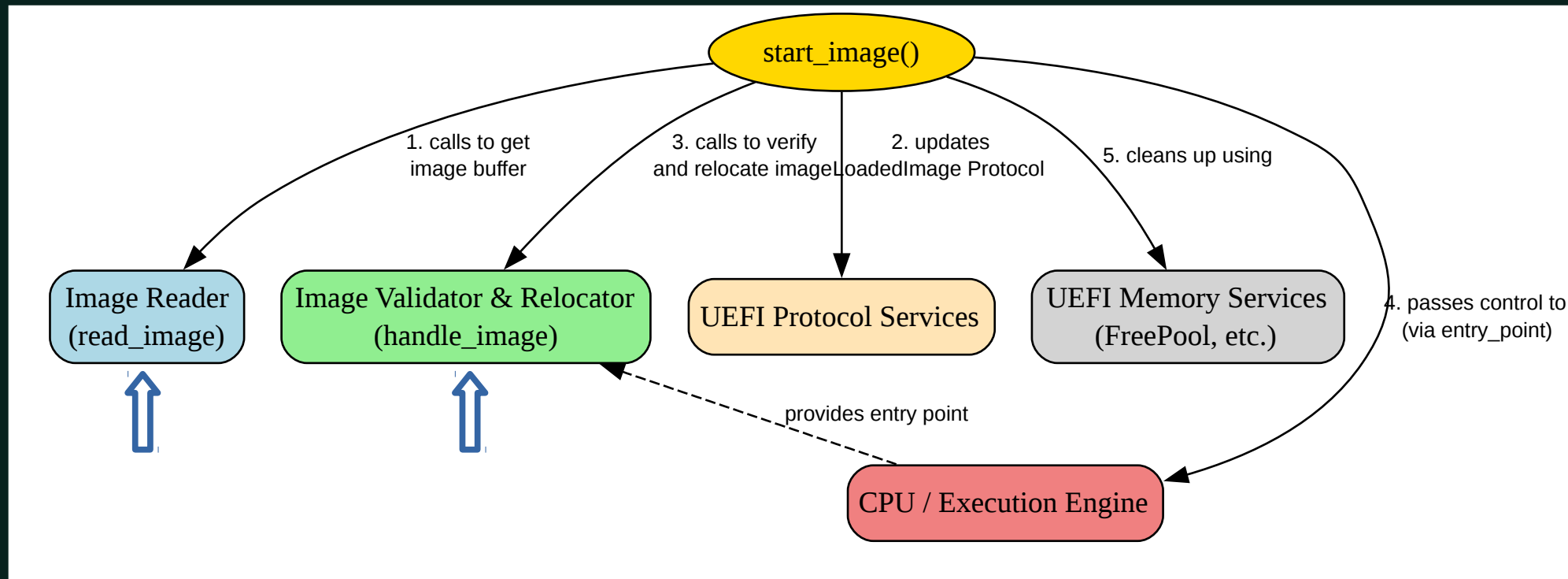
- Prompt: Please generate a **two-column flowchart** for the **start_image()** function, designed to be wider and less tall for presentation slides. Ensure all decision paths eventually lead to a cleanup step and a final return.

Flow Chart from Gemini scans start_image()

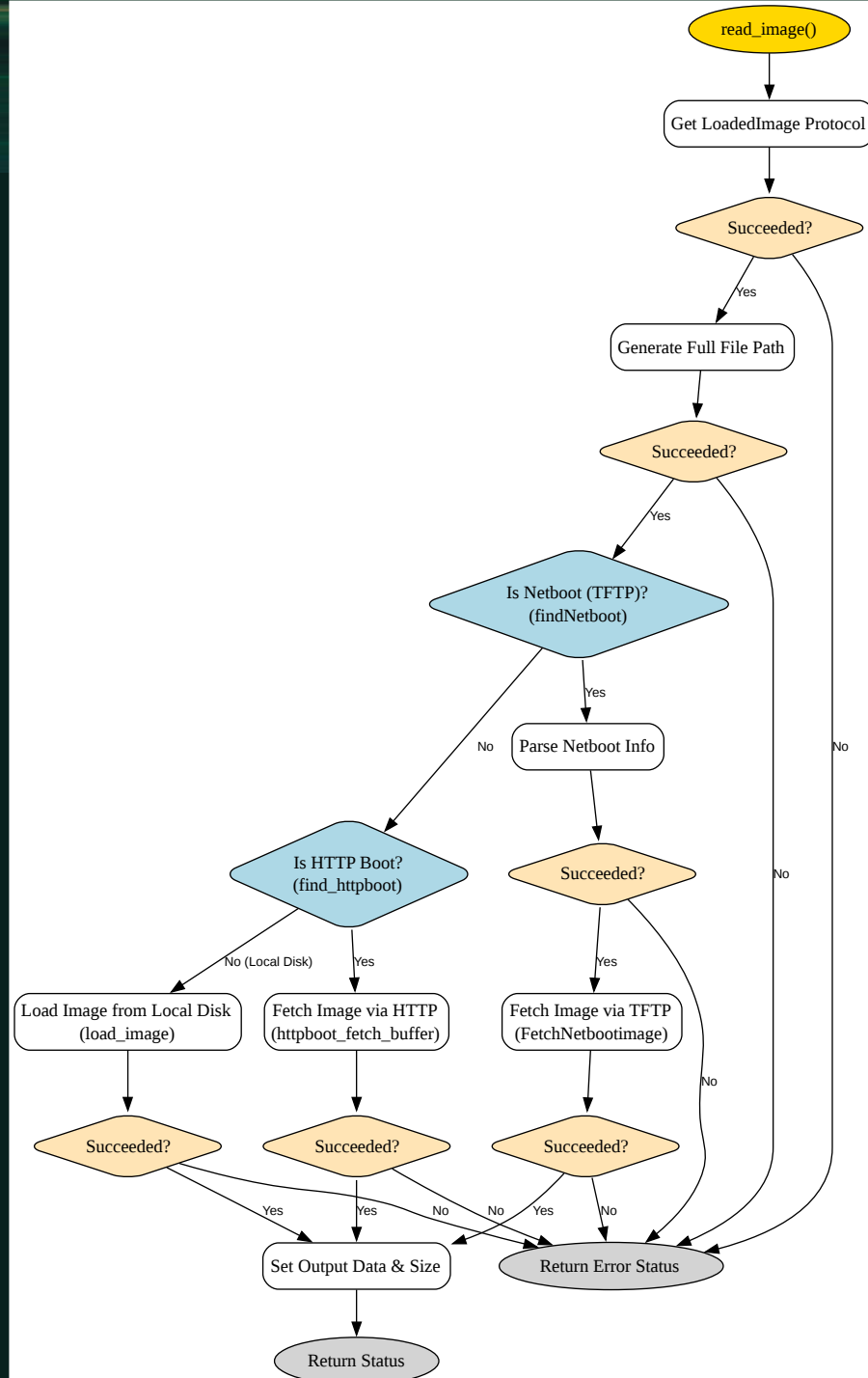


Dig Gemini scans start_image()

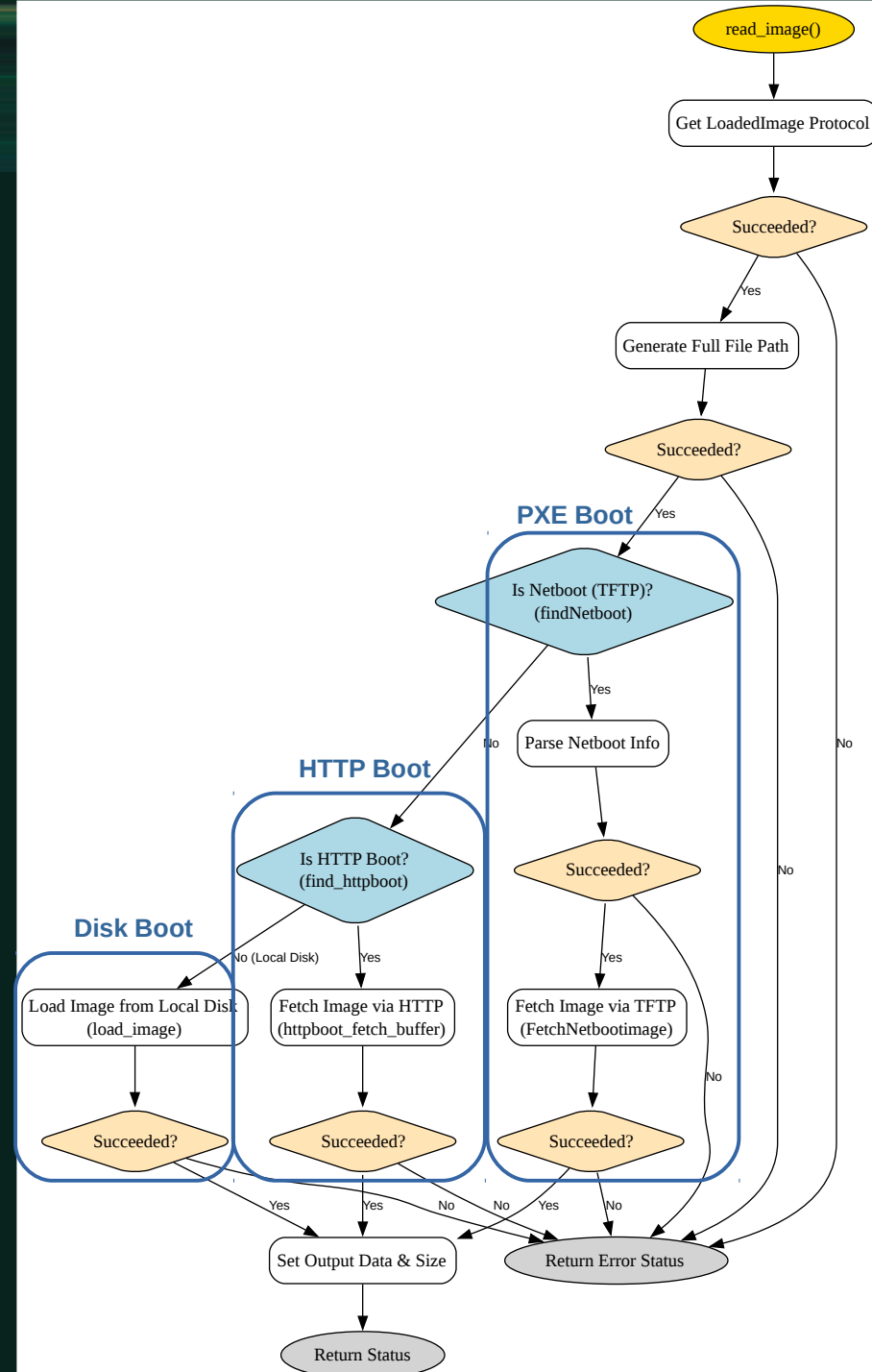
- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the **start_image()** function and its interactions."



Flow Chart from Gemini scans read_image()

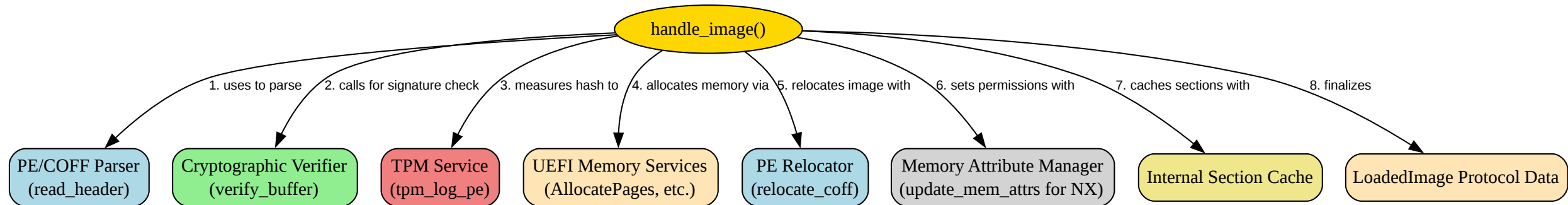


Flow Chart from Gemini scans read_image()



Block Diagram from Gemini scans handle_image()

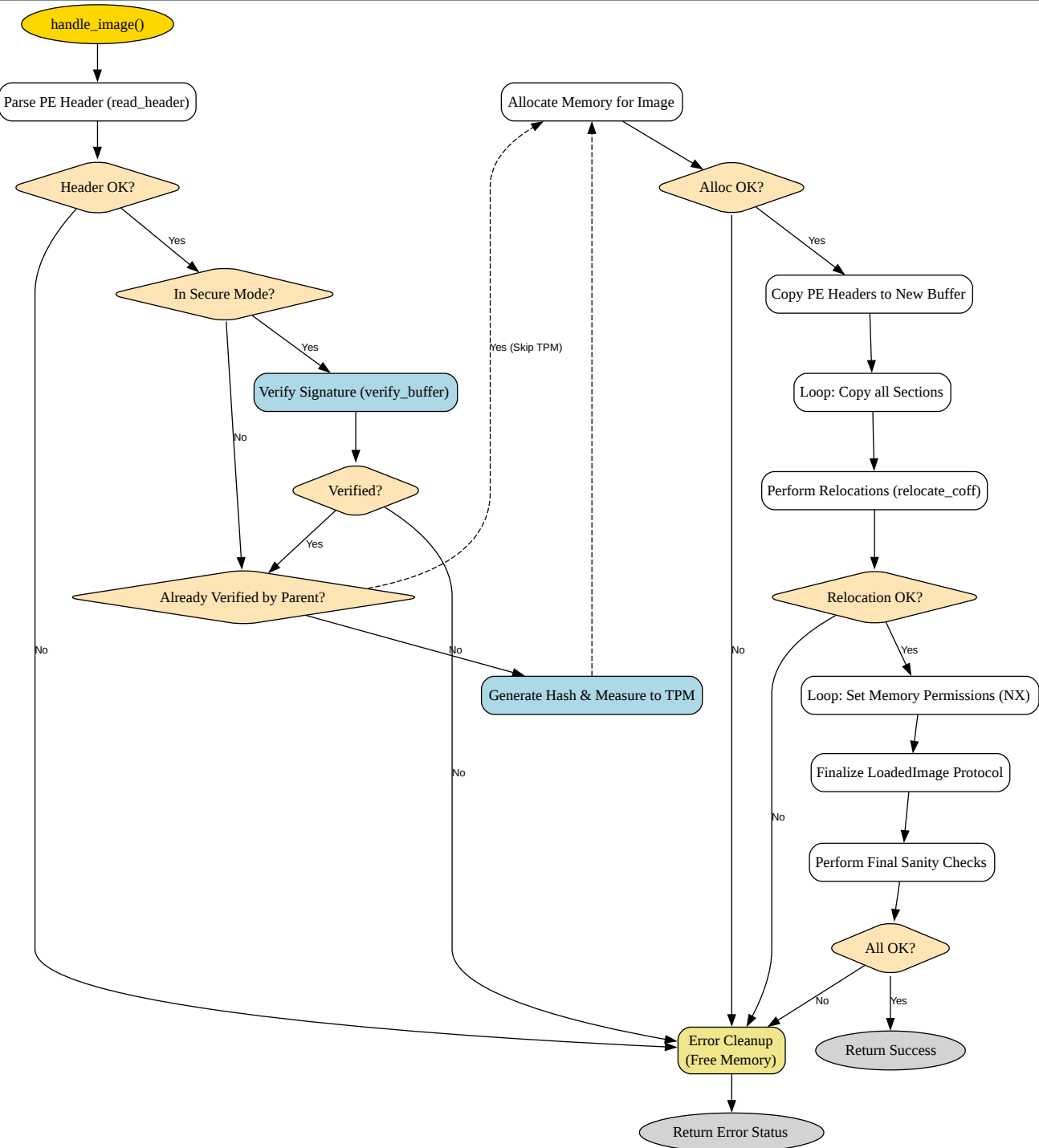
- Prompt: Please generate a block diagram. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the handle_image() function and its interactions."



Flow Chart from Gemini scans handle_image()

- Prompt: Please generate a two-column flowchart for the handle_image() function, designed to be wider and less tall for presentation slides. Ensure all decision paths eventually lead to a cleanup step and a final return.

Flow Chart from Gemini scans handle_image()



Flow Chart from Gemini scans handle_image()

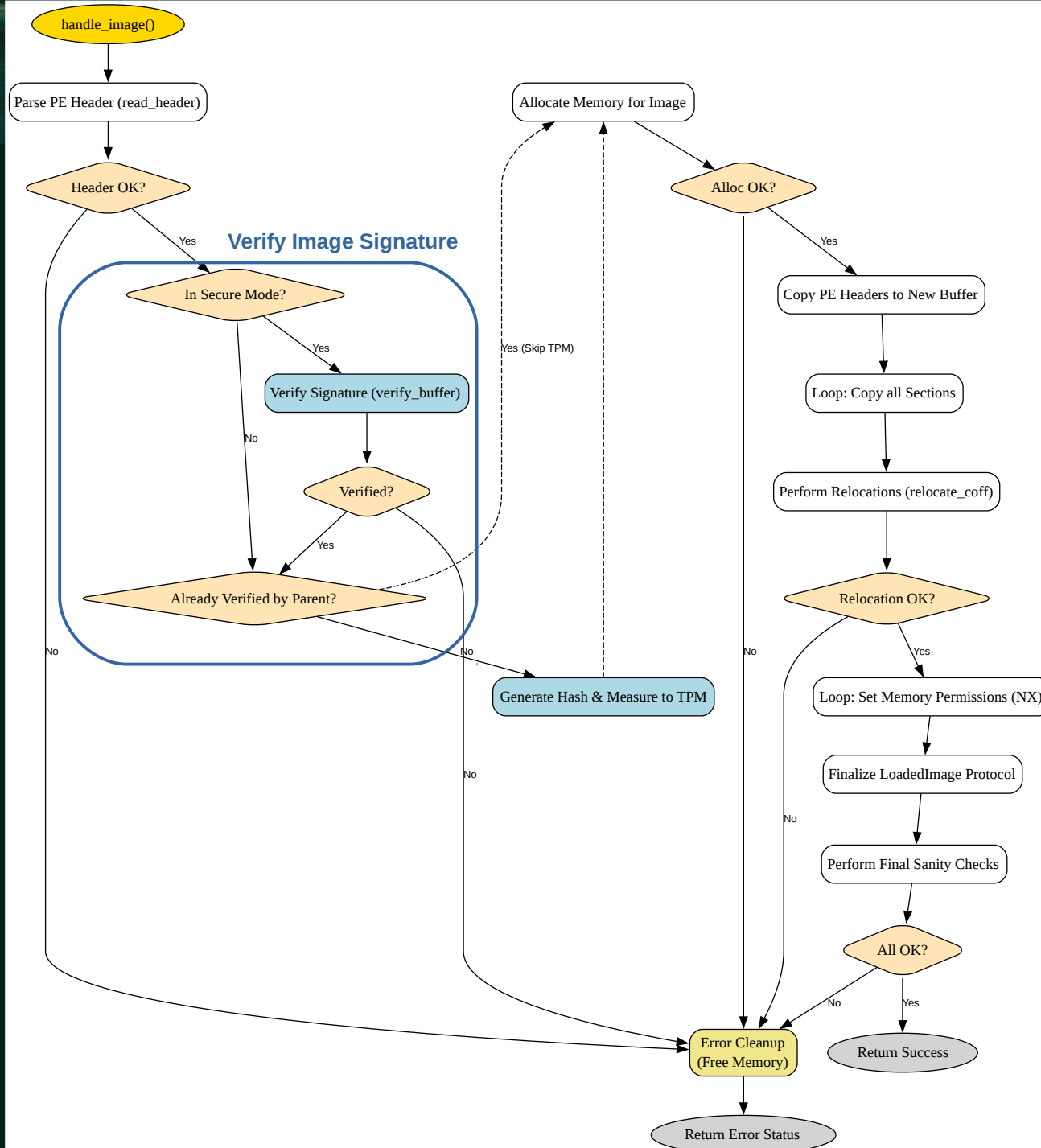
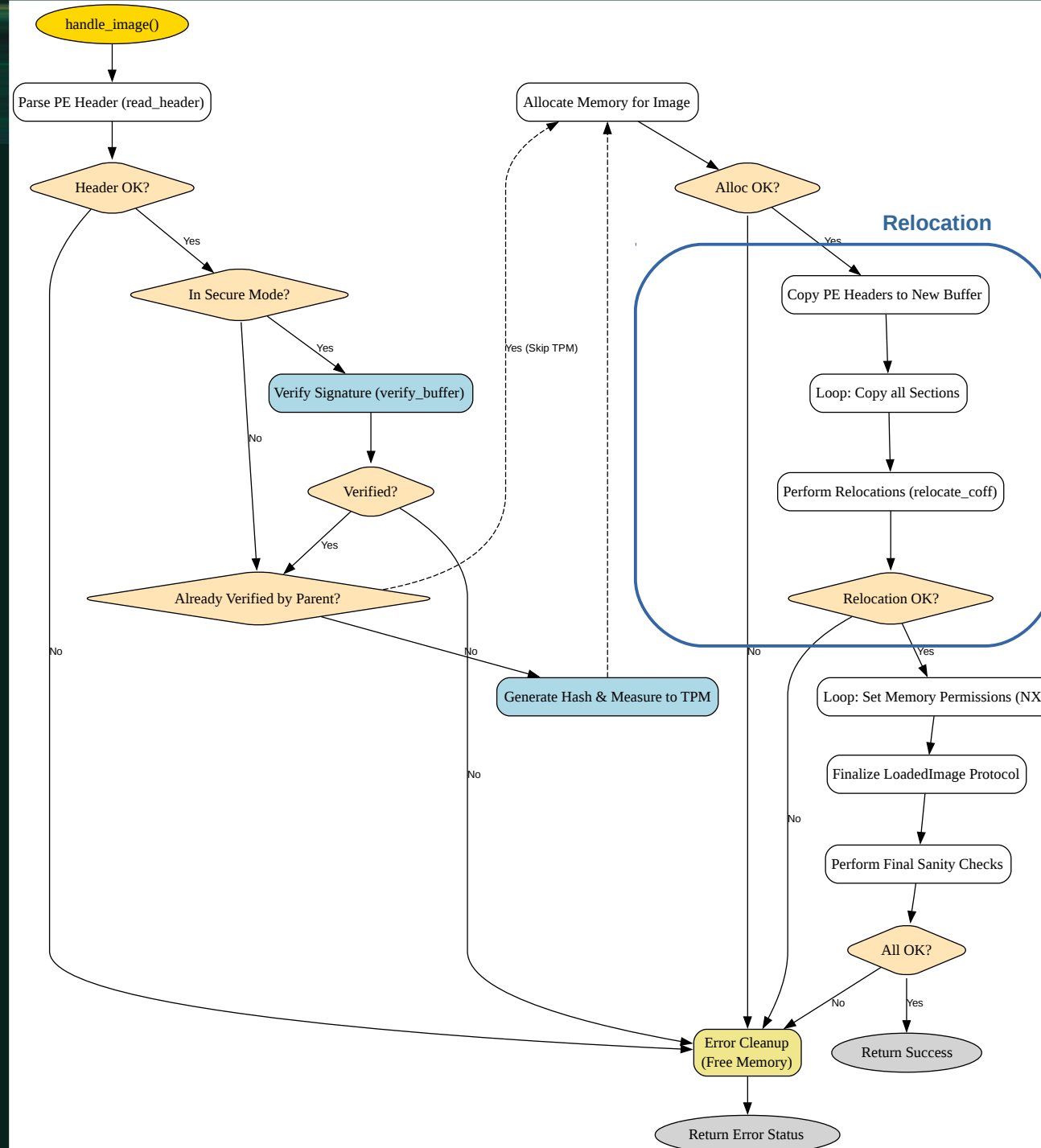
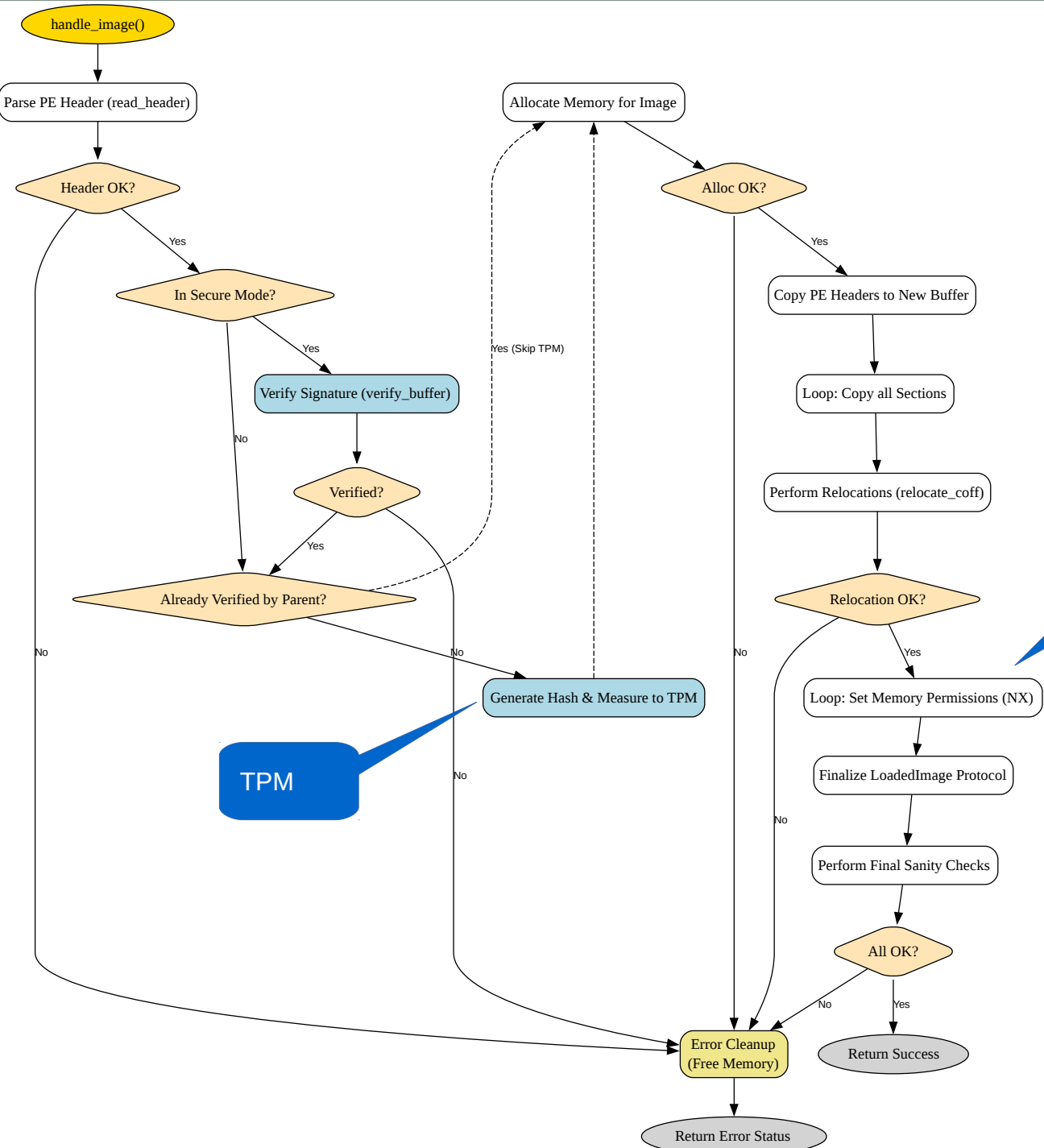


Image generated by Gemini AI

Flow Chart from Gemini scans handle_image()



Flow Chart from Gemini scans handle_image()



TPM

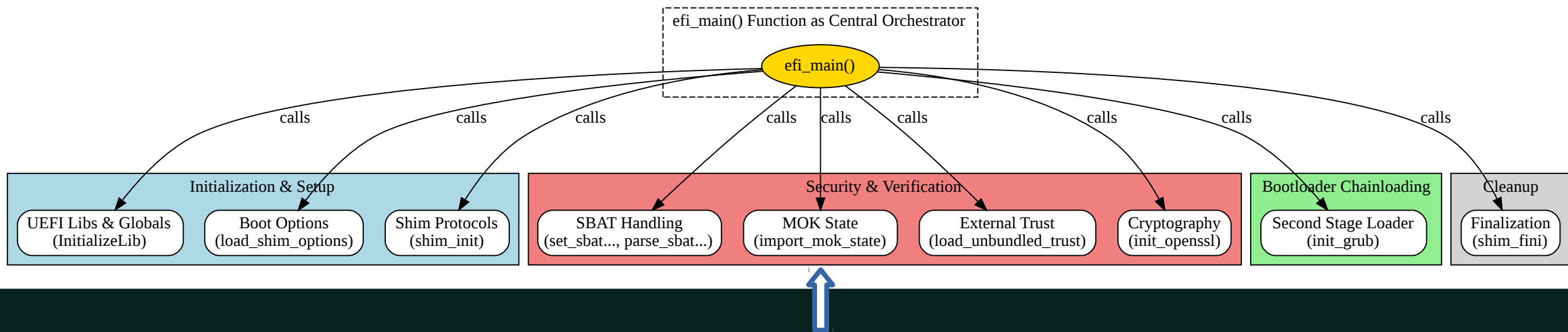
NX (No-Execute)

MOK



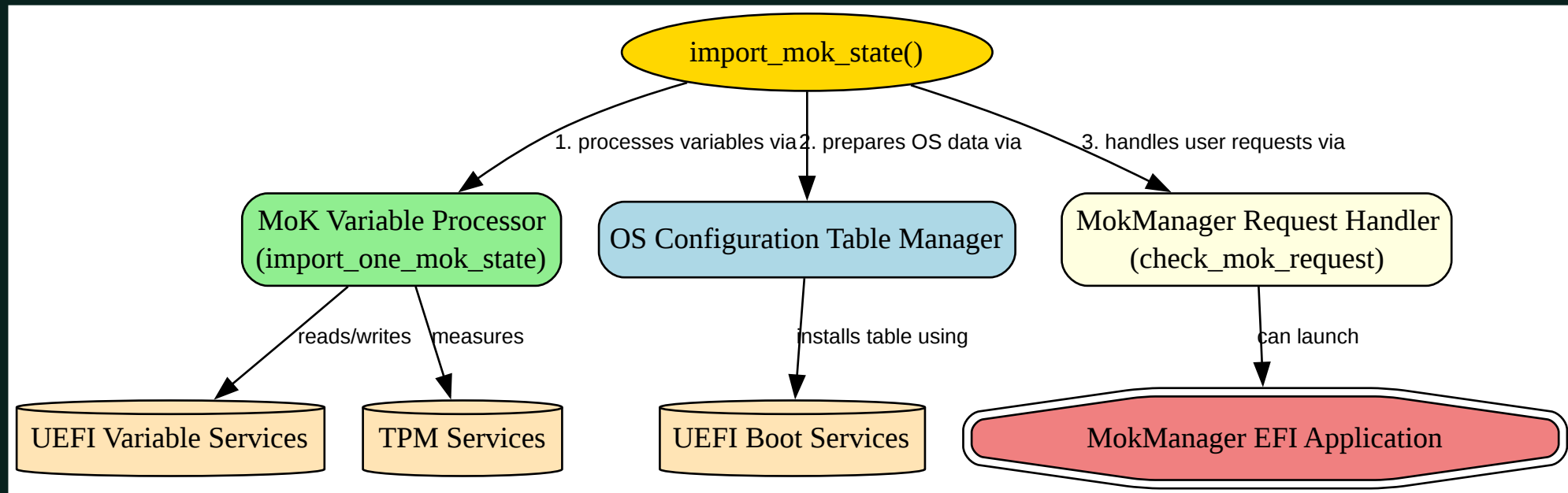
MOK in efi_main()

- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the **efi_main()** function and its interactions."

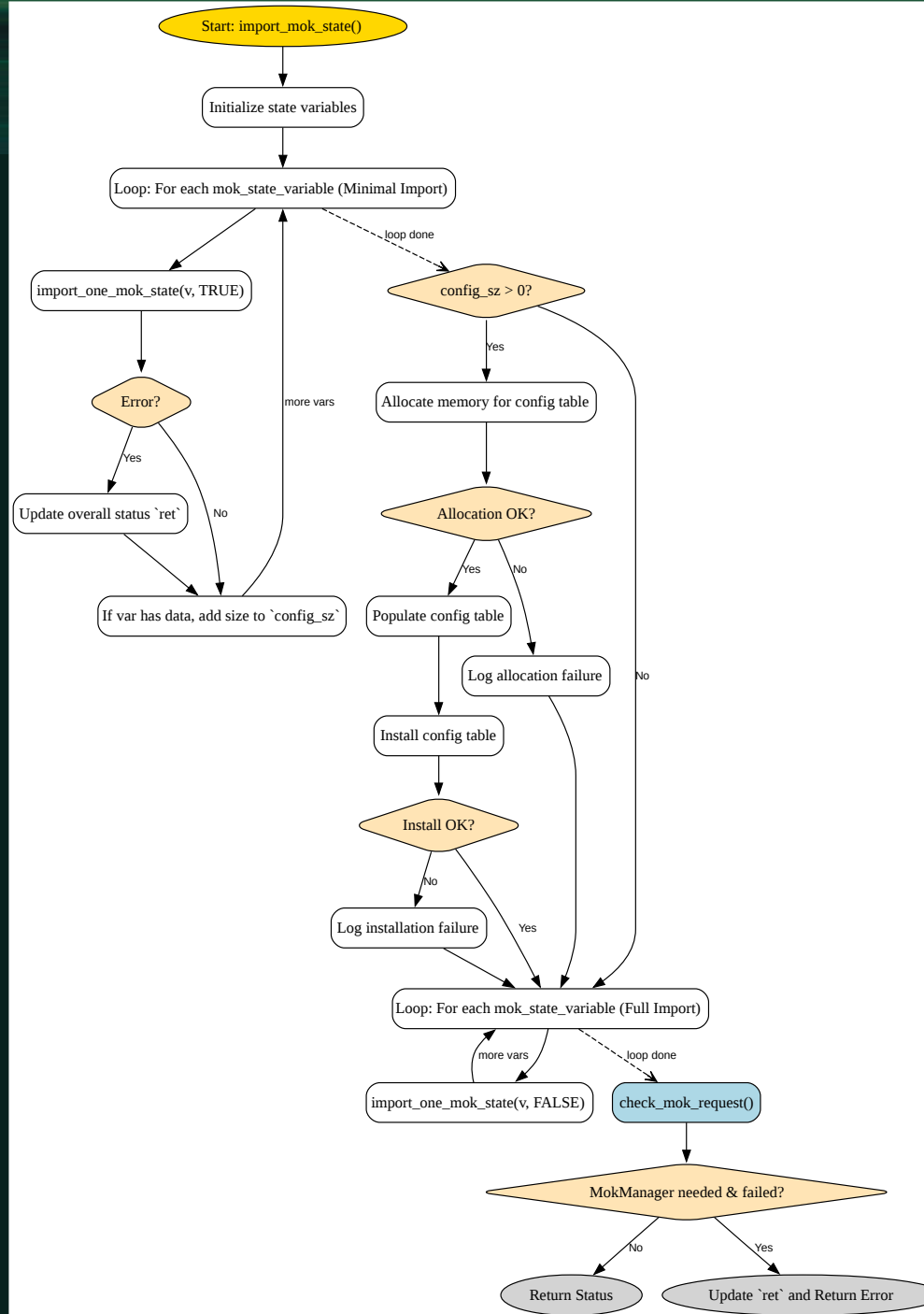


Block diagram from Gemini scans import_mok_state()

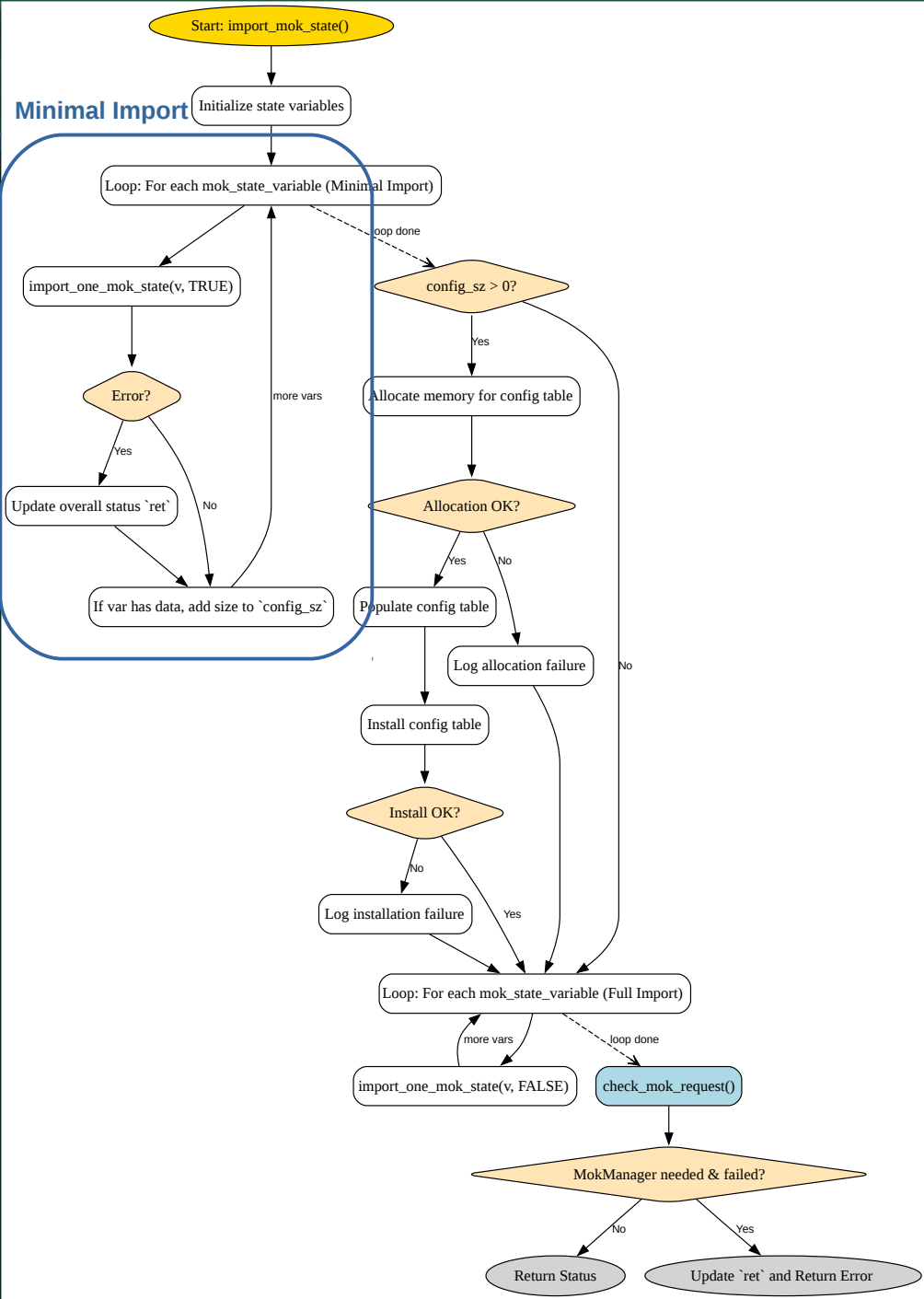
- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the import_mok_state() function and its interactions."



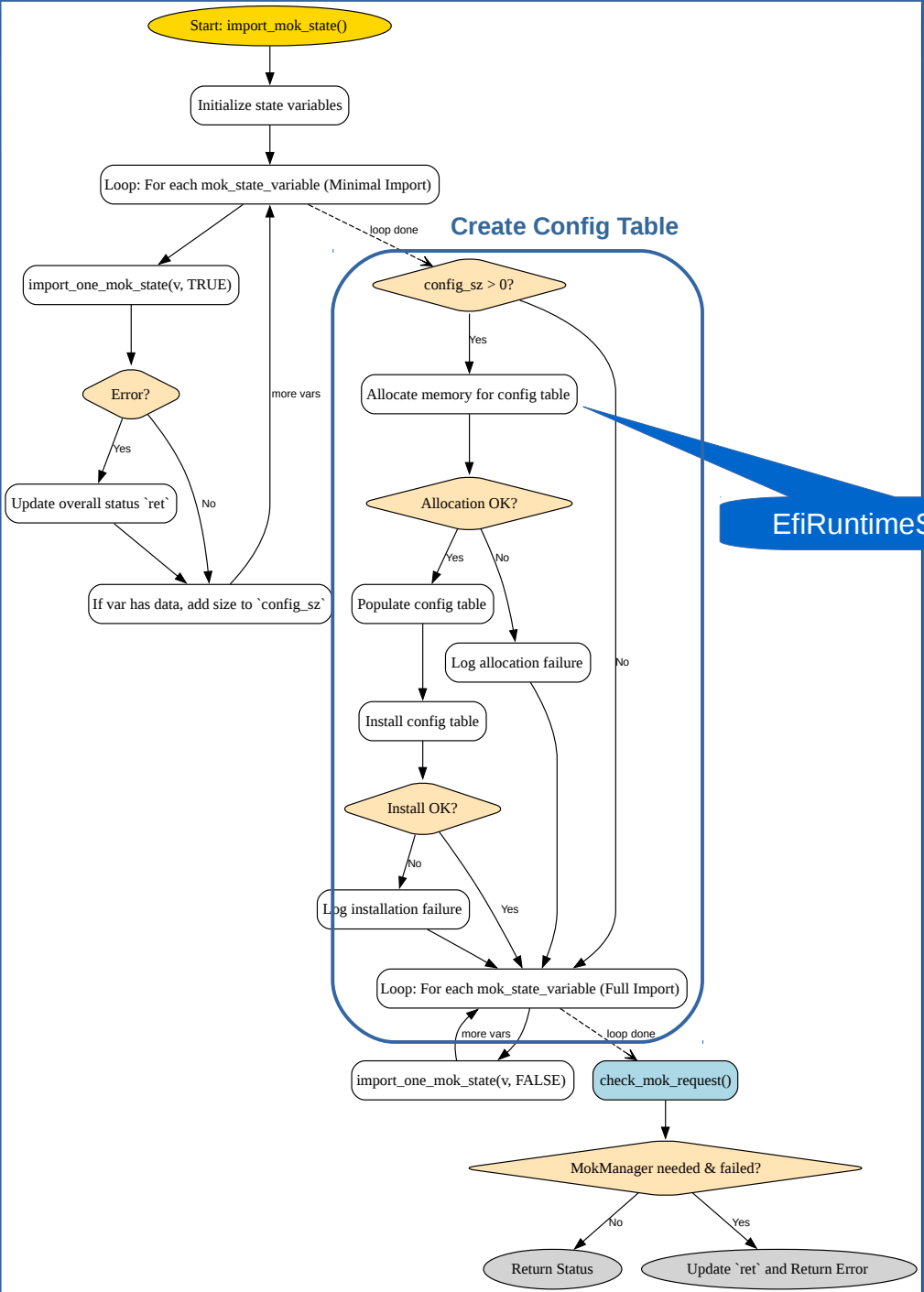
Flow Chart from Gemini scans import_mok_state()



Flow Chart from Gemini scans import_mok_state()

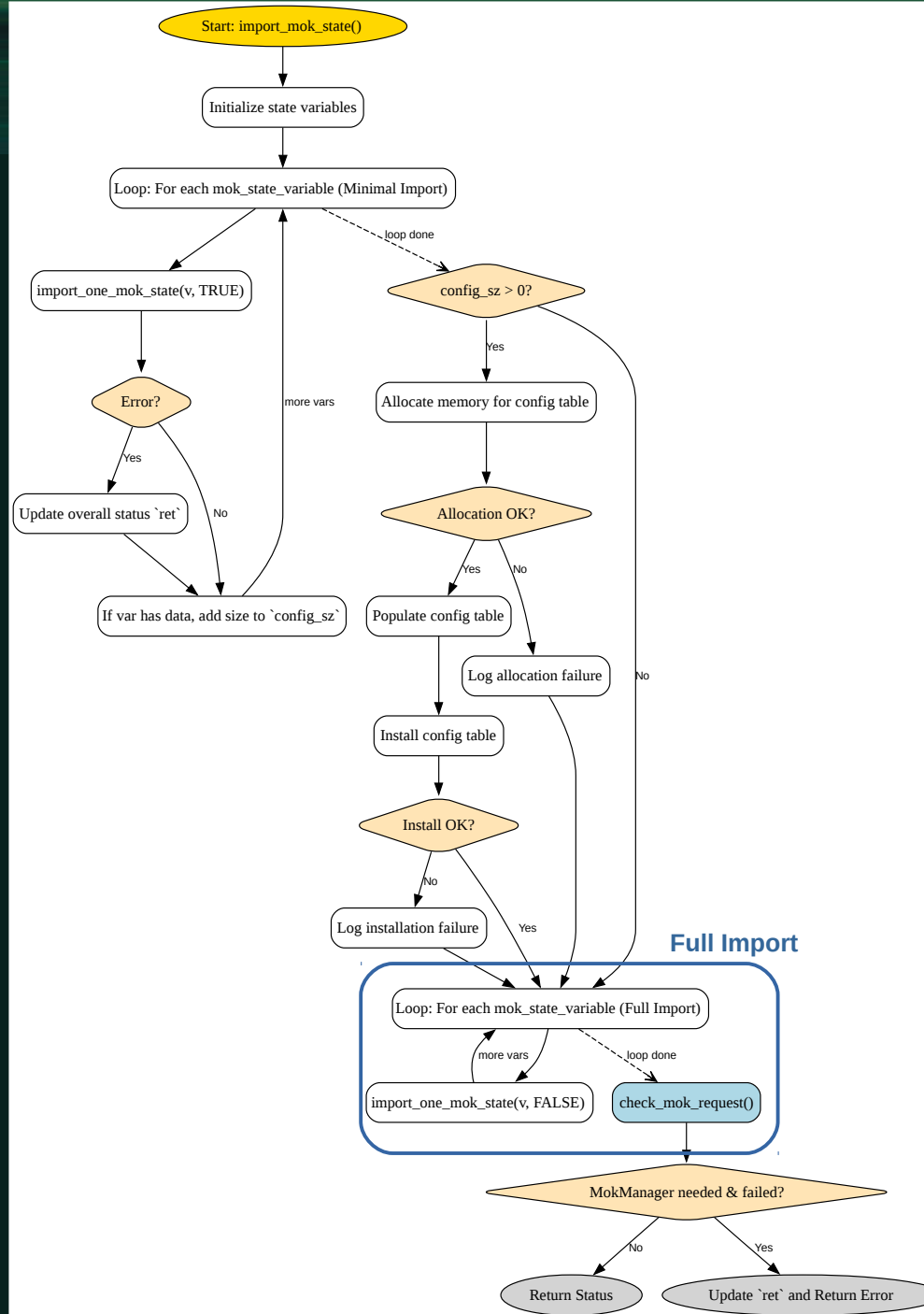


Flow Chart from Gemini scans import_mok_state()



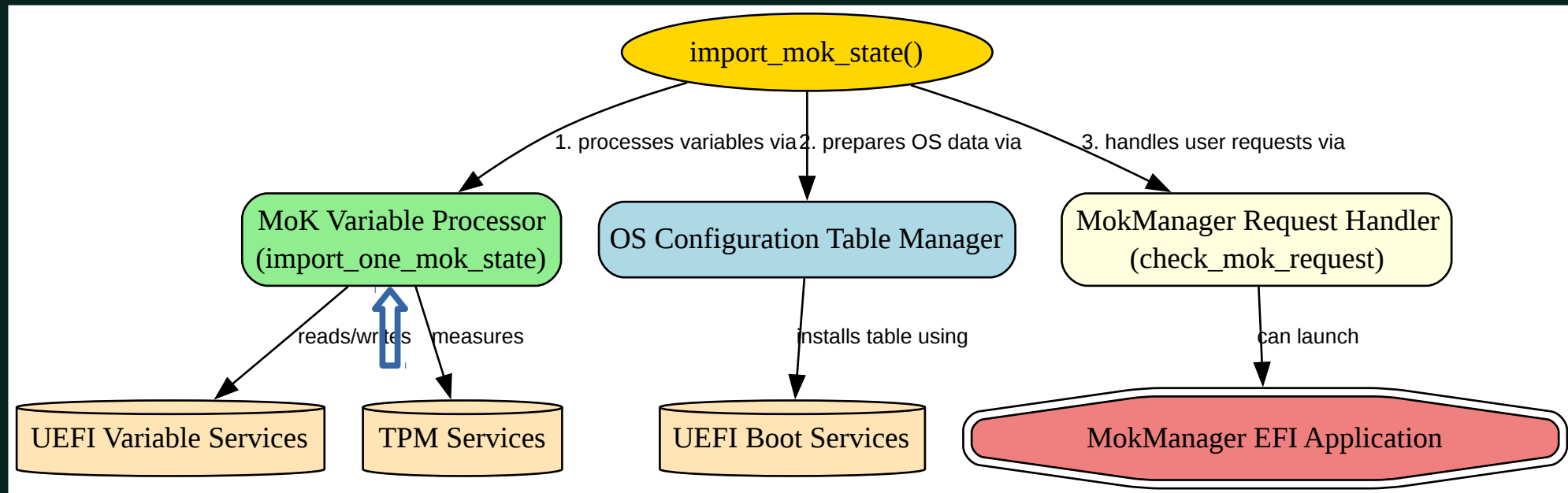
EfiRuntimeServicesData

Flow Chart from Gemini scans import_mok_state()

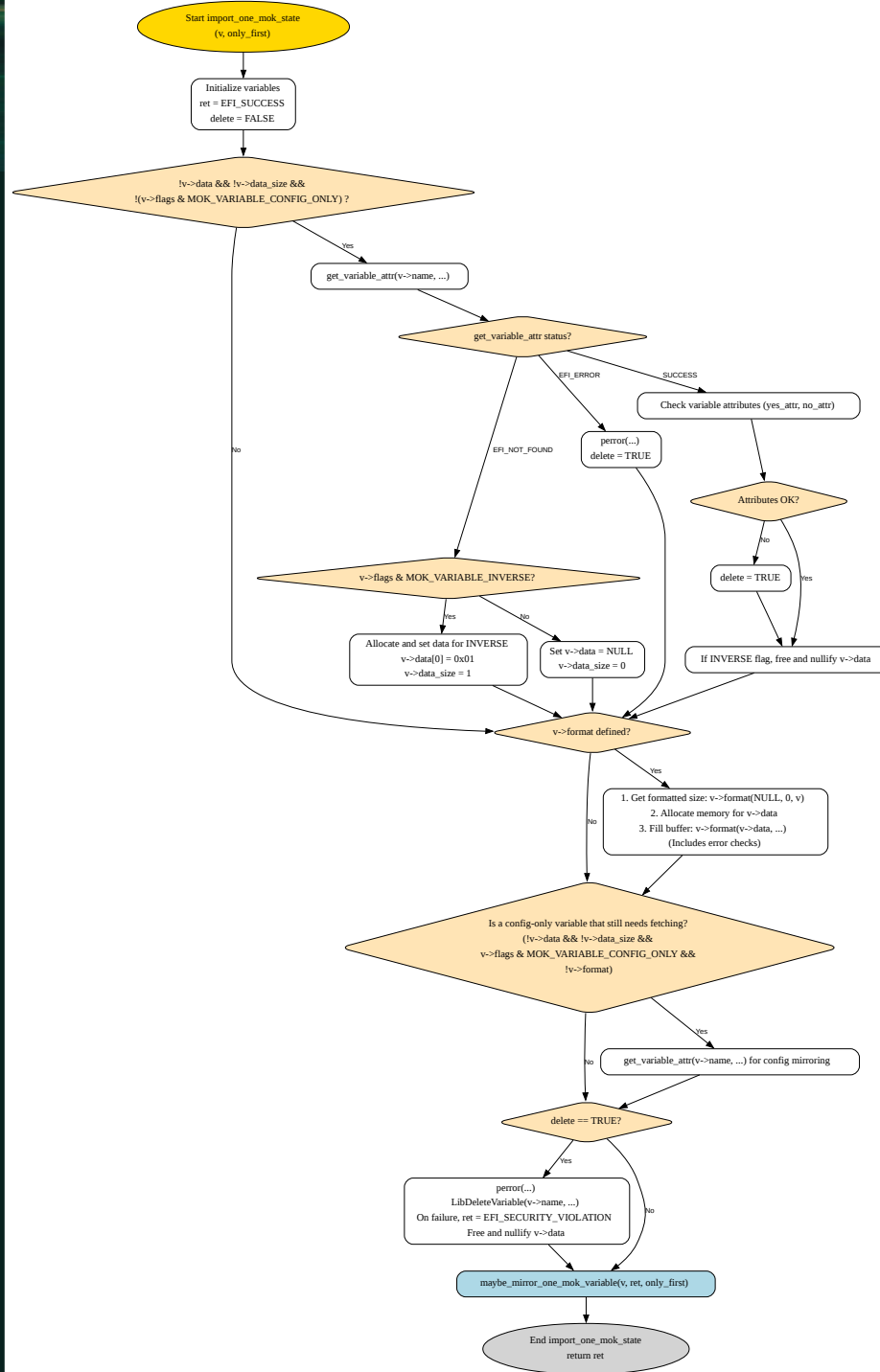


Dig import_one_mok_state()

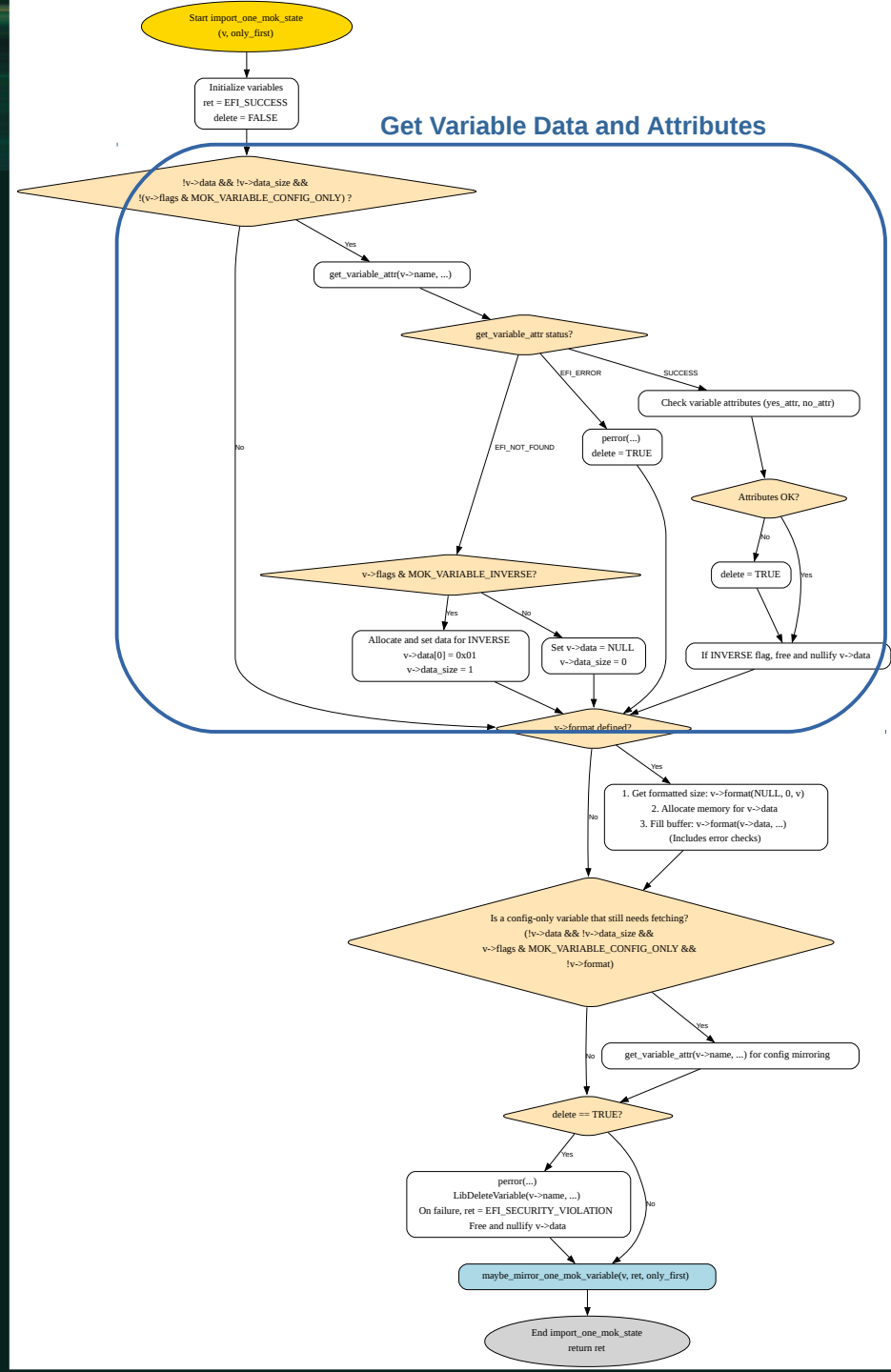
- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the import_mok_state() function and its interactions."



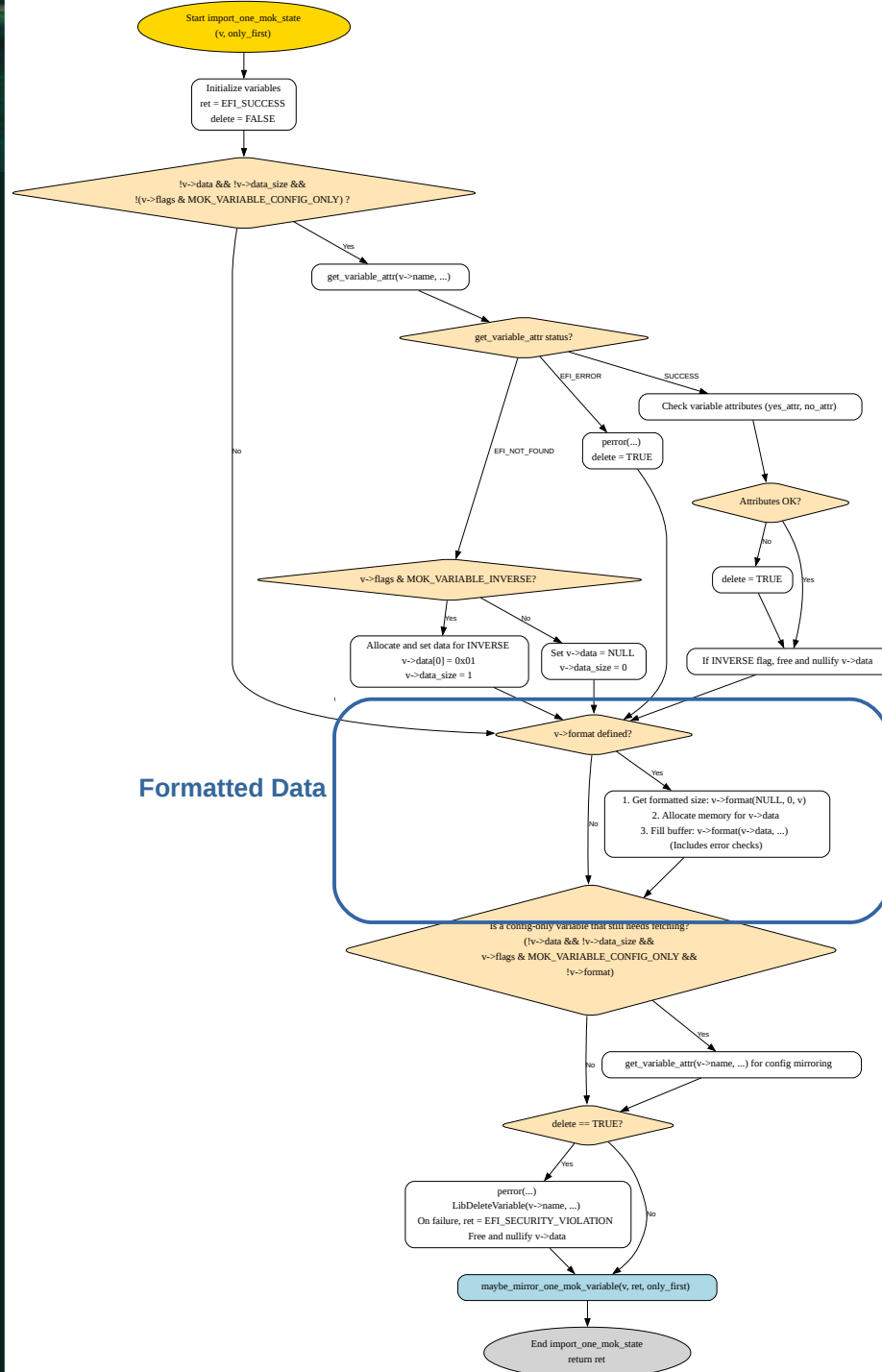
Flow Chart from Gemini scans import_one_mok_s tate()



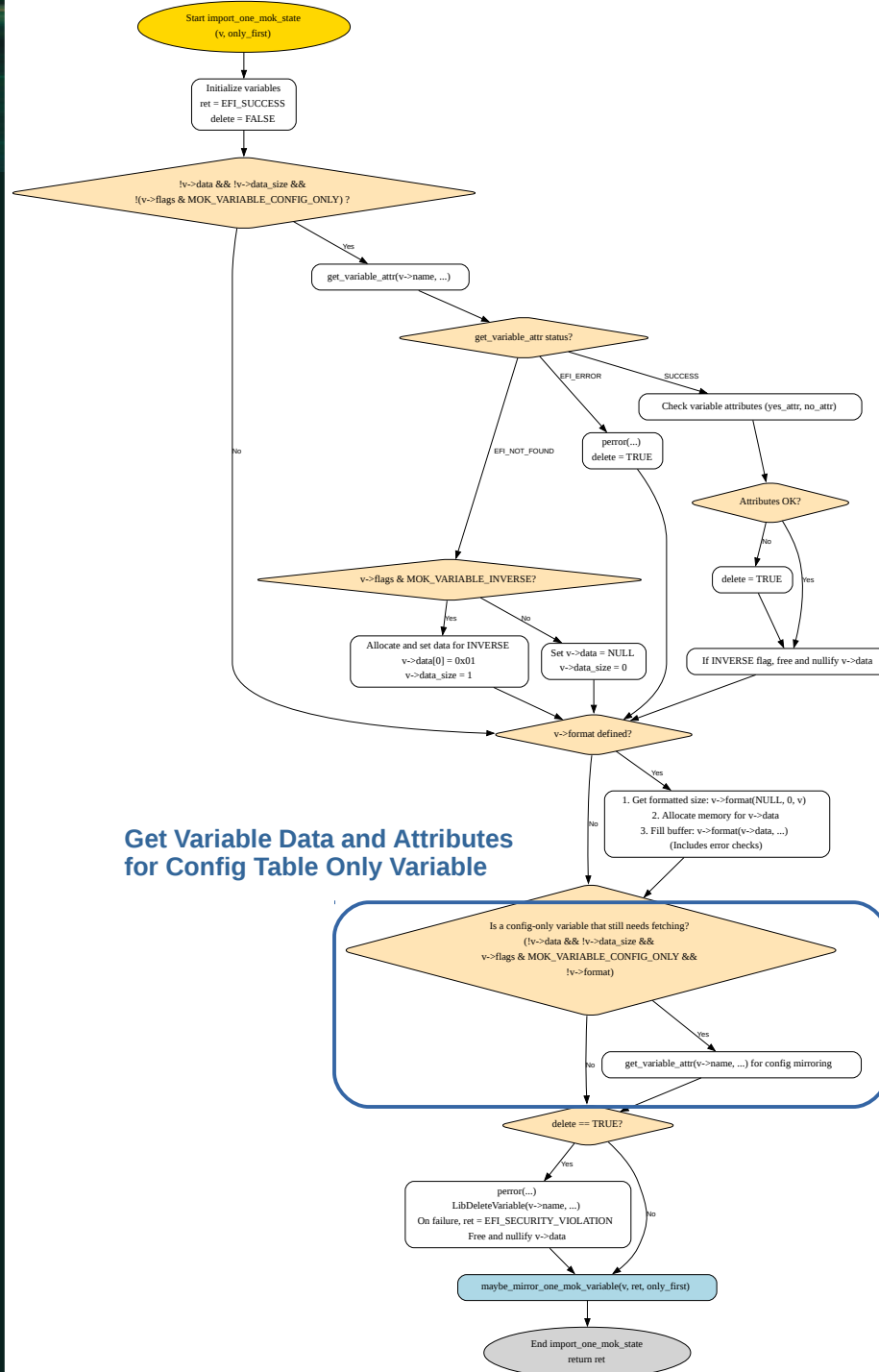
Flow Chart from Gemini scans import_one_mok_s tate()



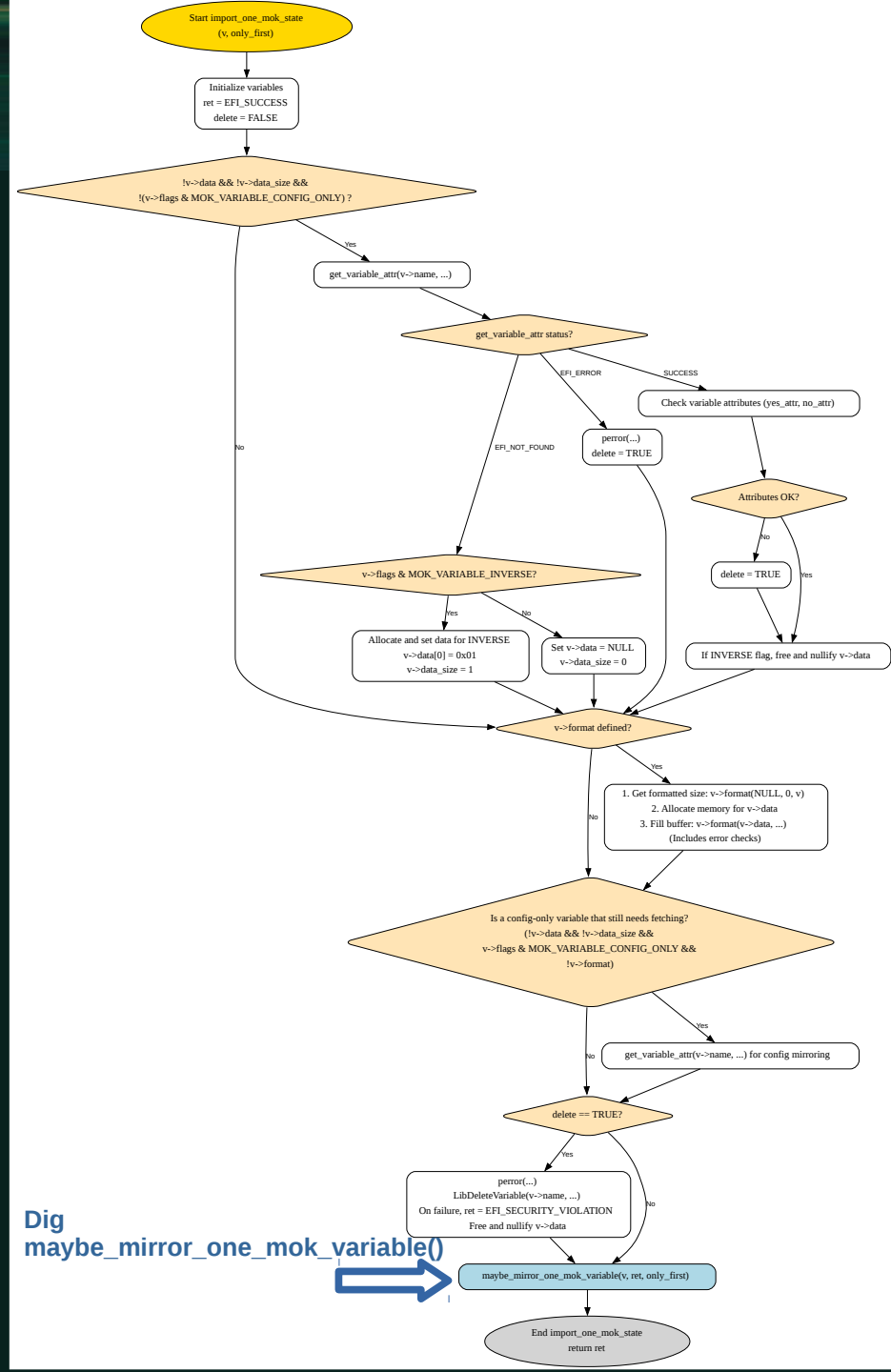
Flow Chart from Gemini scans import_one_mok_s tate()



Flow Chart from Gemini scans import_one_mok_s tate()

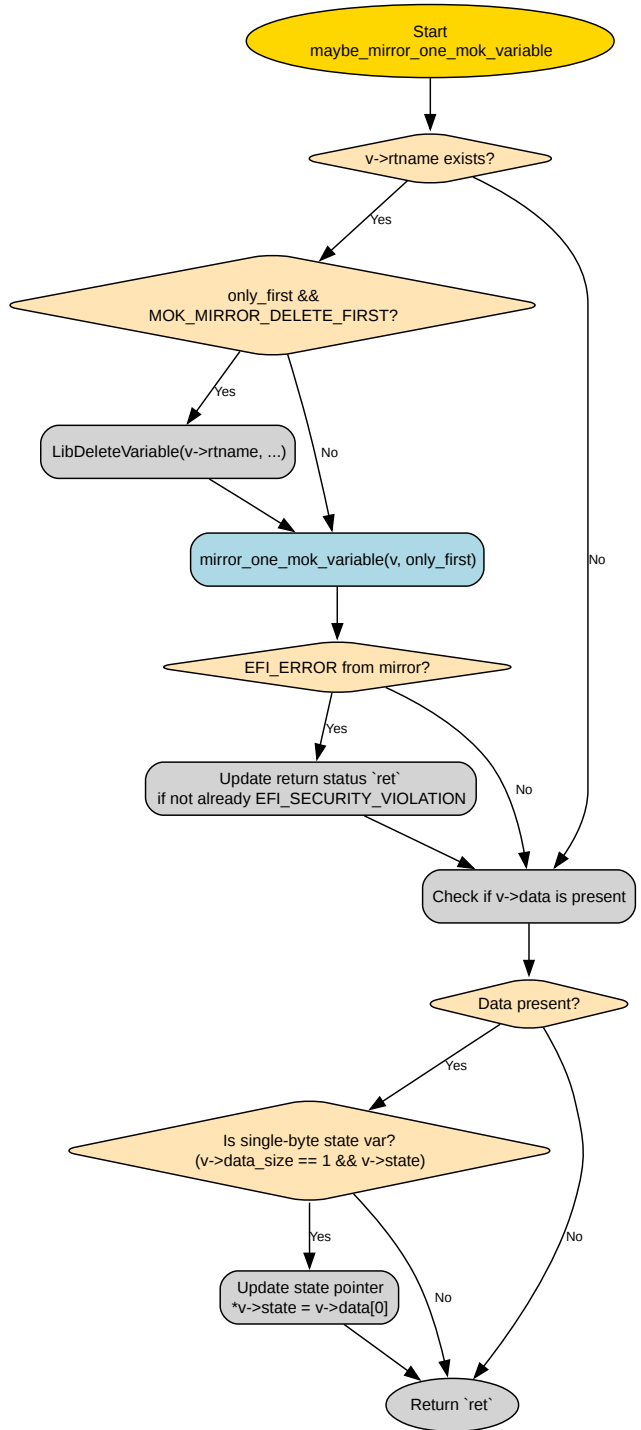


Flow Chart from Gemini scans import_one_mok_s tate()

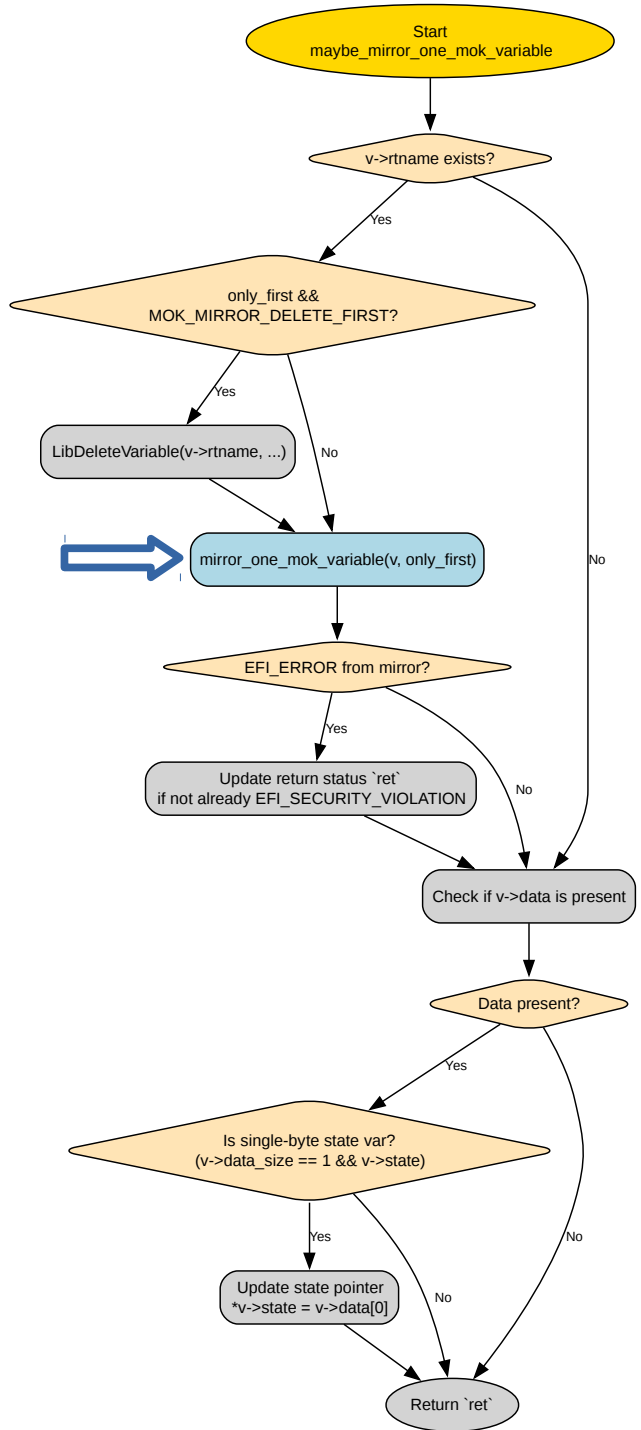


Dig
maybe_mirror_one_mok_variable()

Flow Chart from Gemini scans maybe_mirror_one_mok_variable()



Flow Chart from Gemini scans maybe_mirror_one _mok_variable()



Block Diagram from Gemini scans mirror_one_mo k_variable()

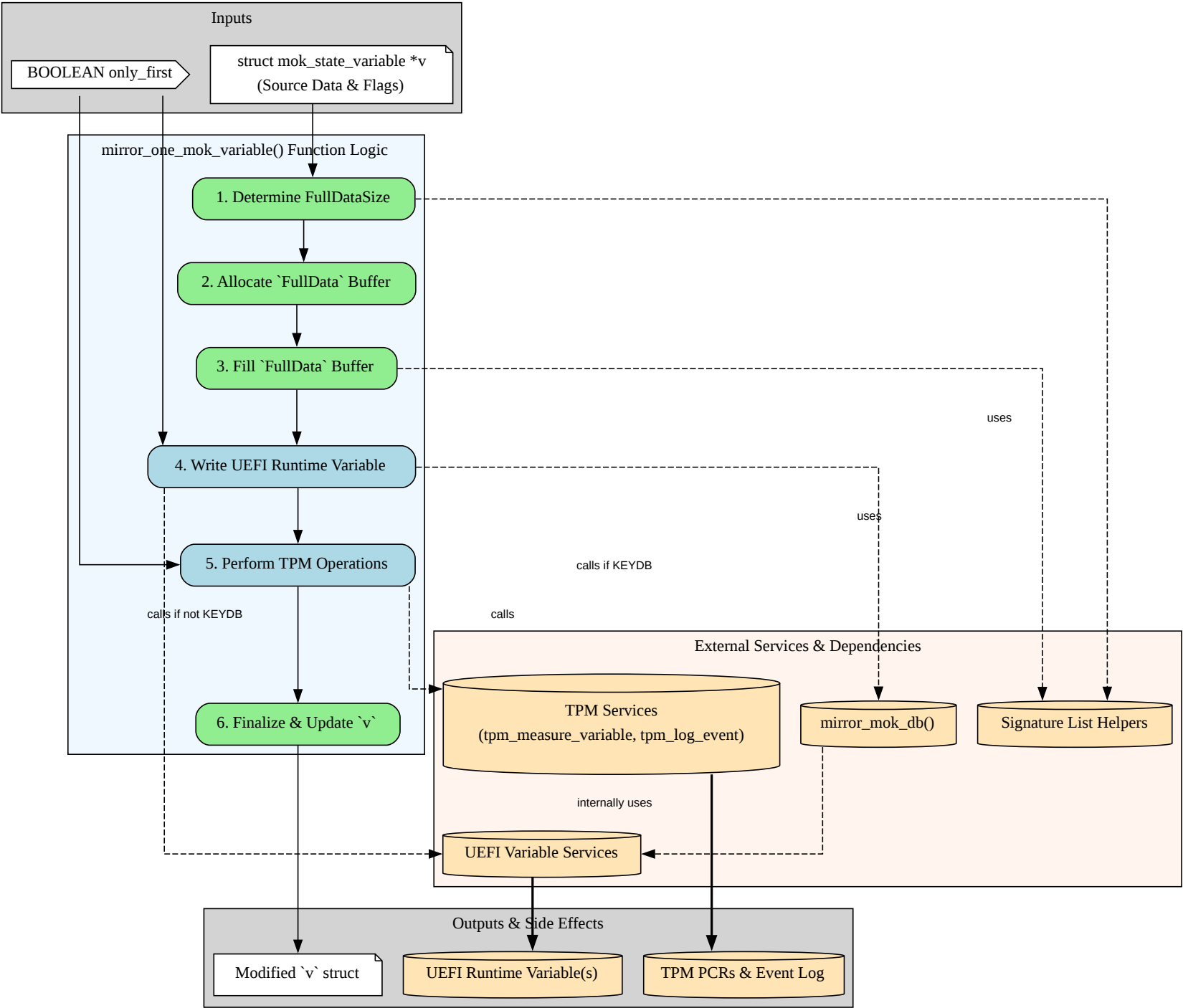
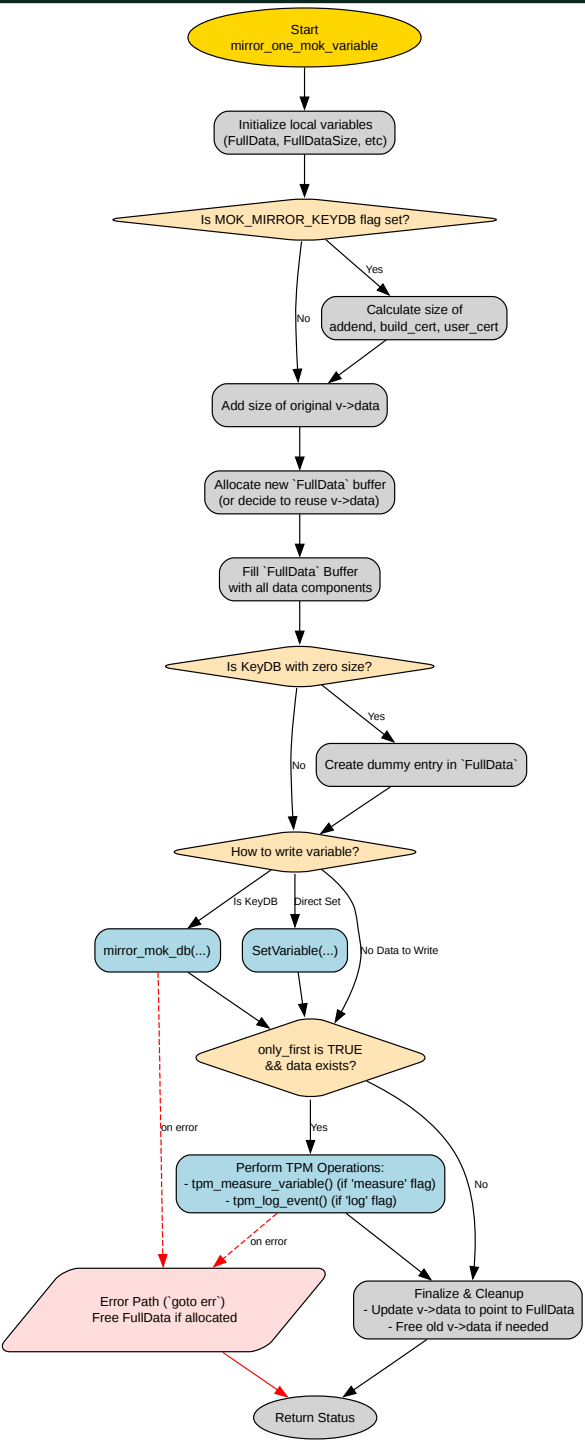
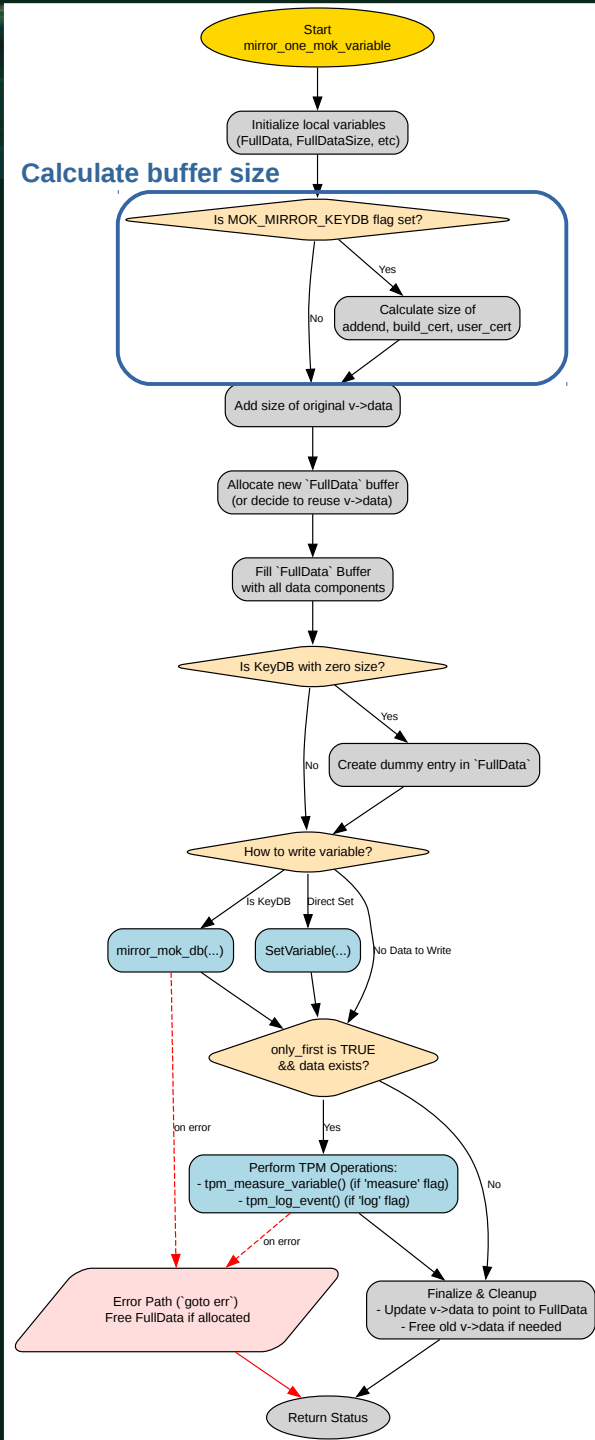


Image generated by Gemini AI

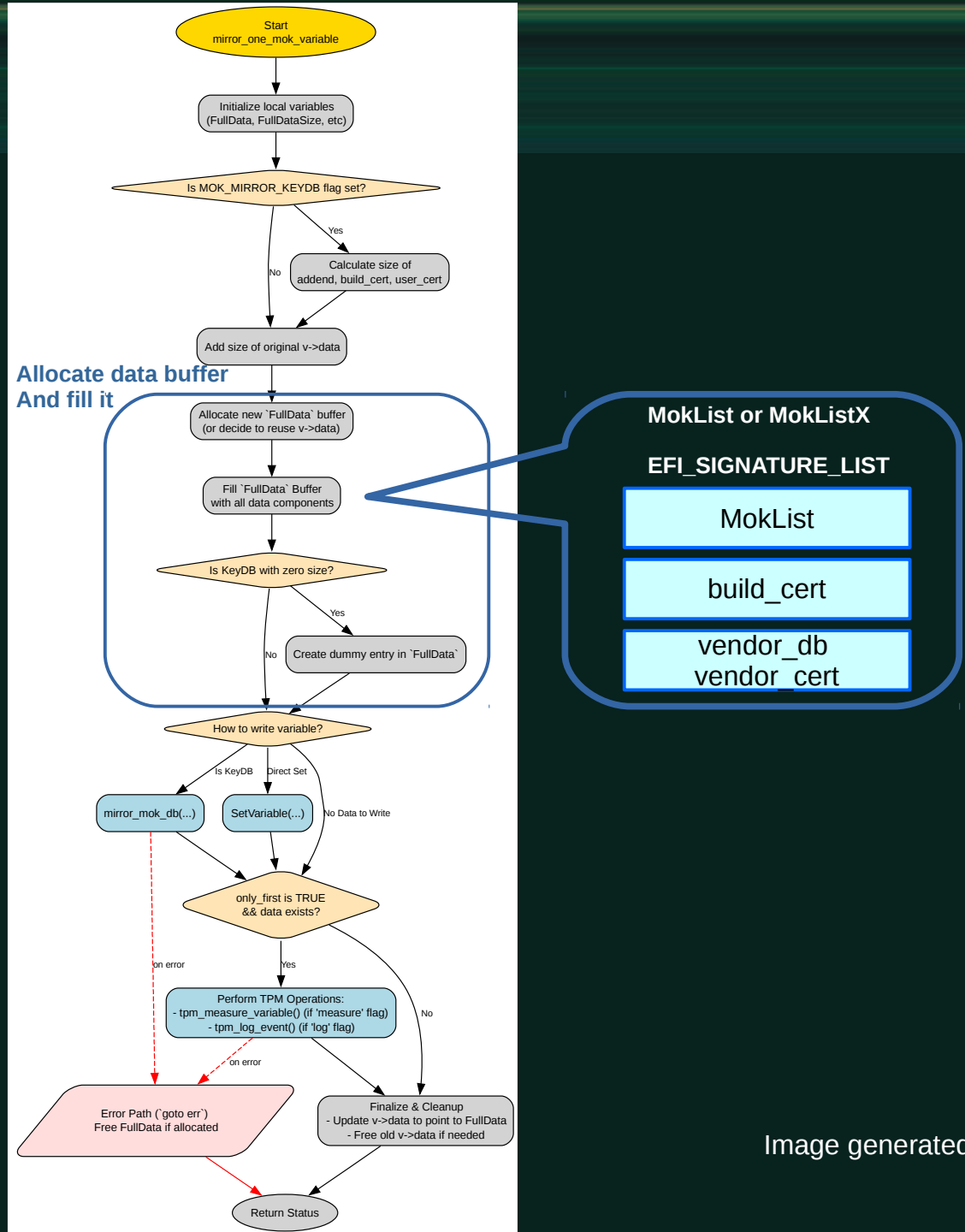
Flow Chart from Gemini scans mirror_one_mok_v variable()



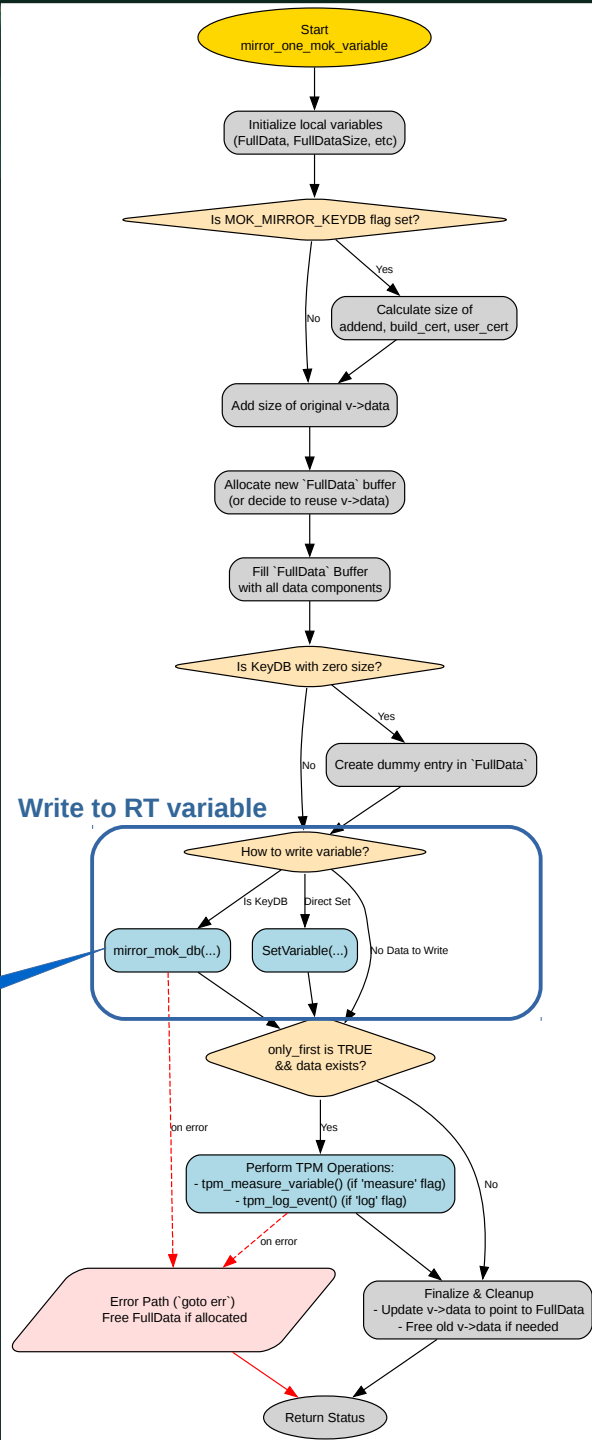
Flow Chart from Gemini scans mirror_one_mok_v variable()



Flow Chart from Gemini scans mirror_one_mok_v variable()

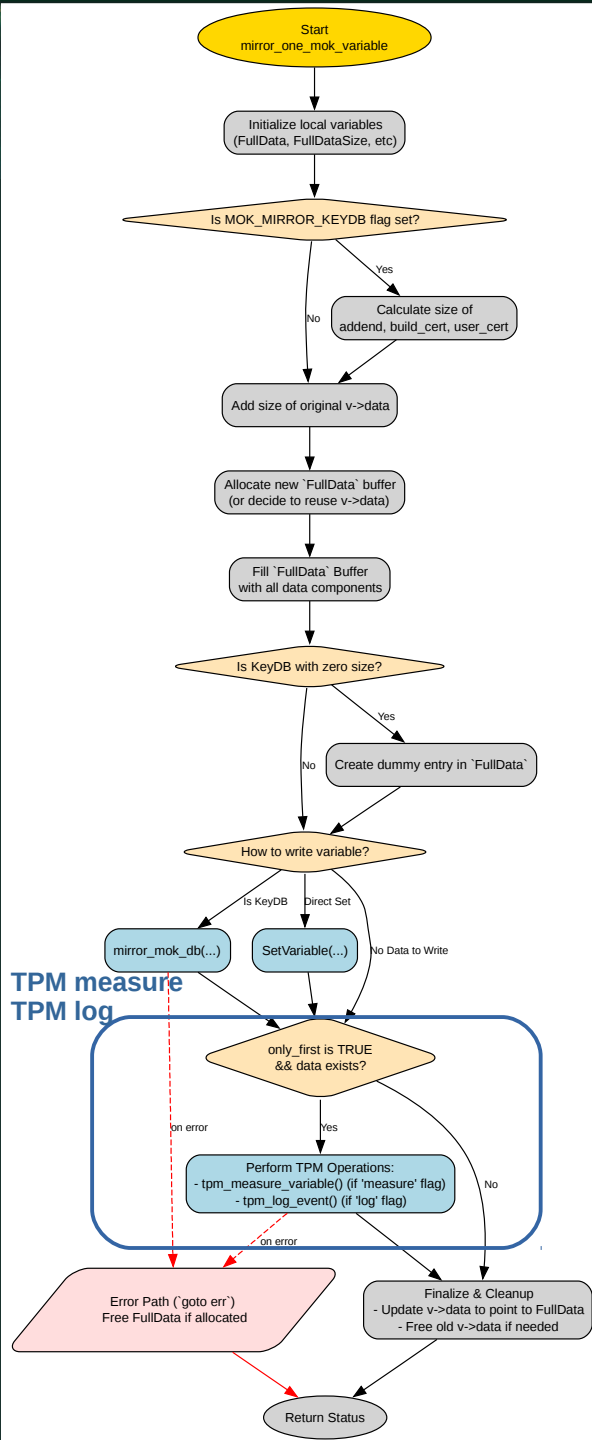


Flow Chart from Gemini scans mirror_one_mok_v variable()

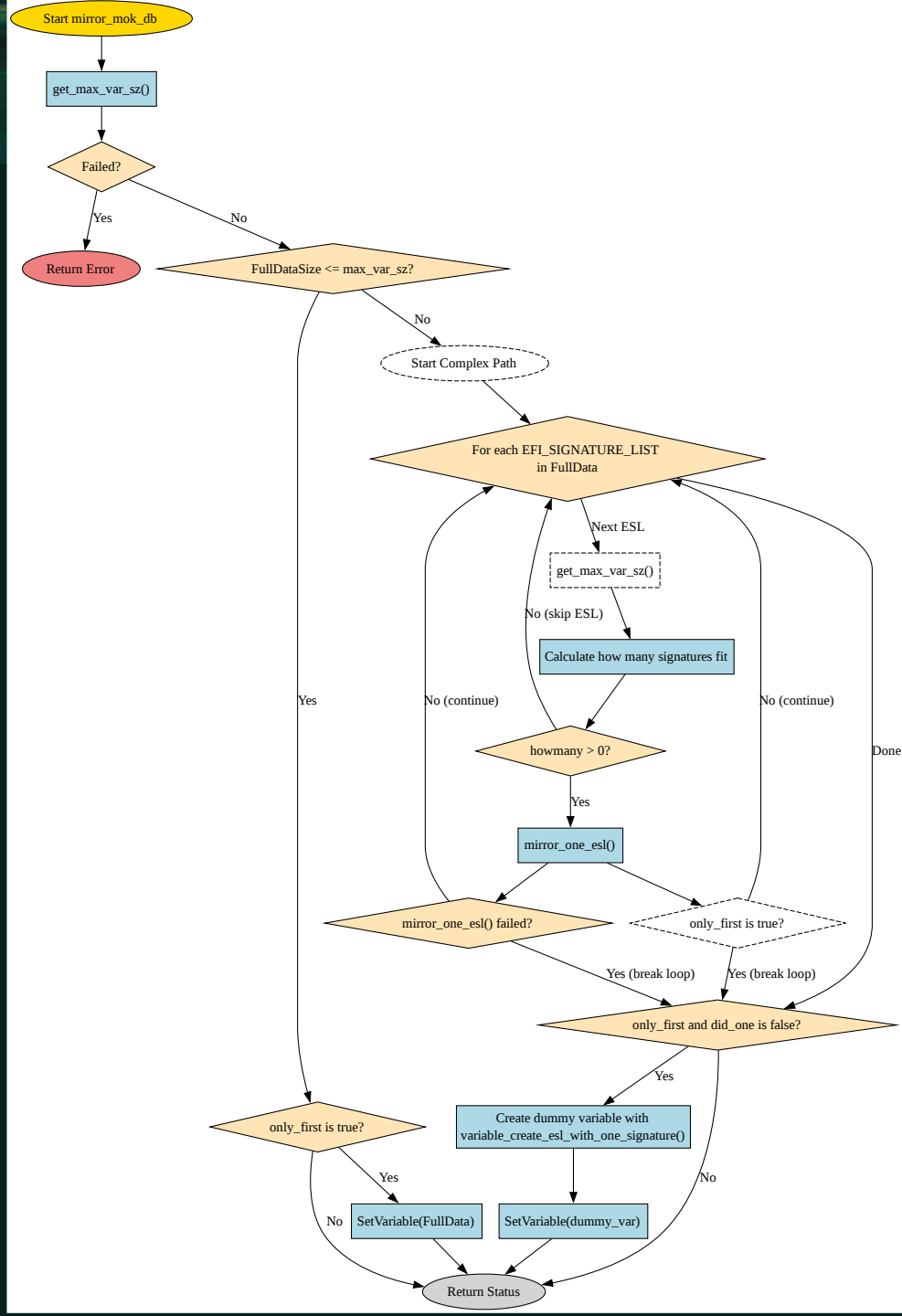


MokList or MokListX

Flow Chart from Gemini scans mirror_one_mok_v variable()



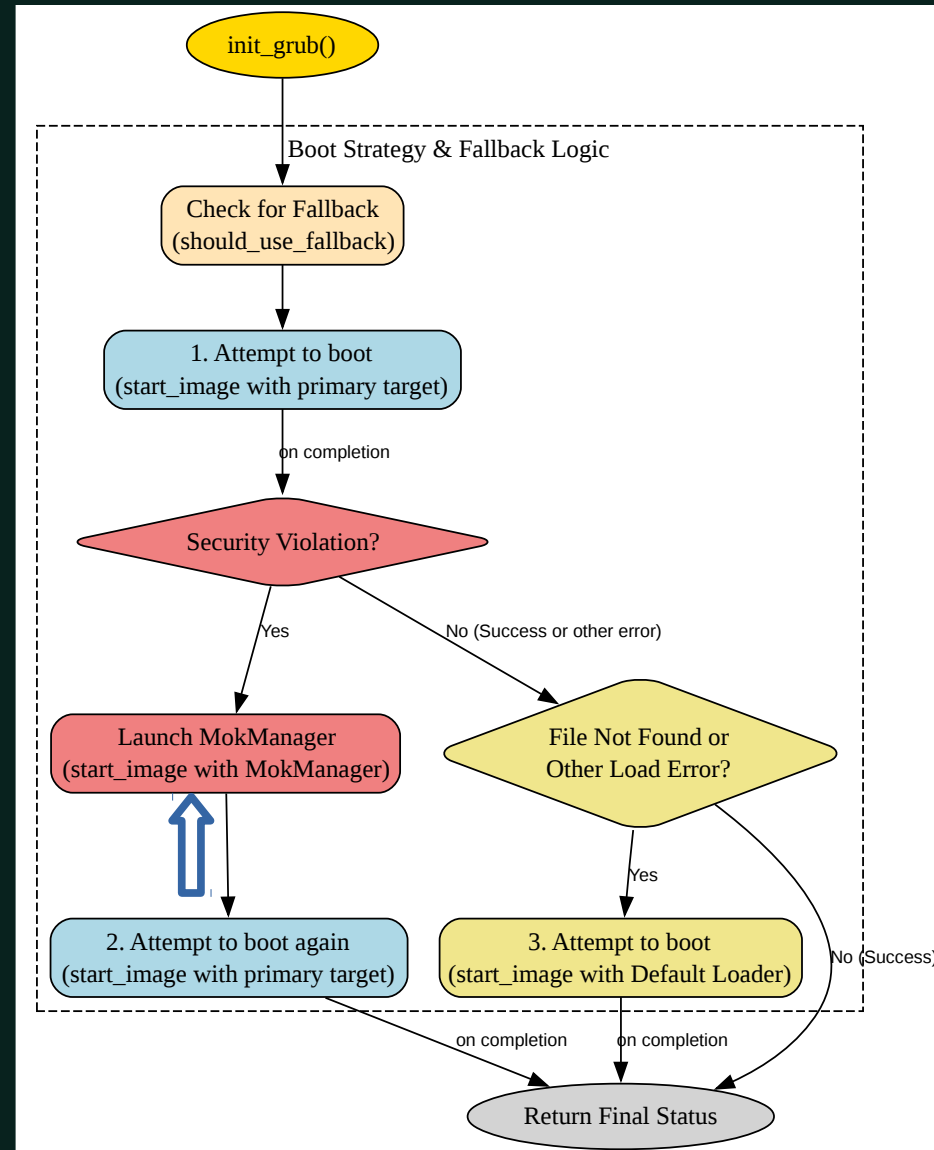
Flow Chart from Gemini scans mirror_mok_db()



MokManager

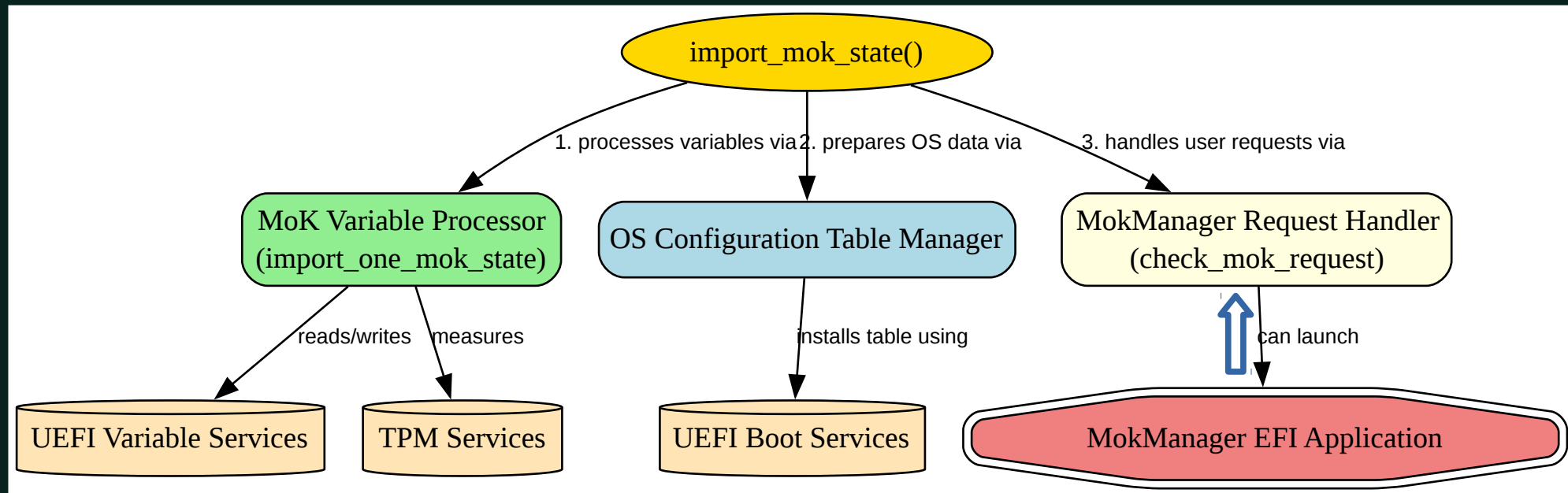


MokManager in init_grub()



MokManager in import_mok_state()

- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the import_mok_state() function and its interactions."



Flow Chart from Gemini scans check_mok_request()

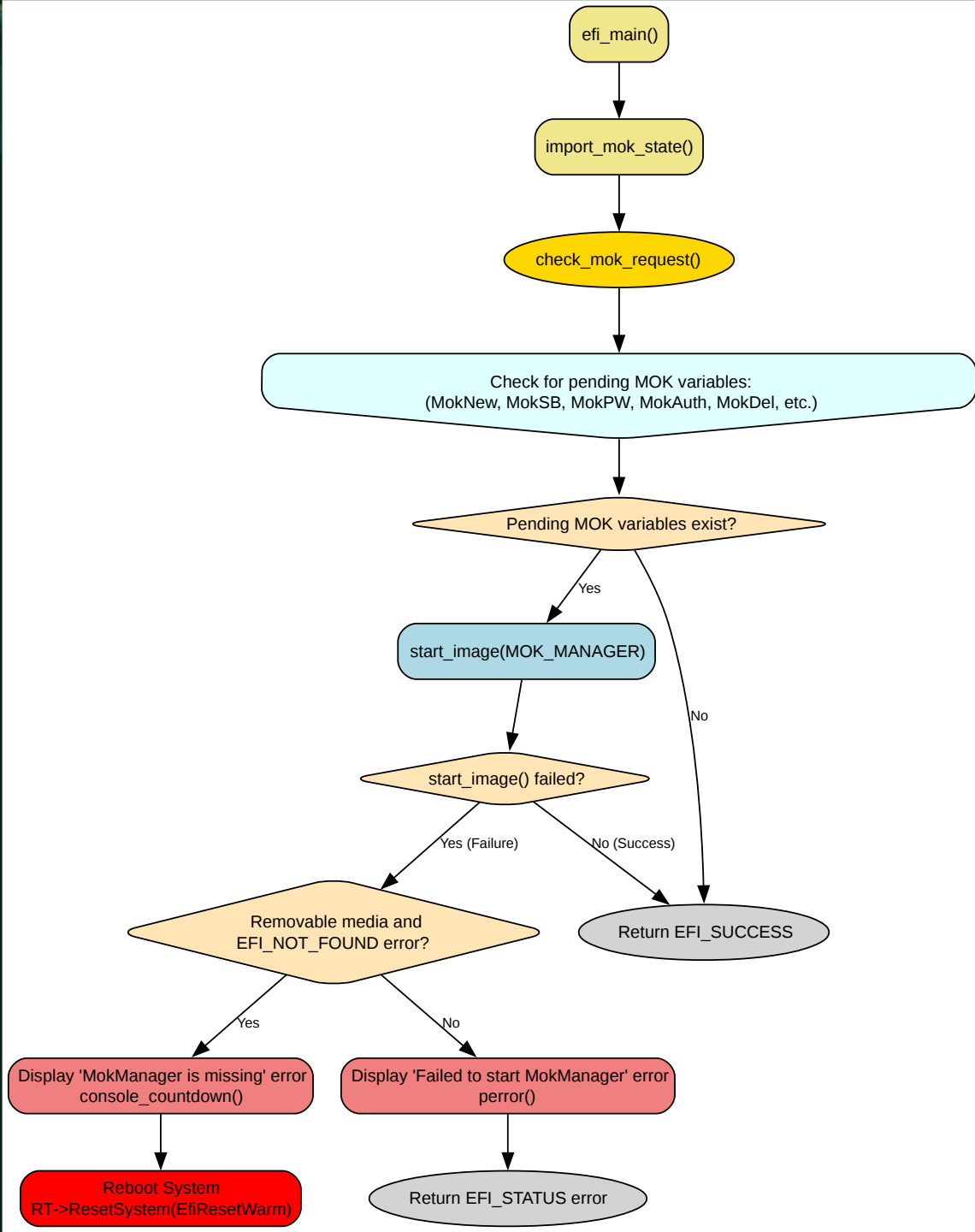
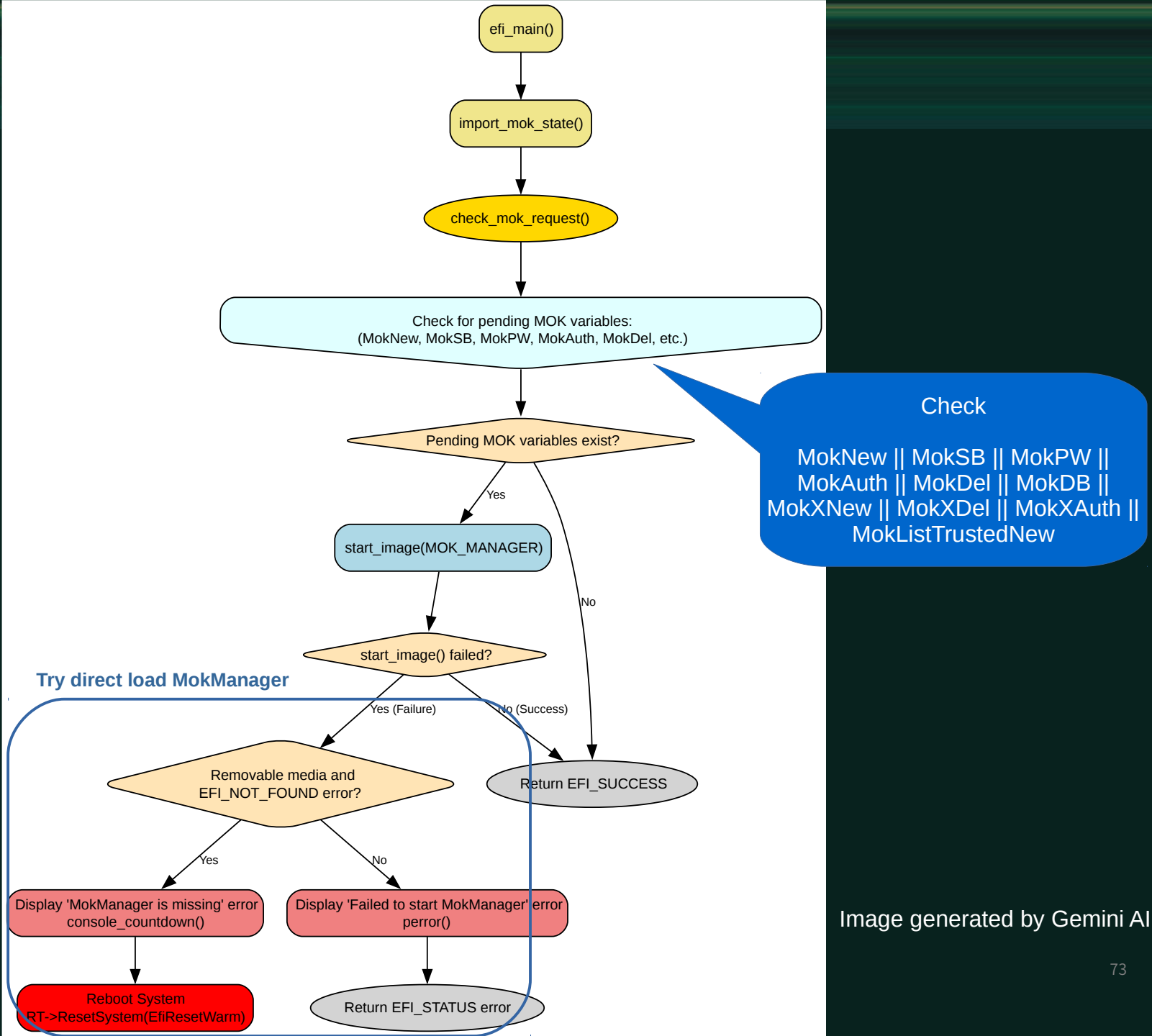


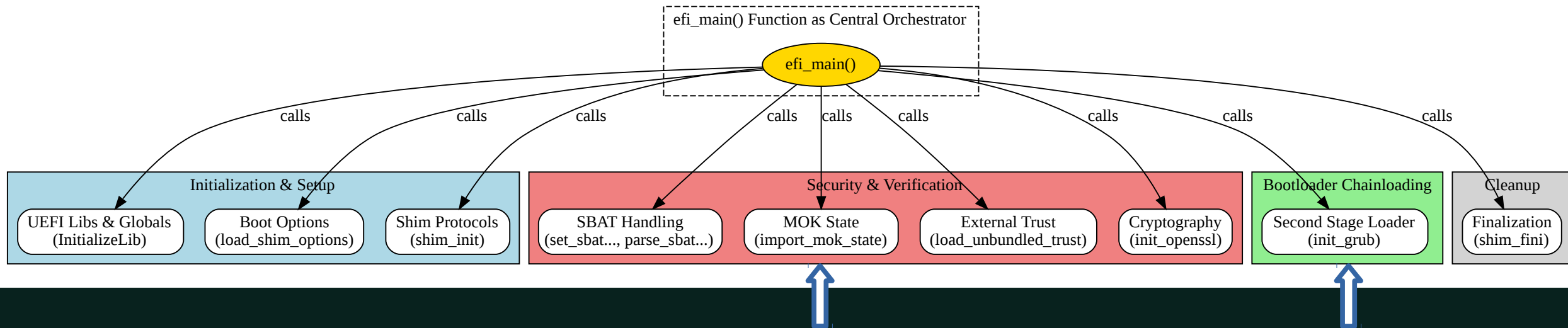
Image generated by Gemini AI

Flow Chart from Gemini scans check_mok_request()

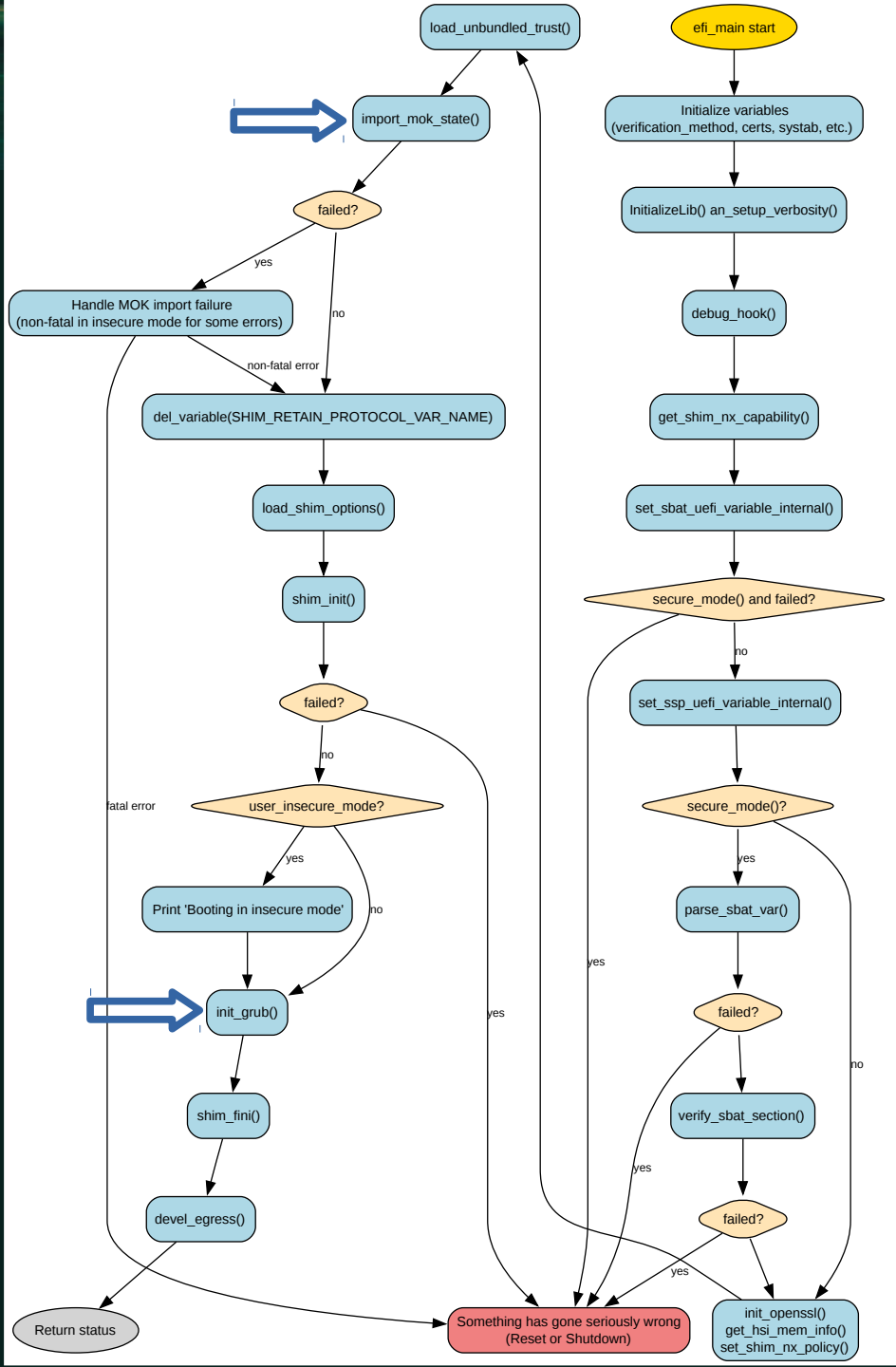


The Order of `import_mok_state()` and `init_grub()`

- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at `~/tmp/gemini/`. Then, create the block diagram with a focus on the **`efi_main()`** function and its interactions."



The order of import_mok_state() and init_grub()

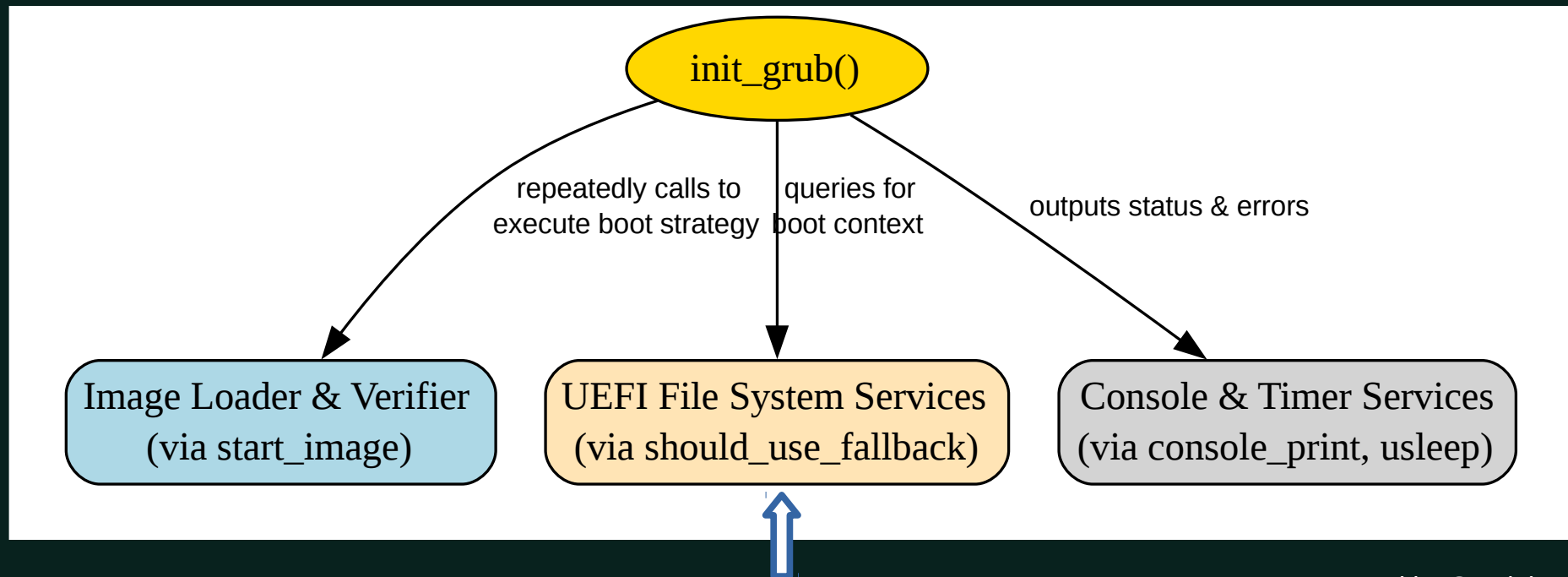


Fallback

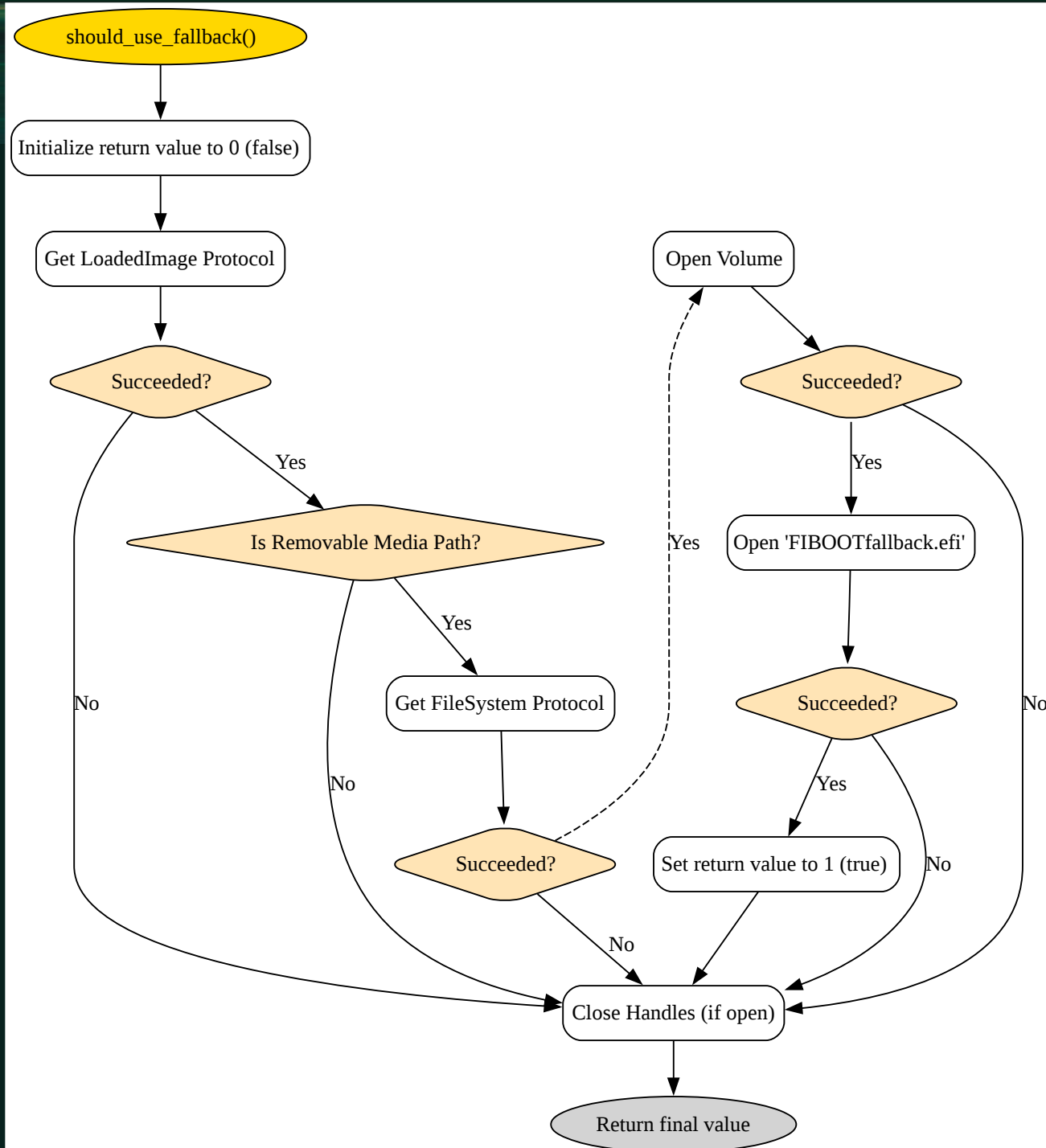


The block diagram related to fallback

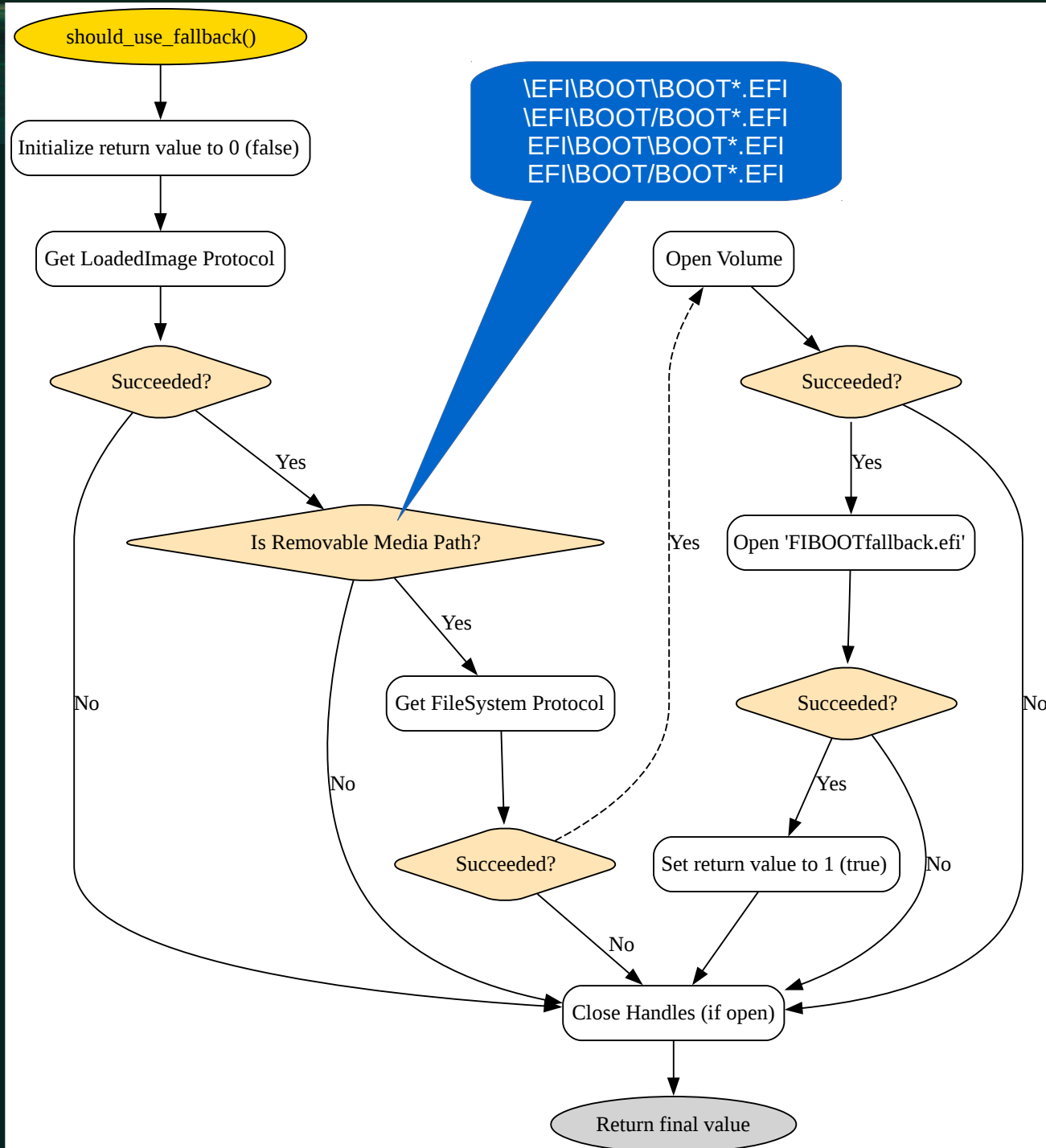
- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the **init_grub()** function and its interactions."



Flow Chart from Gemini scans should_use _fallback()



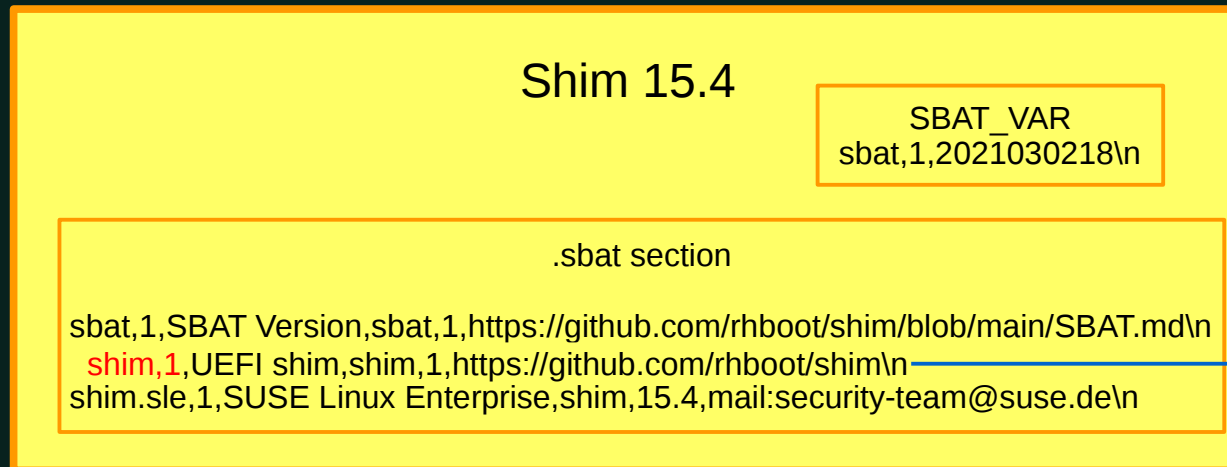
Flow Chart from Gemini scans should_use _fallback()



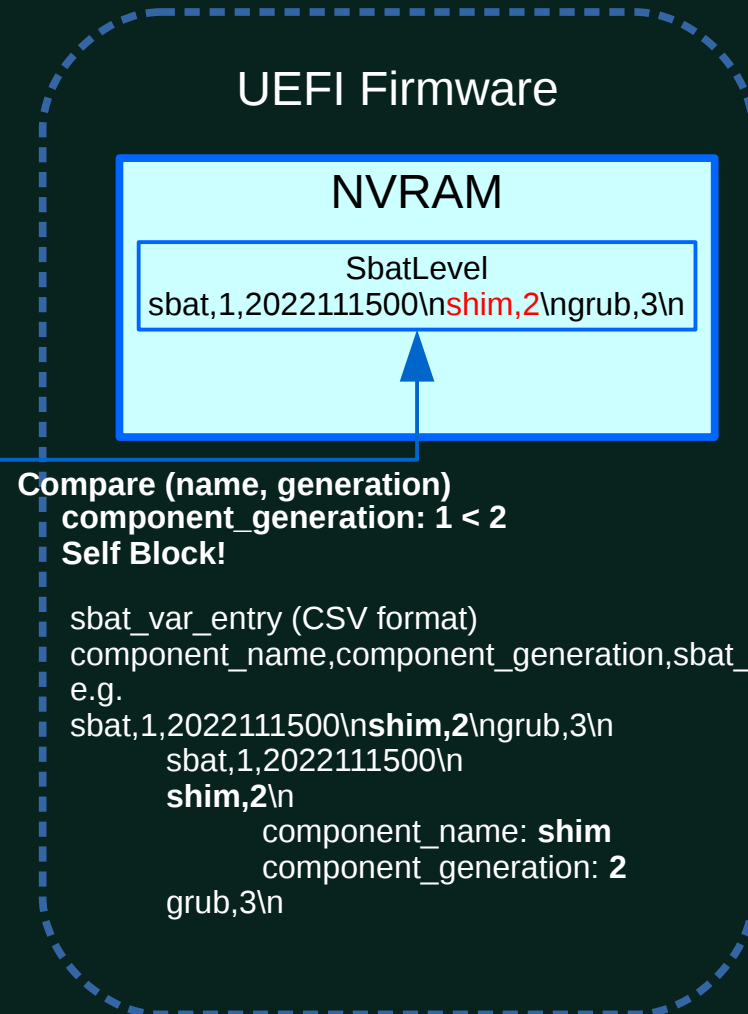
SBAT



Shim SBAT check



sbat_section_entry (CSV format)
component_name,component_generation,
vendor_name,vendor_package_name,vendor_version,vendor_url
e.g.
sbat,1,SBAT Version,sbat,1,https://github.com/rhboot/shim/blob/main/SBAT.md\n
shim,1,UEFI shim,shim,1,https://github.com/rhboot/shim\n
component_name: **shim**
component_generation: **1**
shim.sle,1,SUSE Linux Enterprise,shim,15.4,mail:security-team@suse.de\n

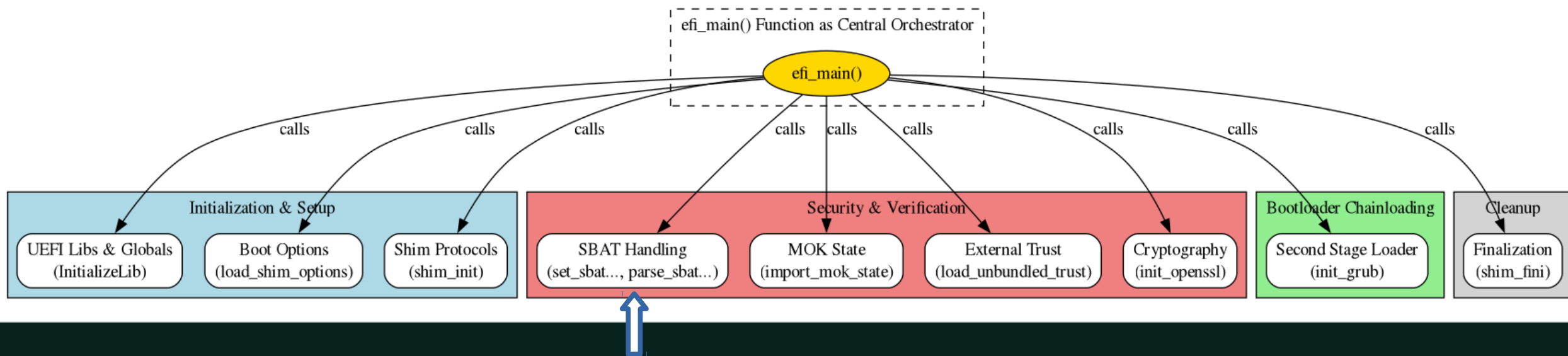


Compare (name, generation)
component_generation: 1 < 2
Self Block!

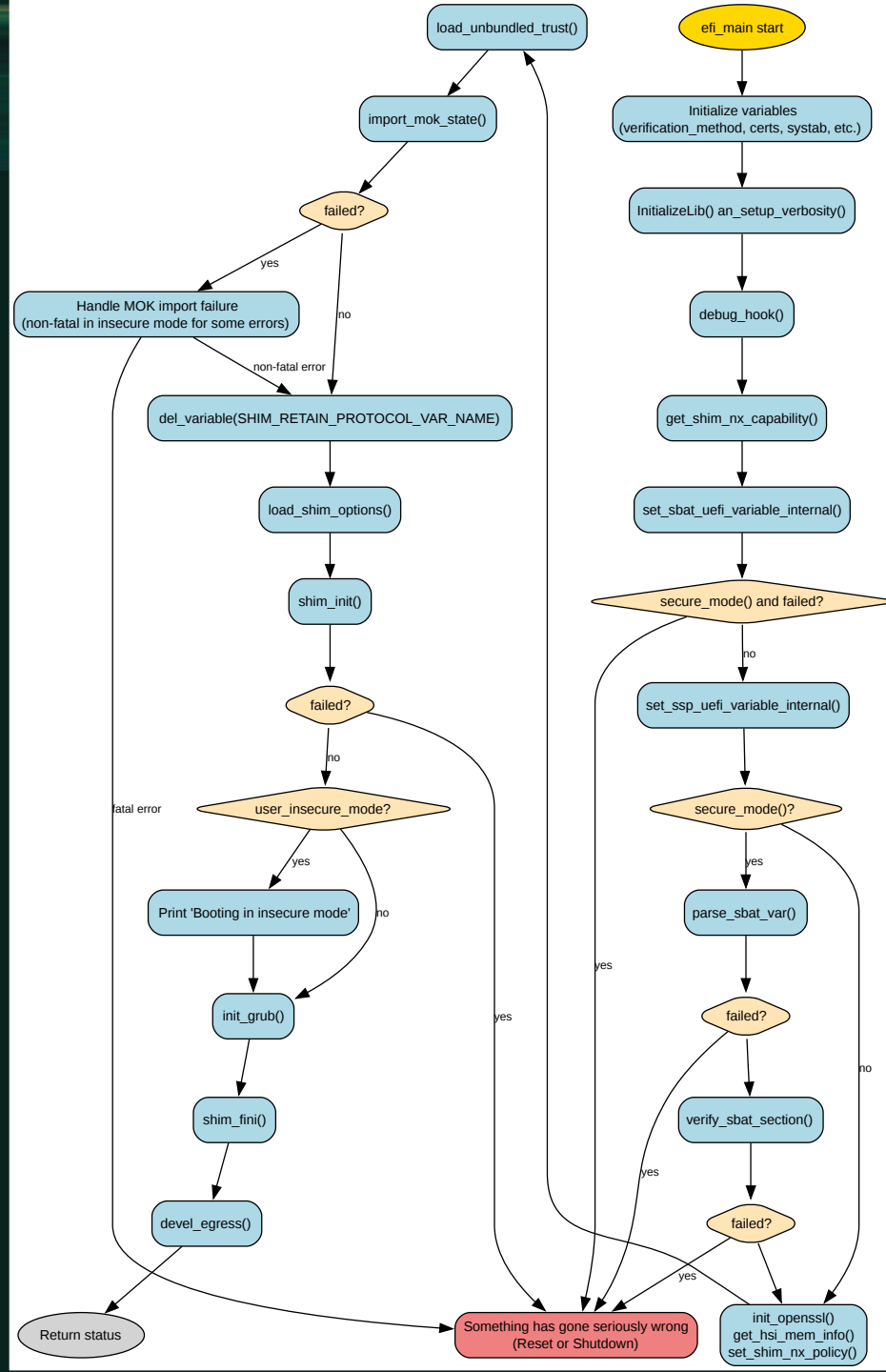
sbat_var_entry (CSV format)
component_name,component_generation,sbat_datestamp
e.g.
sbat,1,2022111500\nshim,2\ngrub,3\n
sbat,1,2022111500\n
shim,2\n
component_name: **shim**
component_generation: **2**
grub,3\n

SBAT in efi_main()

- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the **efi_main()** function and its interactions."

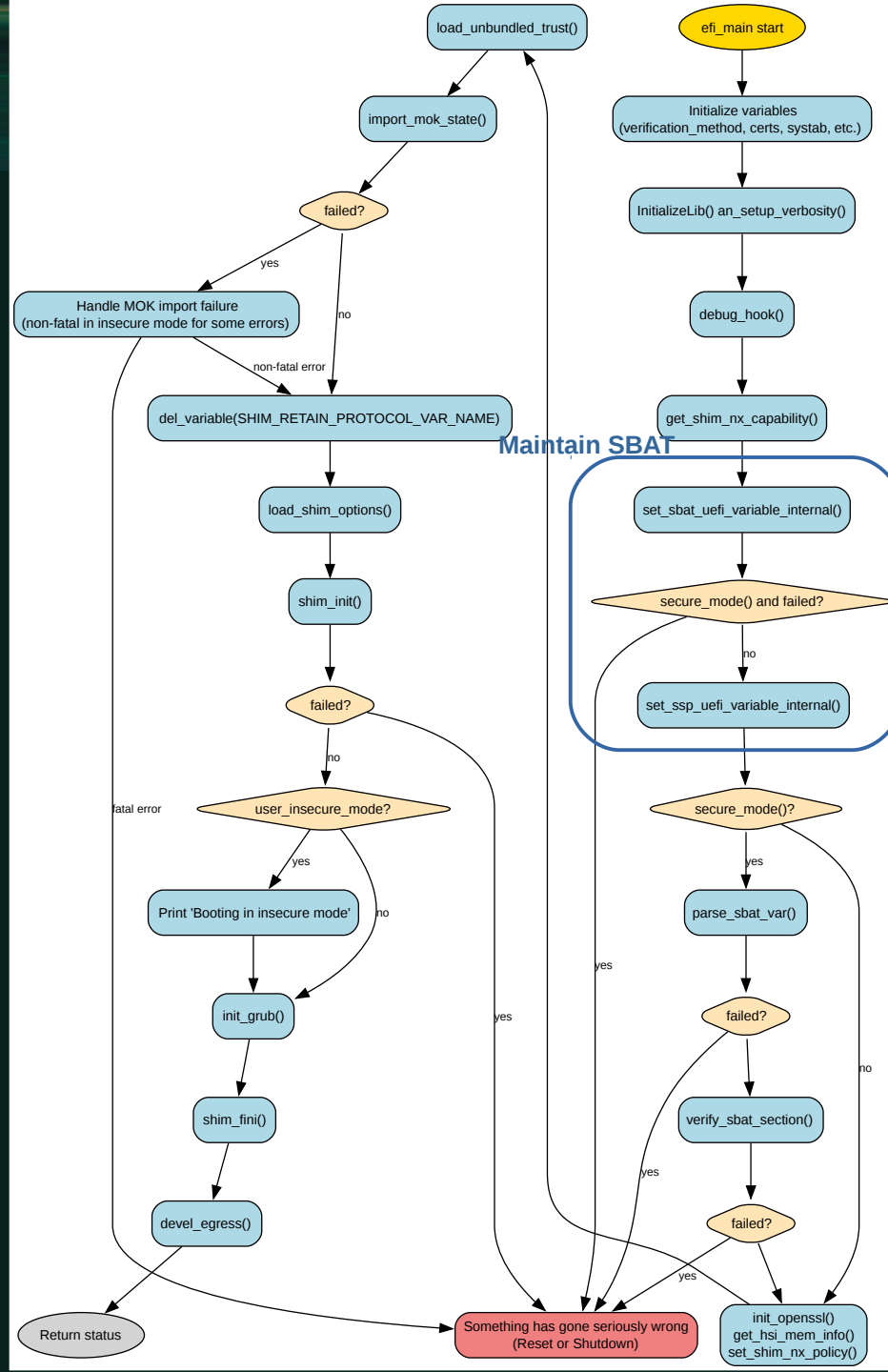


SBAT in efi_main()

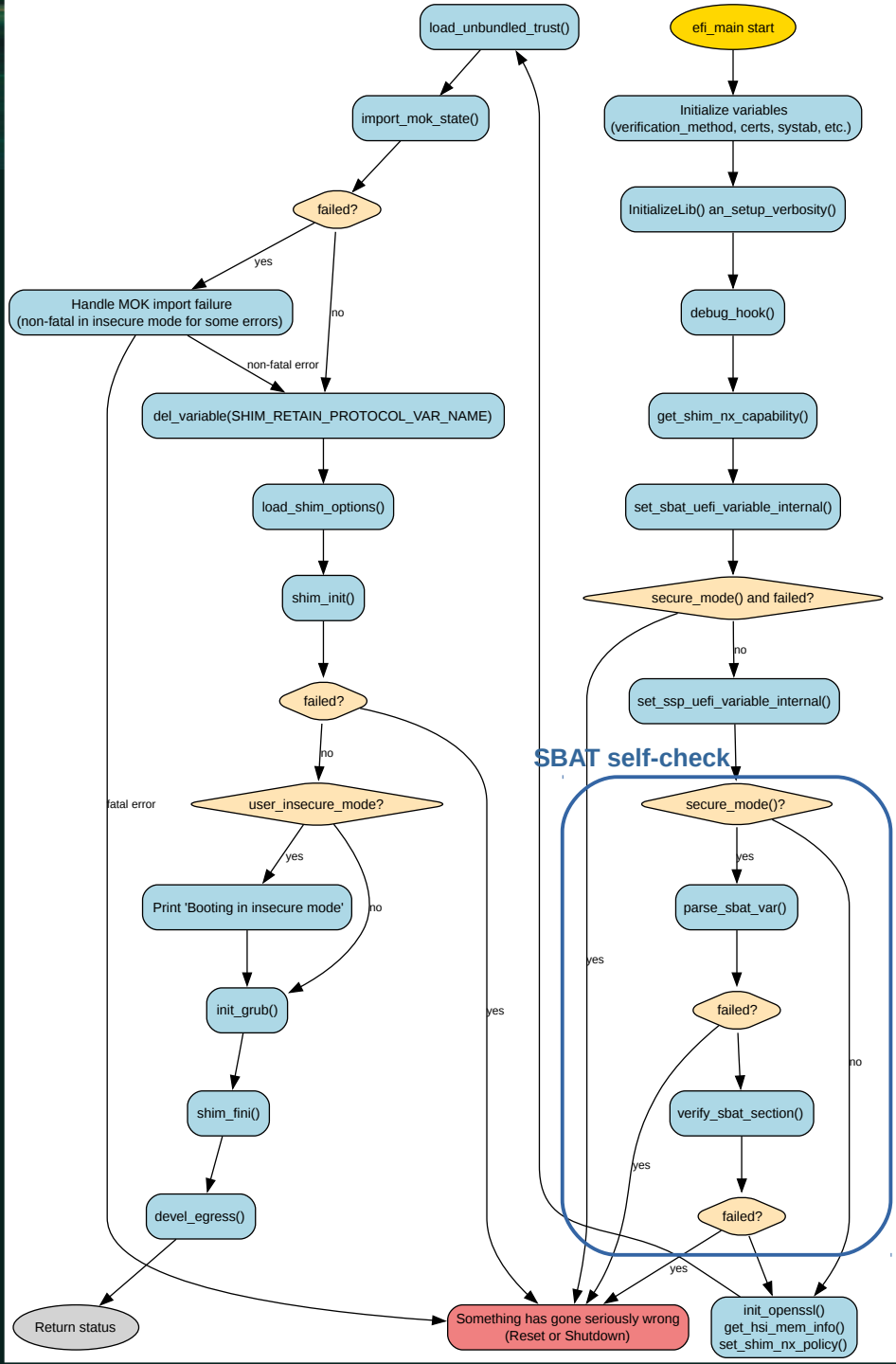


Use Cases

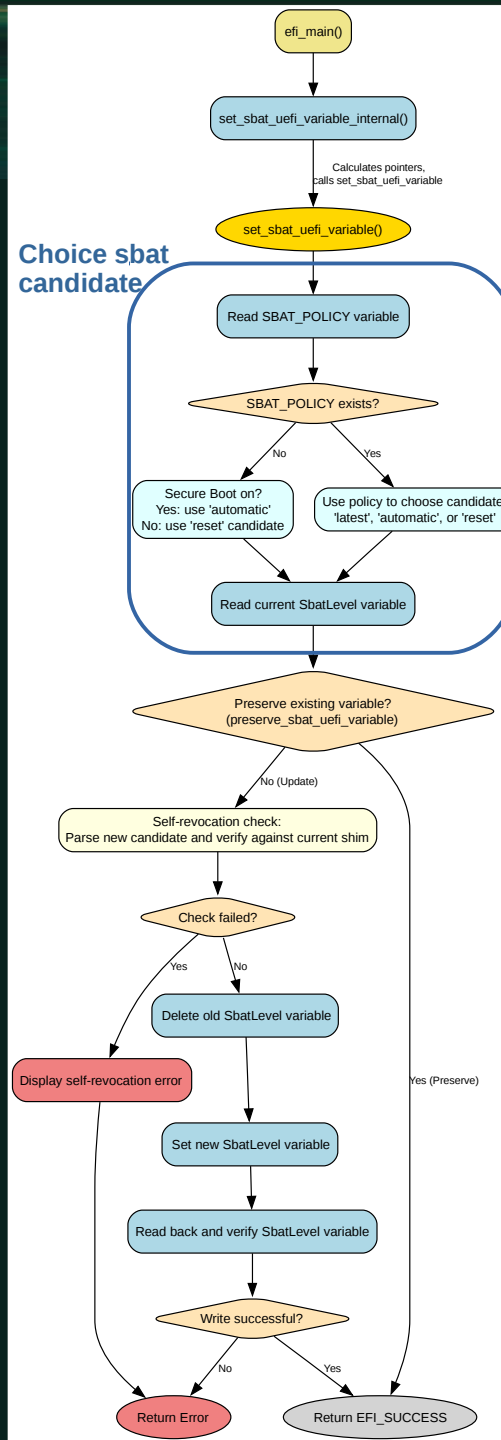
SBAT in efi_main()



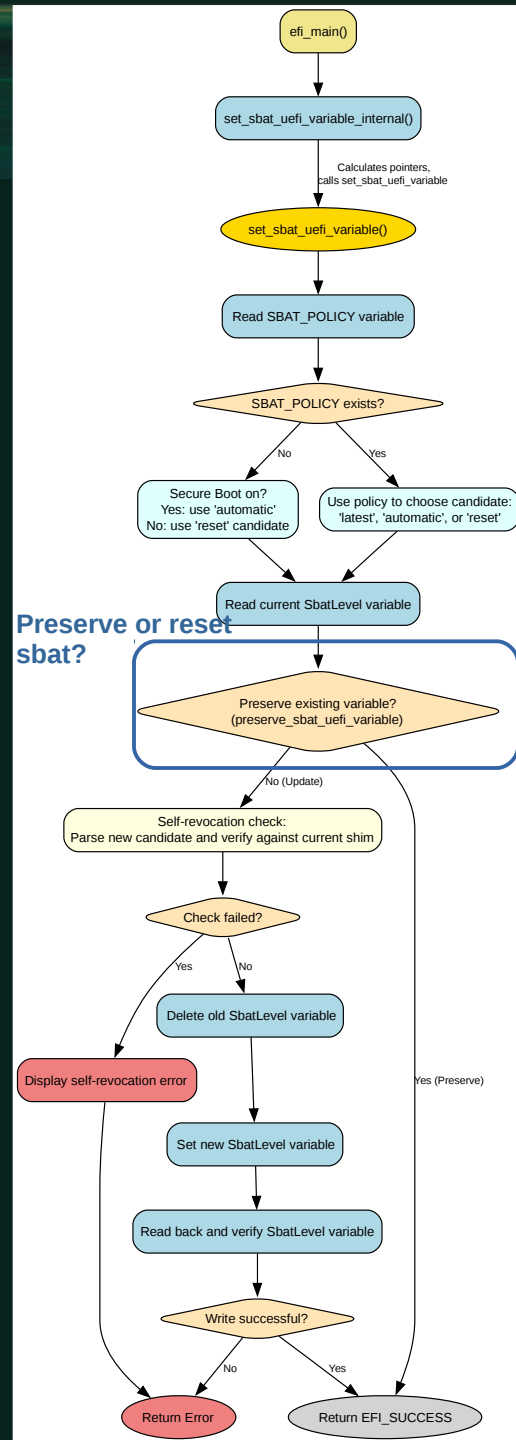
SBAT in efi_main()



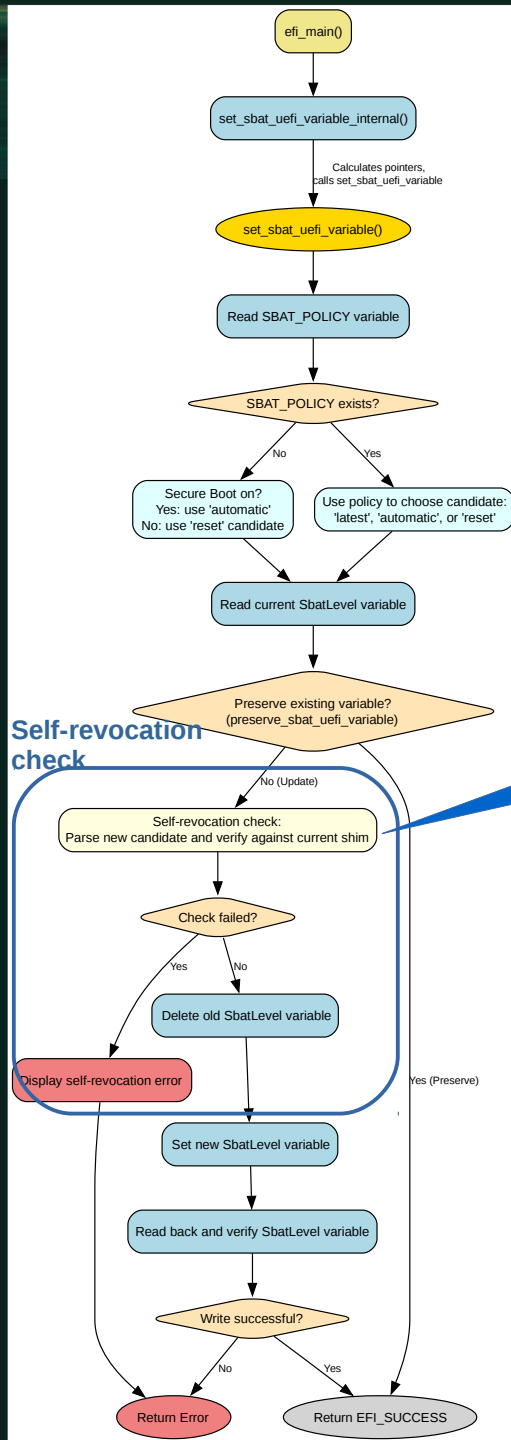
Flow Chart from Gemini scans set_sbat_uefi_variable_ internal()



Flow Chart from Gemini scans set_sbat_uefi_variable_ internal()



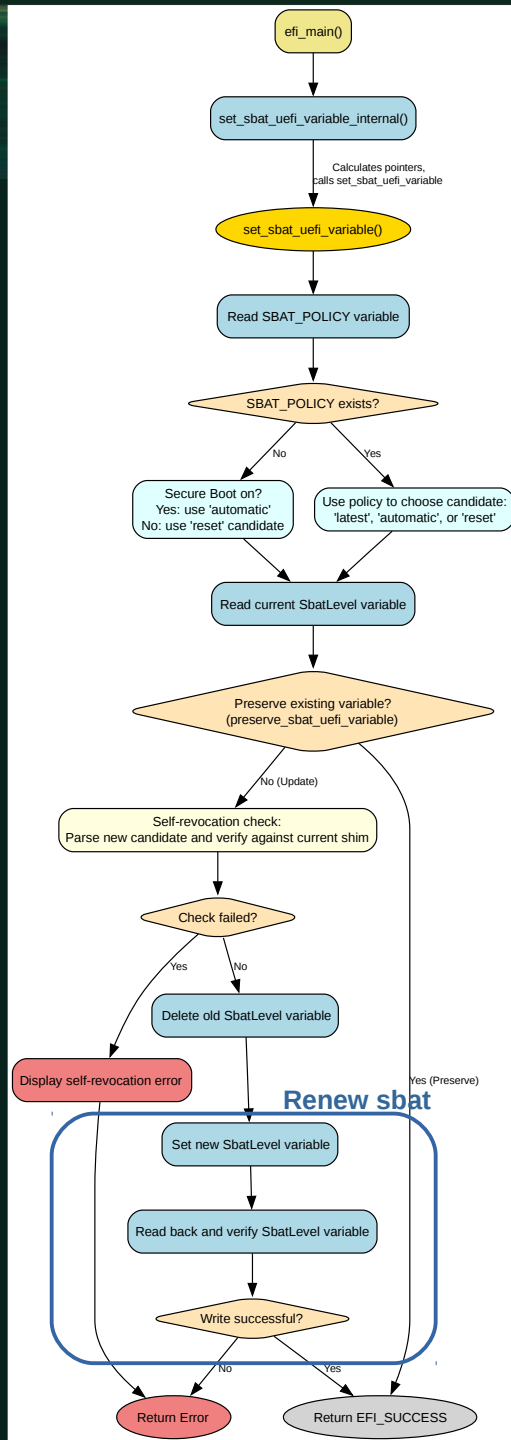
Flow Chart from Gemini scans set_sbat_uefi_variable_ internal()



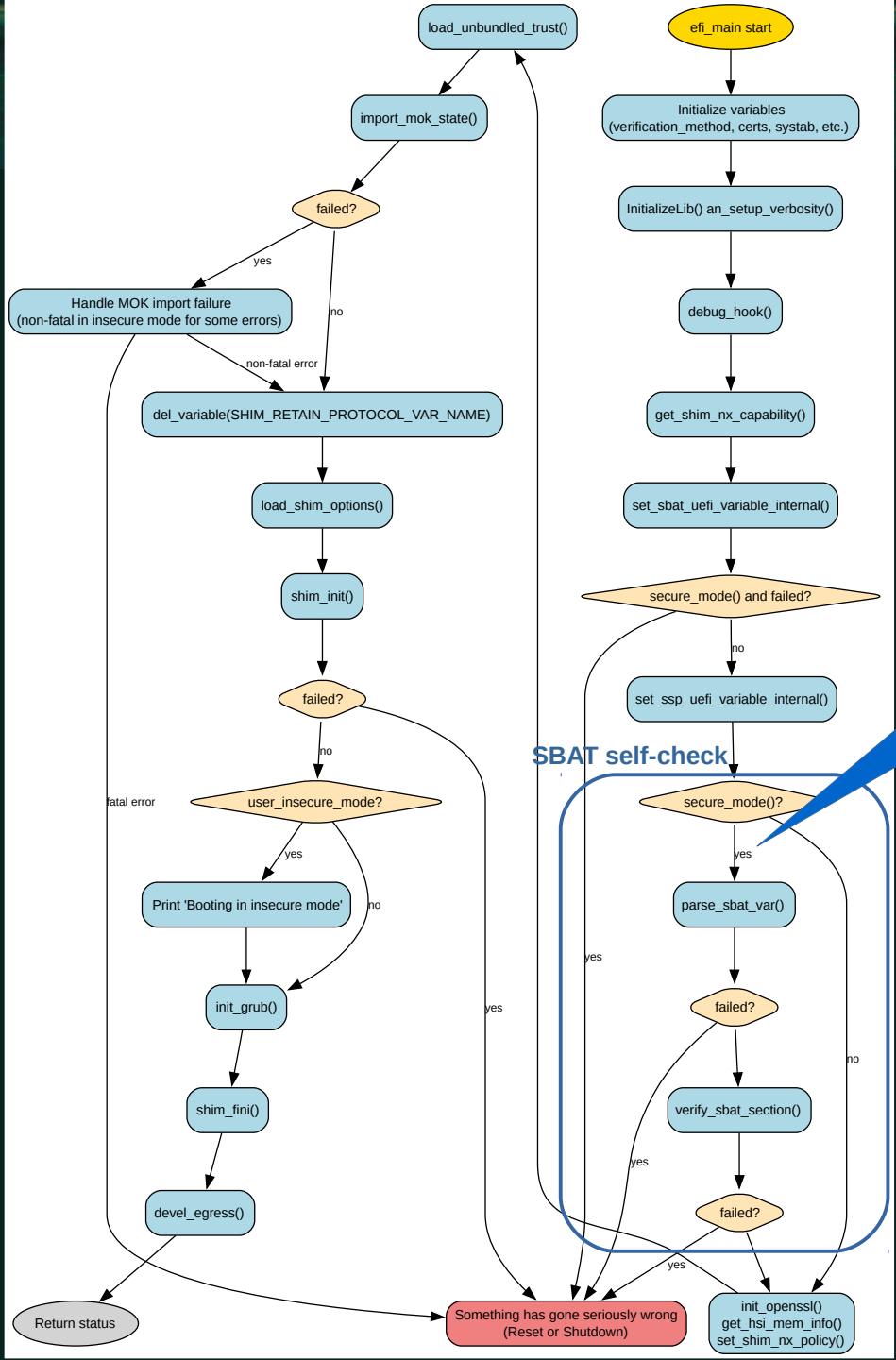
`parse_sbat_var(&sbat_var, sbat_var_candidate)`
`verify_sbat_section(sbat_start, sbat_end - sbat_start - 1)`

Before writing a new sbat, we need to ensure that the new sbat will NOT block the current shim.

Flow Chart from Gemini scans set_sbat_uefi_variable_ internal()



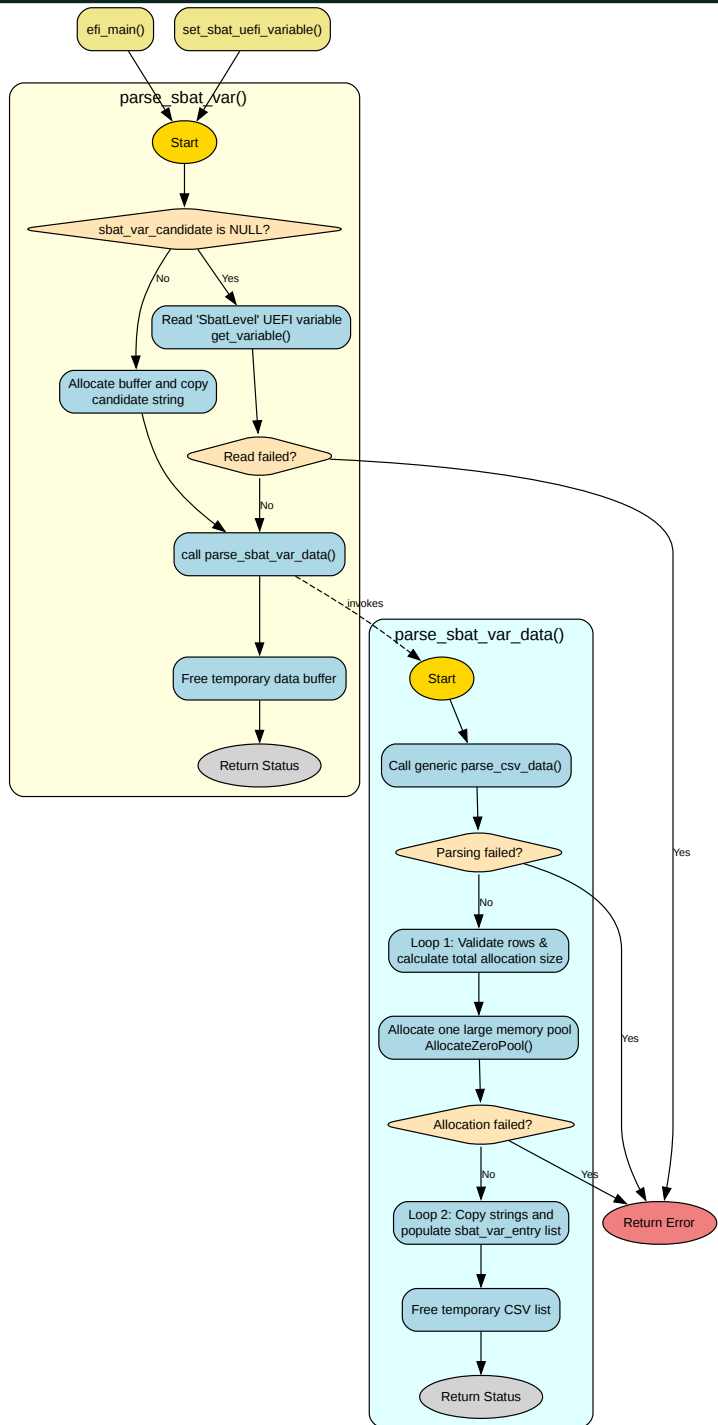
SBAT in efi_main()



INIT_LIST_HEAD(&sbat_var)

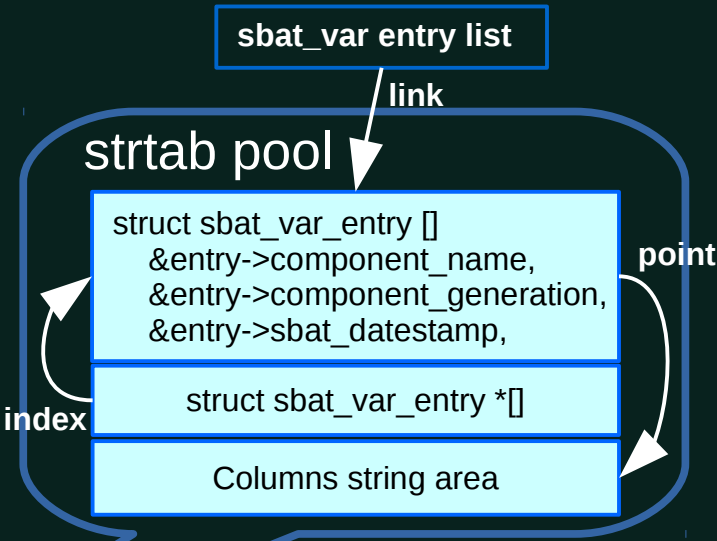
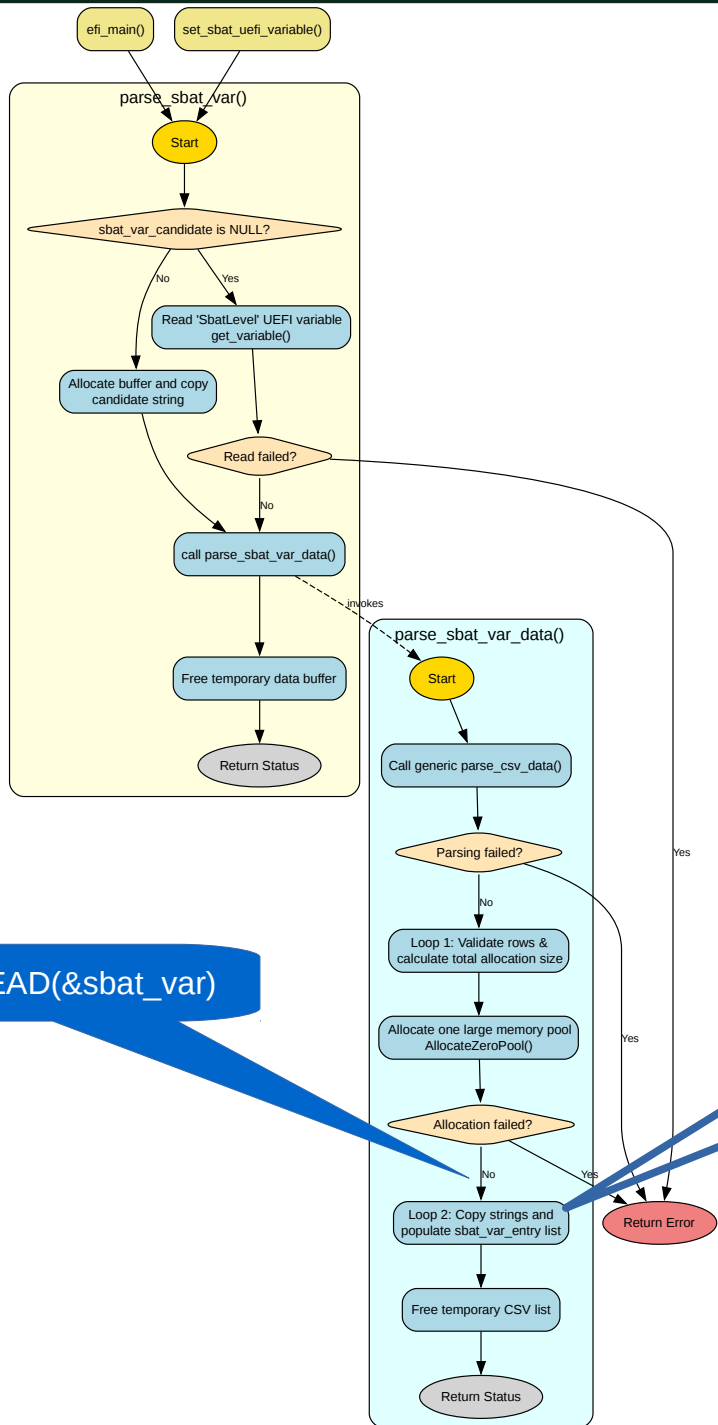
SBAT self-check

Flow Chart from Gemini scans parse_sbat_var()



Flow Chart from Gemini scans parse_sbat_var()

INIT_LIST_HEAD(&sbat_var)

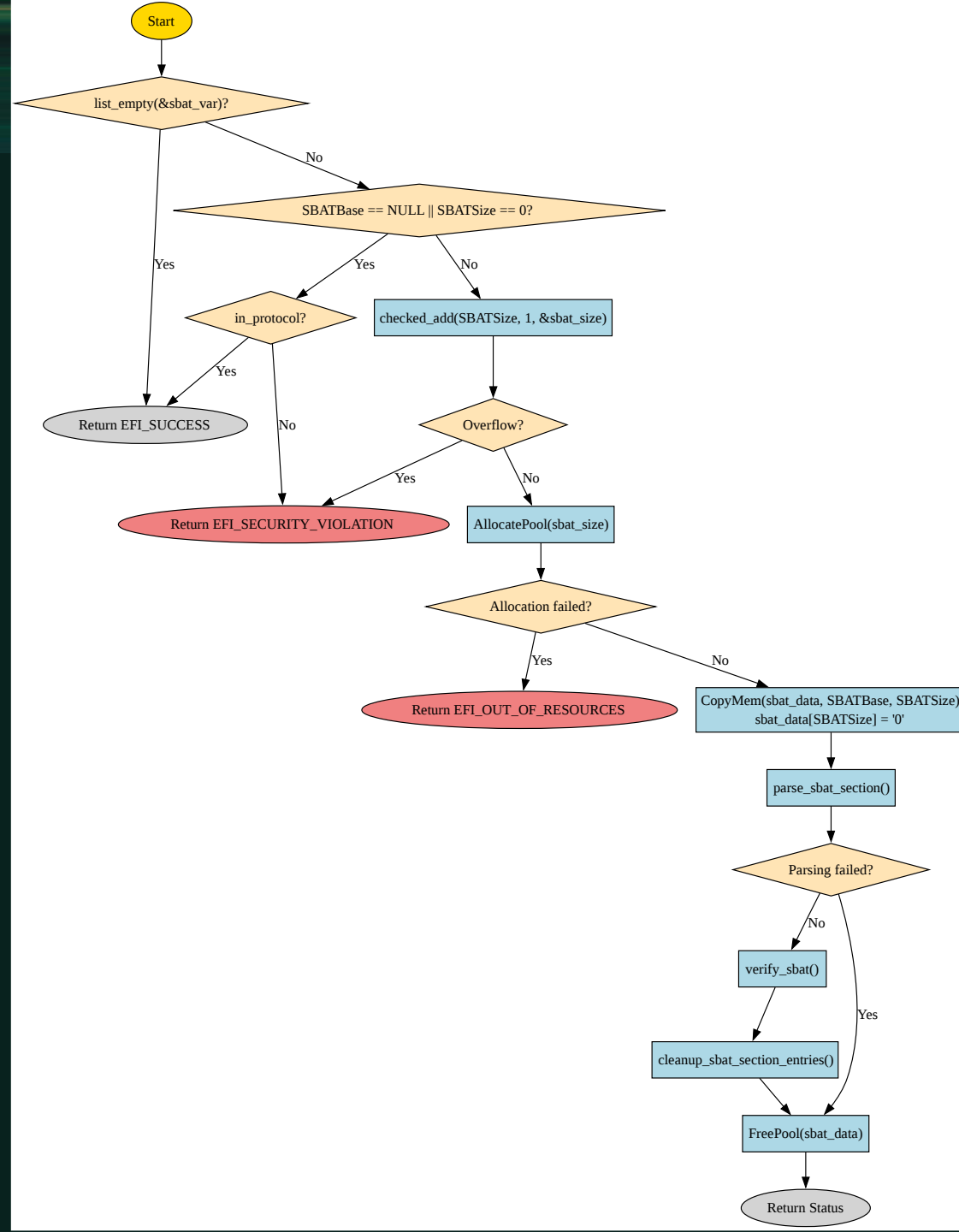


SBAT cvs string example:

```
sbat,1,2022111500\nshim,2\ngrub,3\n\0
```

3 rows
have 3 columns, 2 columns, 2 columns

Flow Chart from Gemini scans verify_sbat_section()



Flow Chart from Gemini scans verify_sbat_section()

- grub2 be loaded by shim
- sbat is mandatory
- Linux kernel loaded by shim protocol
- sbat is NOT mandatory

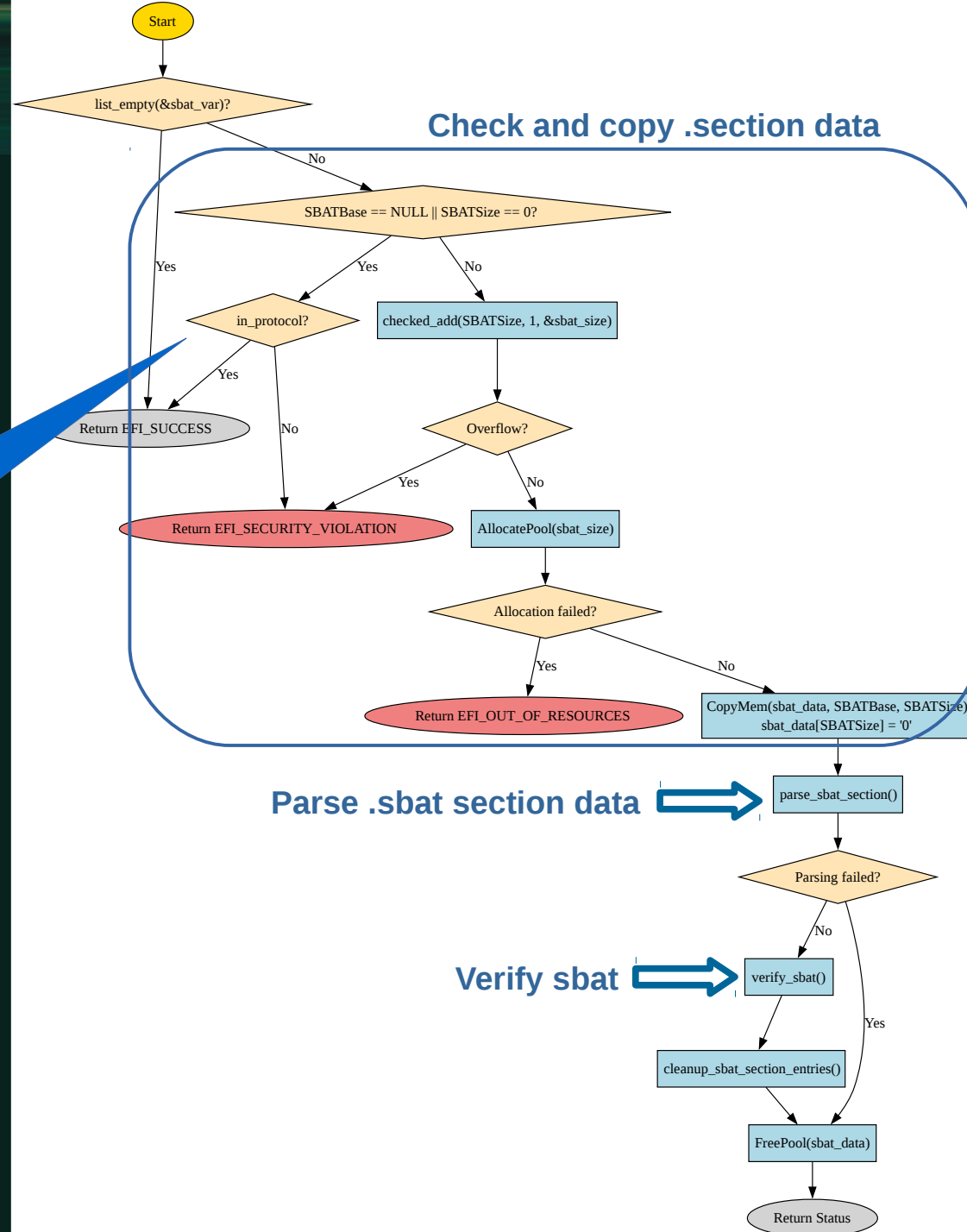
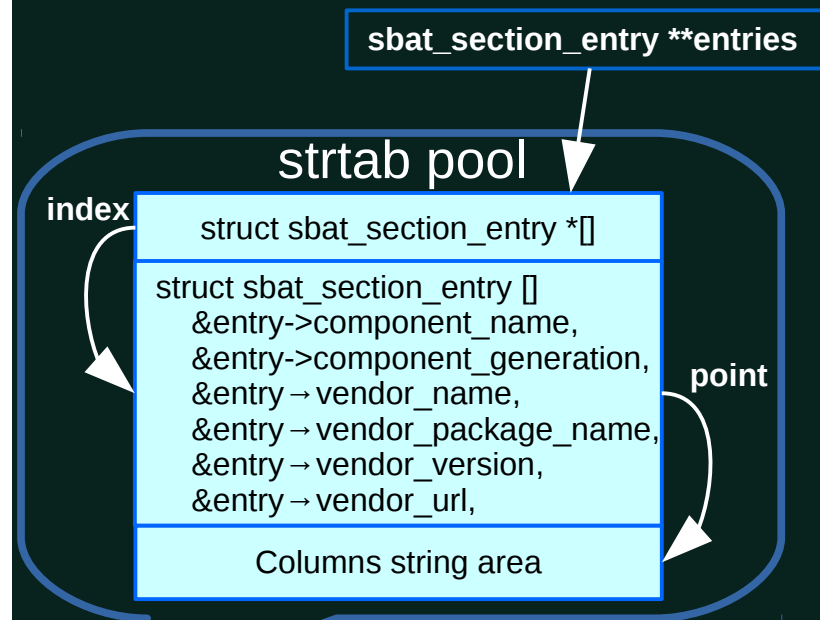
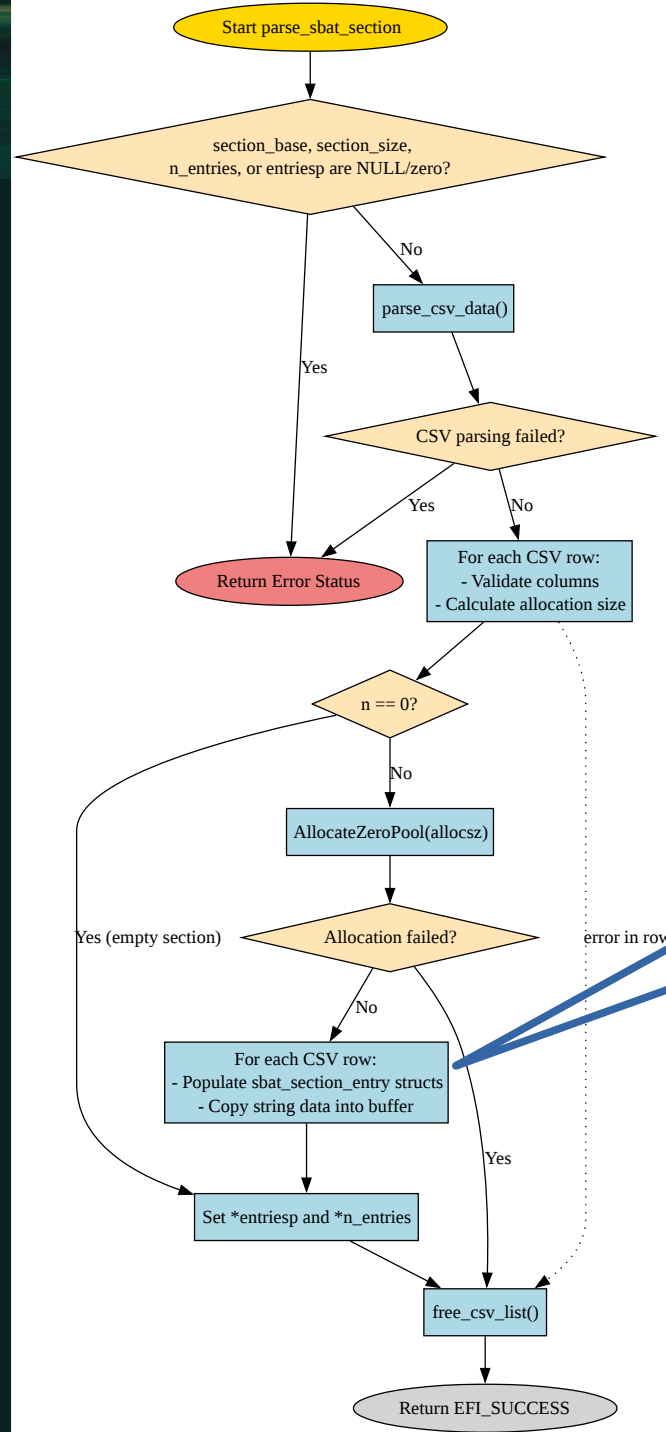


Image generated by Gemini AI

Flow Chart from Gemini scans parse_sbat_section()

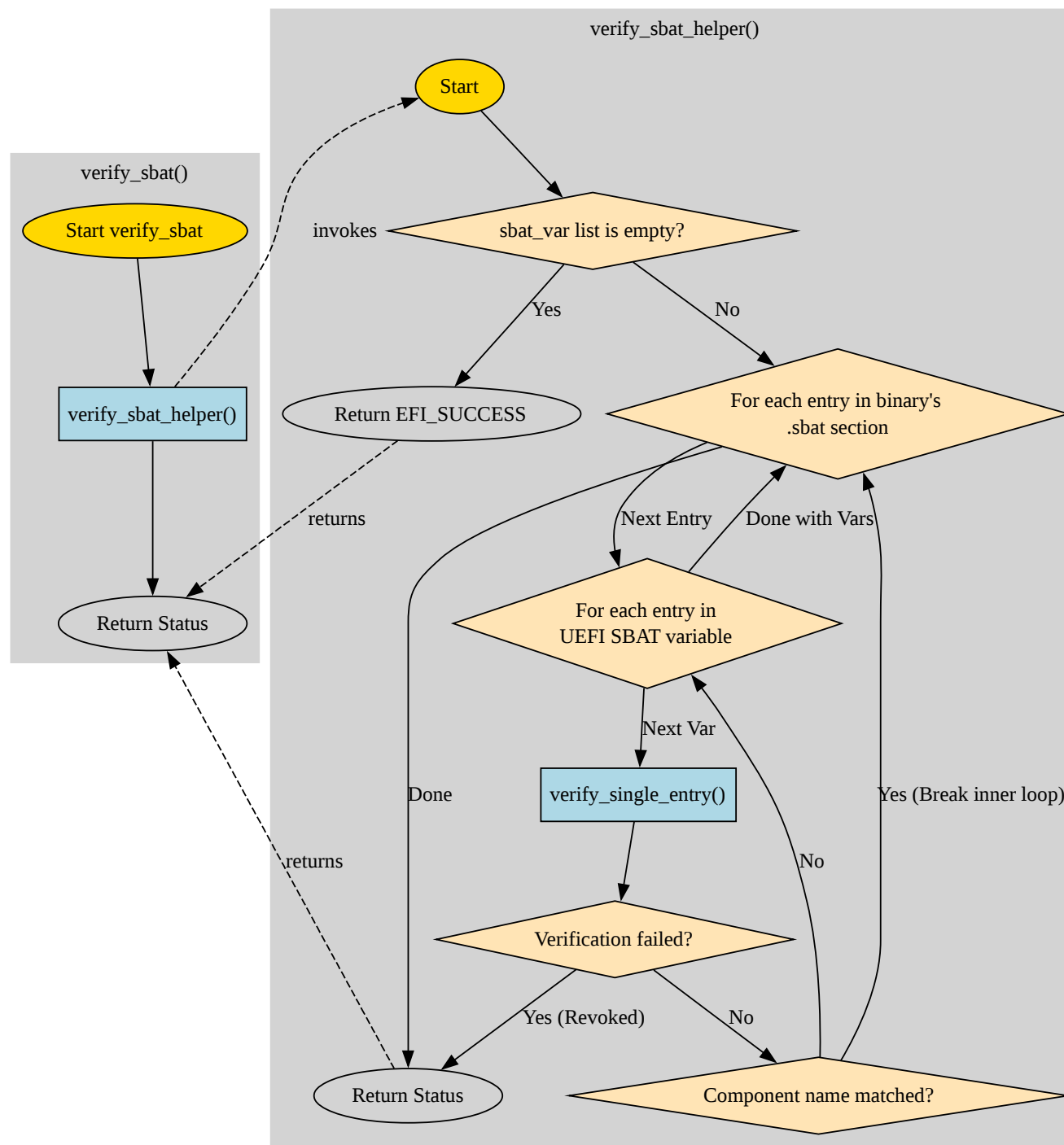


SBAT sectoin (.sbat) cvs string example:

shim,1,UEFI shim,shim,1,https://github.com/rhboot/shim\n

1 row 6 columns

Flow Chart from Gemini scans verify_sbat()



Flow Chart from Gemini scans verify_sbat()

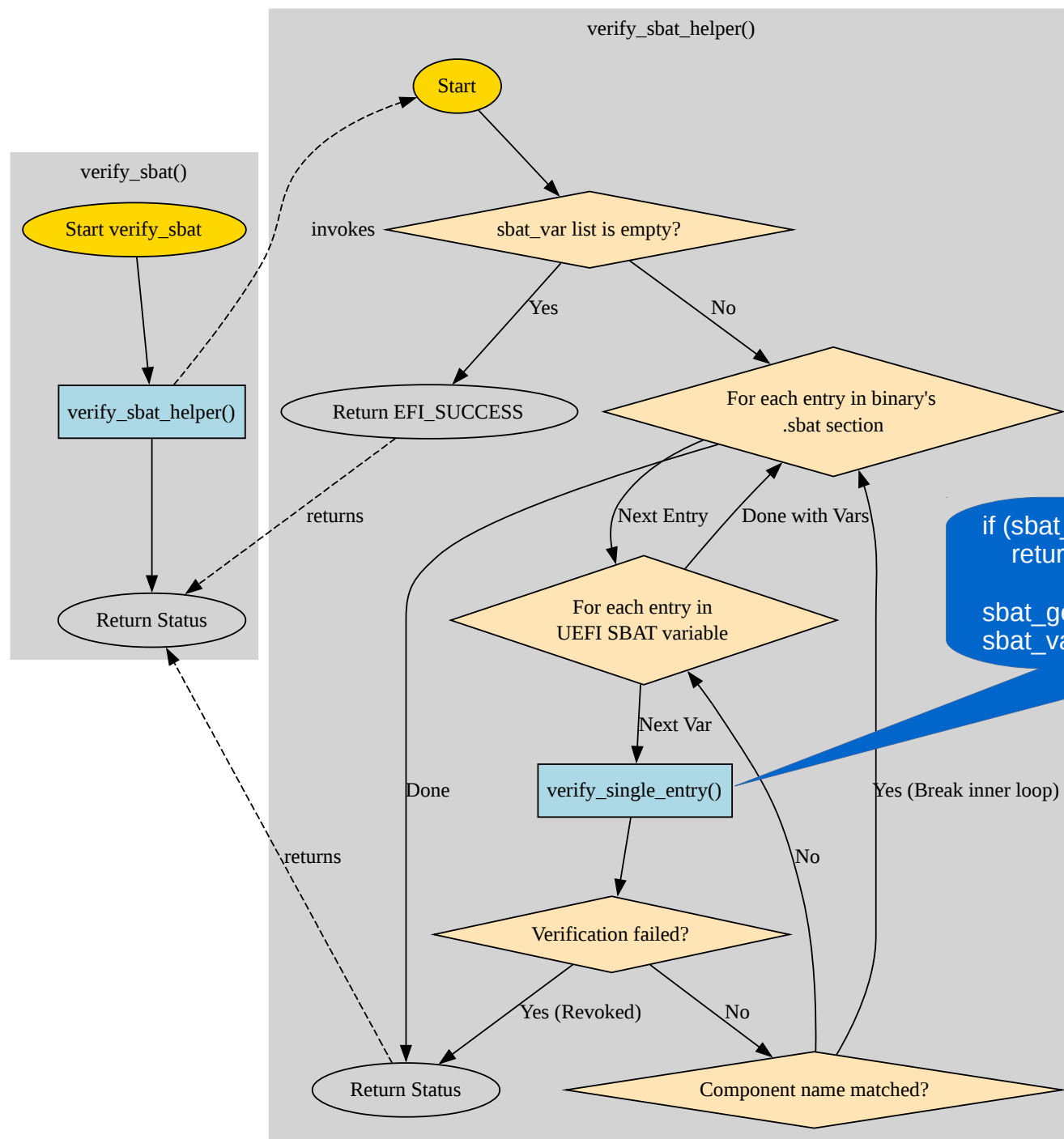


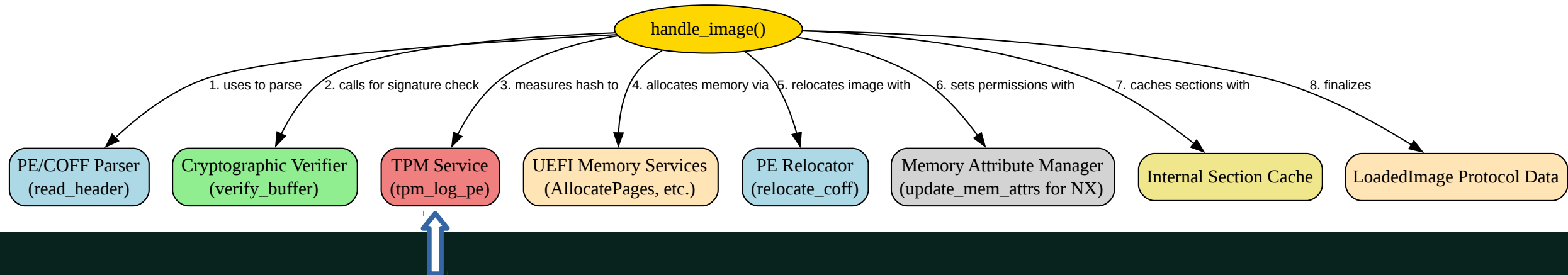
Image generated by Gemini AI

TPM



TPMinhandle_image()

- Prompt: Please generate a block diagram. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the handle_image() function and its interactions."



TPM in handle_image()

README.tpm

PCR4:

- the Authenticode hash of the binary being loaded will be extended into PCR4 before SB verification.
- the hash of any binary for which Verify is called through the shim_lock protocol

shiim, grub2, kernel

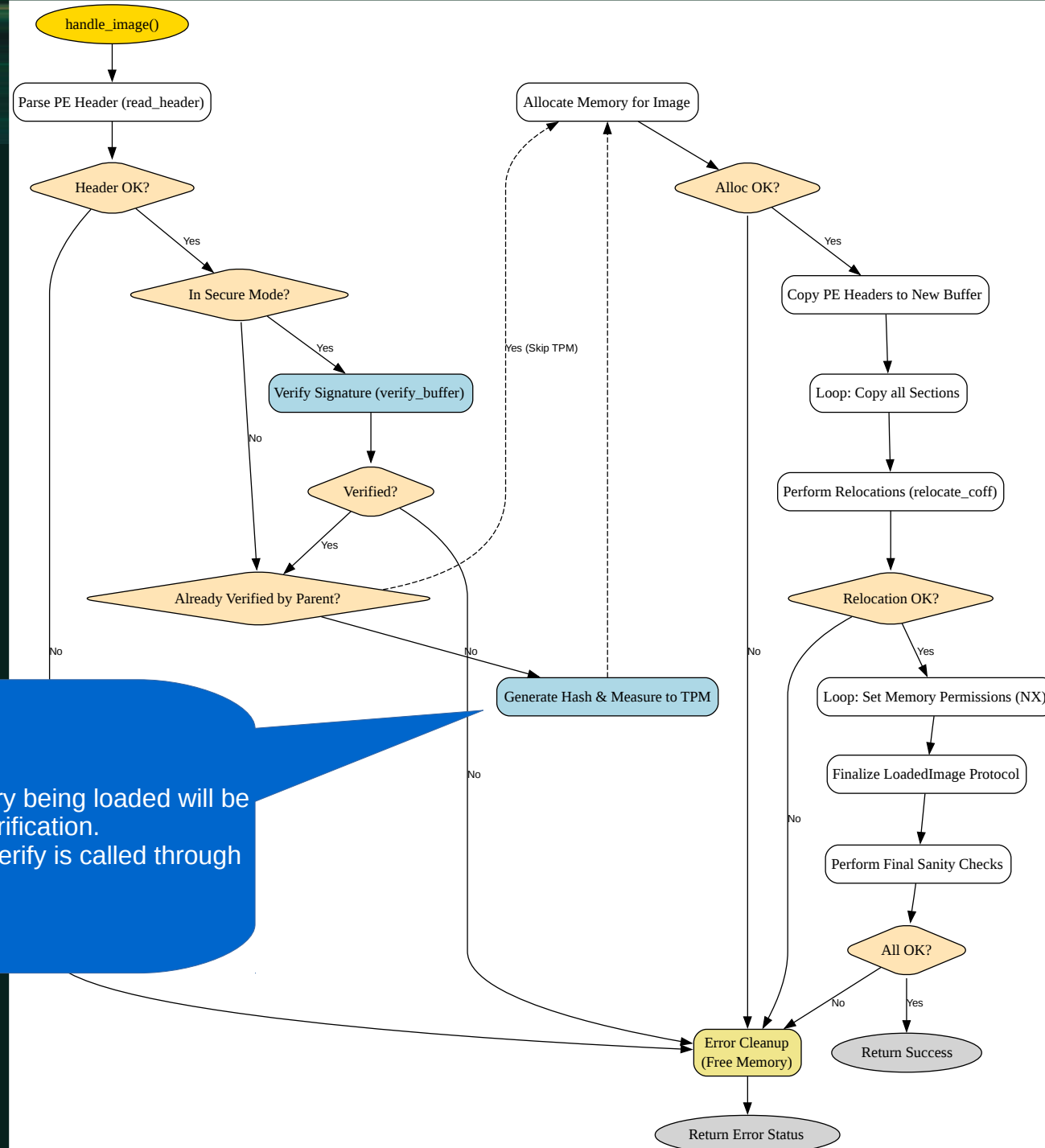
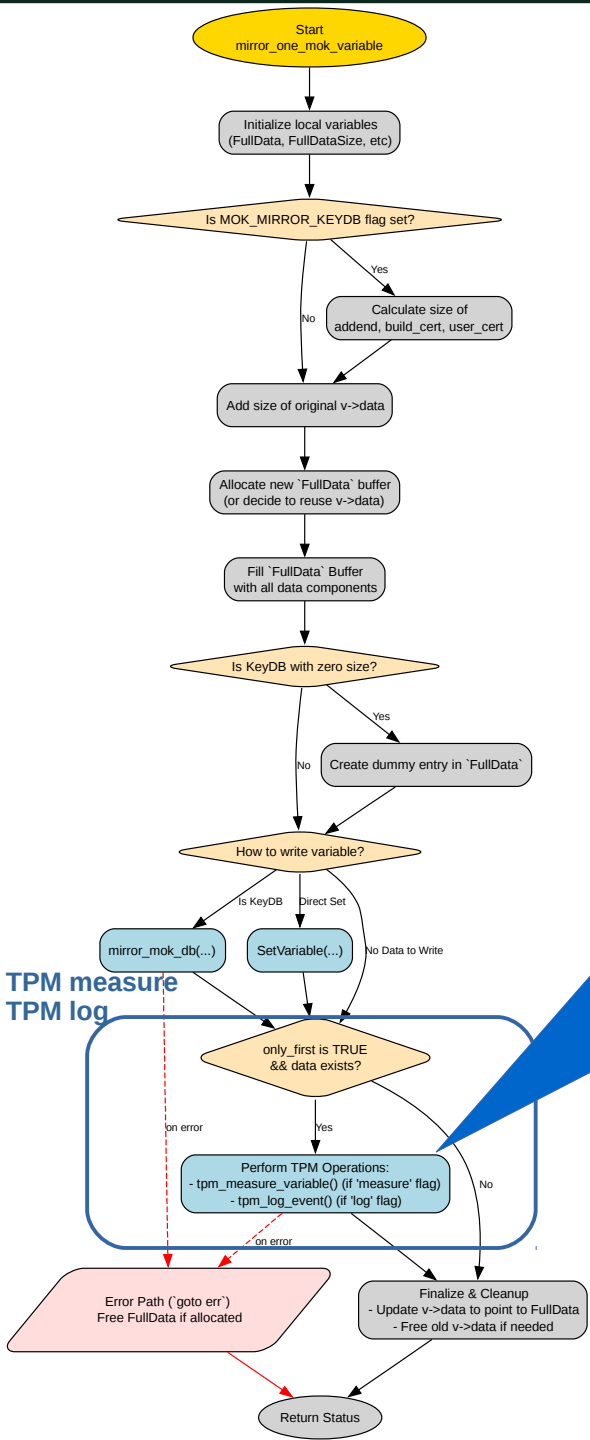


Image generated by Gemini AI

TPM in mirror_one_mok_v variable()

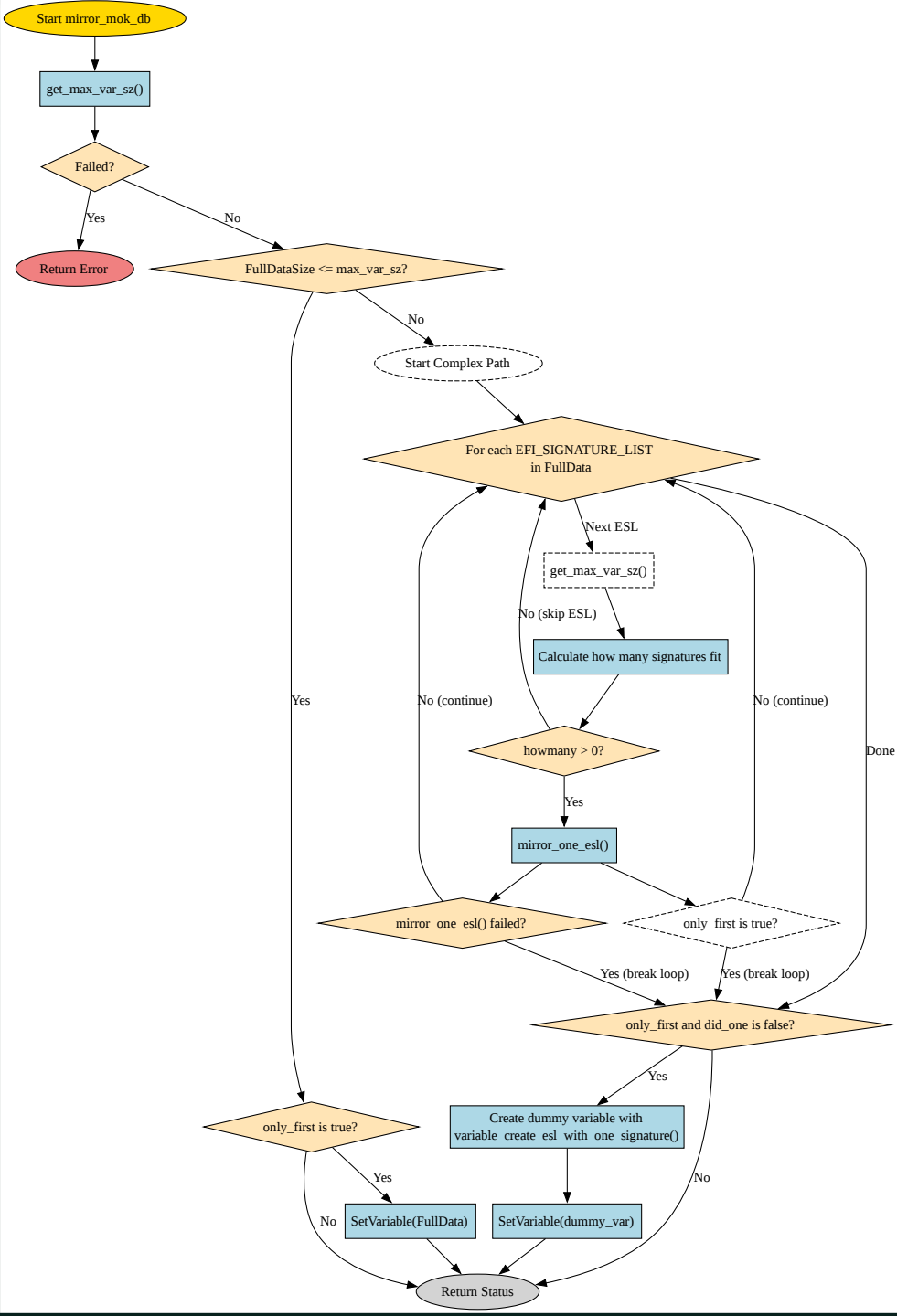


`mok_state_variable_data[]`

MokList	measure,log	pcr 14
MokListX	measure,log	pcr 14
MokSBState	measure,log	pcr 7, pcr 14
SbatLevel	measure,log	pcr 7
MokListTrusted	measure,log	pcr 14
MokPolicy	measure,log	pcr 14

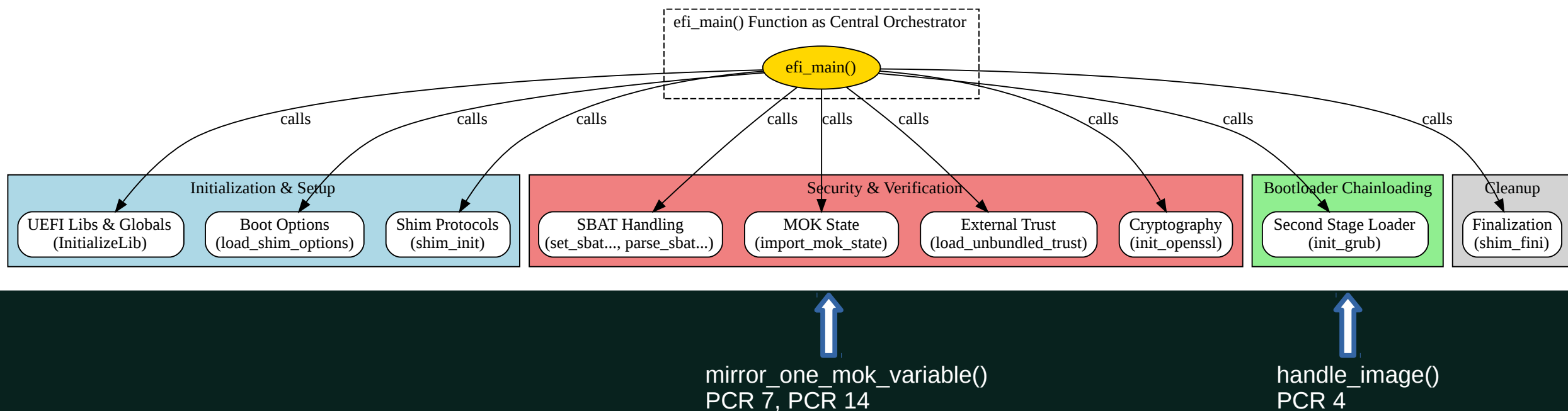
Note:
`tpm_measure_variable()`: measure and log to pcr 7
`tpm_log_event()`: measure and log to .pcr

Flow Chart from Gemini scans tpm_log_event_raw()



TPM in efi_main()

- Prompt: Please generate a **block diagram**. To do this, first perform your own analysis of the shim git repository located at ~/tmp/gemini/. Then, create the block diagram with a focus on the **efi_main()** function and its interactions."



References

- [1] shim git repo

<https://github.com/rhboot/shim>



Thank you

© SUSE LLC. All Rights Reserved. SUSE and the SUSE logo are registered trademarks of SUSE LLC in the United States and other countries. All third-party trademarks are the property of their respective owners.

For more information, contact SUSE at:

+1 800 796 3700 (U.S./Canada)

Frankenstrasse 146

90461 Nürnberg

www.suse.com